arXiv:2606.27154v2 [cs.AI] 30 Jun 2026

OpenRCA 2.0: From Outcome Labels to Causal Process Supervision

Aoyang Fang **Yifan Yang** **Jin'ao Shang*** **Qisheng Lu** **Junjielung Xu**
Rui Wang Songhan Zhang Yuzhong Zhang Boxi Yu¶ Pinjia He‡*
The Chinese University of Hong Kong, Shenzhen
aoyangfang, yifanyang6, qishenglu, junjielongxu, 224040299, 222010549, yuzhongzhang@link.cuhk.edu.cn
*jinao_s@stu.xjtu.edu.cn ¶boxi.yu@lero.ie ‡hepinjia@cuhk.edu.cn

Abstract

Root cause analysis (RCA) poses a holistic test of LLM agentic capabilities, such as long-context understanding, multi-step reasoning, and tool use. However, existing datasets suffer from a fundamental gap: they label only the root cause, not the propagation path connecting it to the observed symptom, which largely simplifies the task to naive pattern matching. To support rigorous evaluation, we introduce PAVE, a step-wise labeling protocol that leverages known interventions from fault injection to reconstruct causal propagation paths. The mechanism is *forward verification*: reasoning from cause to effect rather than inferring backward from symptoms. Applying PAVE yields OpenRCA 2.0 (500 instances), the first cross-system RCA benchmark with step-wise causal annotations for LLM agents. Across 11 frontier LLMs, recovering the exact root-cause set succeeds in only 20.7% of cases on average. To locate where this difficulty lies, we relax the criterion and find what we call the *ungrounded diagnosis*: agents identify at least one correct root-cause service in 76.0% of cases but ground that service in a verified causal propagation path to the observed symptom in only 61.5%. Outcome-only evaluation hides this failure mode; step-wise causal ground truth is the missing piece for trustworthy LLM-based RCA agents.

1 Introduction

Root cause analysis (RCA), pinpointing which component caused a failure in a large software system, has emerged as a demanding testbed for LLM agents [1–4]. The originating fault is rarely where symptoms first appear: modern cloud applications are split into dozens of small components communicating over the network [5], so a local fault can cascade through callers and reach distant parts of the system before any human notices. From this cascade, the agent receives heterogeneous telemetry: request traces showing how each request propagated, time-series metrics, and free-form logs. It must reason backward from this telemetry to the originating fault.

To evaluate such reasoning, all widely used RCA benchmarks build labeled data through *fault injection*: the procedure deliberately breaks a known component (e.g., throttling its CPU, delaying its network, crashing its process), records the resulting telemetry, and treats the broken component as the ground-truth label [6, 7, 2]. Evaluation then scores only the outcome: whether the agent identified the correct component (Figure 1). The derivation that produced this answer is not examined.

Scoring the derivation rather than just the outcome reveals what we call the *ungrounded diagnosis*: the agent identifies a correct root-cause service without a verified causal propagation path linking it to the observed failure. Detecting it at scale requires step-wise causal ground truth, but building it runs

*‡Corresponding author.

arXiv:2606.27154v2 [cs.AI] 30 Jun 2026

OpenRCA 2.0：从结果层标签到因果过程监督

方傲阳 杨一凡 尚金傲* 陆启胜 徐俊杰 王锐 张松翰 张宇忠 于博西¶ PinjiaHe‡*香港中文大学（深圳） aoyangfang, yifanyang6, qishenglu, junjielongxu, 224040299, 222010549, yuzhongzhang@link.cuhk.edu.cn
*jinao_s@stu.xjtu.edu.cn¶boxi.yu@lero.ie ‡hepinjia@cuhk.edu.cn

摘要

根因分析（RCA）对大语言模型智能体能力构成了全面测试，涉及长上下文理解、多步推理和工具使用。然而，现有数据集存在一个根本性缺口：它们仅标注根本原因，而未标注将其与观测症状连接起来的因果传播路径，这在很大程度上将任务简化为朴素的模式匹配。为支持严格评估，我们引入了 PAVE，一种逐步标注协议，利用故障注入中的已知干预来重构因果传播路径。其核心机制是前向验证：从原因向结果进行推理，而非从症状反向推断。应用 PAVE 产生了 OpenRCA 2.0（包含 500 个实例），这是首个具备面向大语言模型智能体的逐步因果标注的跨系统根因分析基准。在 11 个前沿大语言模型中，平均仅在 20.7% 的案例中成功恢复出精确的根因集。为了定位这一难度的根源，我们放宽了评估标准，发现了所谓的无依据诊断现象：智能体在 76.0% 的案例中至少识别出一个正确的根因服务，但仅在 61.5% 的案例中将该服务锚定到通往观测症状的已验证因果传播路径上。仅结果评估掩盖了这种失败模式；逐步因果真值是构建可信的基于大语言模型的 RCA 智能体所缺失的关键环节。

1 引言

根因分析（RCA），即定位大型软件系统中哪个组件导致了故障，已成为大语言模型智能体 [1–4] 的一项严苛测试平台。源故障很少出现在症状首次显现的位置：现代云应用程序被拆分为数十个通过网络通信的小型组件 [5],，因此局部故障可能会通过调用者产生级联效应，并在任何人类察觉之前波及系统的遥远部分。从这种级联效应中，智能体接收异构遥测数据：显示每个请求如何传播的请求追踪、时间序列指标以及自由格式日志。它必须基于这些遥测数据进行反向推理，以找到源故障。

为了评估这种推理，所有广泛使用的RCA基准测试均通过故障注入构建标注数据：该过程故意破坏一个已知组件（例如，对其CPU进行限制、对其网络增加延迟、使其进程崩溃），记录由此产生的遥测数据，并将被破坏的组件视为真值标签 [6, 7, 2]。评估随后仅针对结果层进行评分：即代理是否识别出了正确的组件（图1）。而得出该答案的推导过程则未受检验。

仅对结果进行评分，而对推导过程进行评分，则揭示了我们所称的无依据诊断：代理正确识别了根因服务，但缺乏将根因服务与观测故障联系起来的已验证因果传播路径。在大规模情况下检测无依据诊断需要逐步因果真值，但构建逐步因果真值却

*‡通讯作者。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

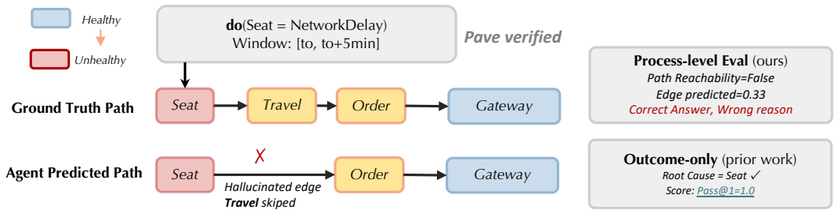


Figure 1: An *ungrounded diagnosis* on the running NetworkDelay failure in Seat service. From the known intervention (top), PAVE reconstructs the verified causal path Seat→Travel→Order→Gateway (middle). The agent names the correct root cause but produces a graph that skips Travel (bottom): outcome-only evaluation scores this as a success, whereas process-level evaluation surfaces the missing edge.

into the agent's problem: from telemetry alone, an annotator would still have to reason backward from cascading effects to the originating fault, since many faults can produce similar telemetry. But fault-injection records hand the annotator one thing the agent never sees: the intervention that started the cascade [8]. This information asymmetry [9] turns the annotator's task from inference into *forward verification*: with the cause given, check which downstream effects actually followed.

We turn this insight into PAVE (Path Annotation via Verified Effects), an annotation protocol that reconstructs verified causal propagation paths from fault-injection recordings. No single signal is sufficient: a structurally plausible chain may not actually have carried the failure, a statistical co-occurrence may be coincidental, and aligned timing alone does not imply a causal link. PAVE therefore admits a candidate path only when it satisfies all three conditions jointly: conformance to known propagation mechanisms (structural), statistically significant deviation from a pre-injection baseline (statistical), and upstream-to-downstream timing alignment (temporal). Admitted paths thus carry consistent structural, statistical, and temporal evidence (Section 2.3).

Applying PAVE across 3 architecturally distinct microservice systems (TrainTicket, the OpenTelemetry Demo, and DeathStarBench Hotel Reservation) and 27 fault types yields OpenRCA 2.0, the first cross-system RCA benchmark with step-wise causal annotations: 500 evaluable instances carrying 7.5 verified causal edges on average.

On OpenRCA 2.0, the gap between identifying a correct service and grounding it in a verifiable path is substantial across frontier LLMs. Averaged across 11 such models, agents recover the exact root-cause set in only 20.7% of cases. To locate where this difficulty lies, we relax the criterion: identifying at least one correct root-cause service rises to 76.0%, but grounding that service in a verified causal propagation path to the observed symptom is satisfied in only 61.5%, leaving roughly one in five correct identifications ungrounded. Outcome-only evaluation hides this failure mode; step-wise causal ground truth is the missing piece for trustworthy LLM-based RCA agents.

2 PAVE: Path Annotation via Verified Effects

A microservice system runs as a directed dependency graph over services, pods, and hosts (Figure 2a), and a fault at one node propagates along two distinct channels: horizontally along RPC call chains, and vertically across shared infrastructure. The diagnosing agent sees only the telemetry these channels leave behind (traces, metrics, and logs; Figure 2b) and must reason backward to the originating fault.

We now describe the PAVE pipeline (Figure 3). Given fault injection records and the system dependency graph, it reconstructs verified causal propagation paths in two phases that together enforce three conditions on every admitted path. *Structural Pruning* (Section 2.2) enumerates candidate paths consistent with the topology and a propagation rule set, enforcing *mechanism conformance*. *Causal Verification* (Section 2.3) then checks each candidate against pre-injection telemetry, enforcing *statistical deviation* and *timing alignment* jointly.

For each injection, PAVE consumes three inputs (the dependency graph $\mathcal{G}$, raw telemetry $\mathbf{O}^{(t)}$ over the injection window, and the recorded intervention $do(v_{root})$) and emits one output: a verified path

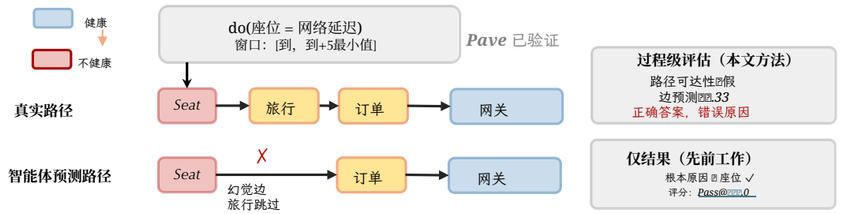


图1：针对座位服务中正在运行的网络延迟故障的无依据诊断。根据已知干预（顶部），PAVE重构了经过验证的因果路径座位→旅行→订单→网关（中部）。代理正确指出了根本原因，但生成的图跳过了旅行（底部）：仅结果评估将此计为成功，而进程级别评估则揭示了缺失的边。

这将转化为代理面临的一个问题：仅凭遥测数据，标注员仍然必须从级联效应反向推理到源故障，因为许多故障会产生相似的遥测数据。但故障注入记录提供了代理永远无法看到的一项信息：引发级联效应的干预 [8]。这种信息不对称 [9] 将标注员的任务从推断转变为前向验证：在已知原因的情况下，检查哪些下游效应实际随之发生。

我们将这一洞见转化为 PAVE（基于已验证效应的路径标注），一种从故障注入记录中重构已验证因果传播路径的标注协议。单一信号不足以完成验证：结构上合理的链可能并未实际传导故障，统计上的共现可能是巧合，仅凭对齐的时间也不能暗示因果关系。因此，PAVE 仅在候选路径同时满足以下三个条件时才予以认可：符合已知传播机制（结构性的）、相对于注入前基线具有统计显著性的偏差（统计的）以及上下游时序对齐（时间的）。被认可路径因此具备一致的结构、统计和时间证据（第 2.3 节）。

将PAVE应用于3个架构各异的微服务系统（TrainTicket、OpenTelemetry Demo和DeathStarBench酒店预订）以及27种故障类型，得到了OpenRCA 2.0，这是首个具备逐步因果标注的跨系统根因分析基准：包含500个可评估实例，平均每个实例携带7.5条已验证因果边。

在 OpenRCA 2.0 上，前沿大语言模型在识别正确服务与将其锚定在可验证路径之间均存在显著差距。在 11 个此类模型的聚合结果中，智能体仅在 20.7% 的案例中恢复了确切的根因集。为了定位这一困难所在，我们放宽了评判标准：识别出至少一个正确根因服务的比例上升至 76.0%，但将该服务锚定在到观测症状的已验证因果传播路径上的比例仅为 61.5%，这意味着大约五分之一正确识别的案例缺乏锚定。仅结果评估掩盖了这种失效模式；逐步因果真值是构建可信基于大语言模型的 RCA 代理所缺失的关键环节。

2 PAVE：基于已验证效应的路径标注

微服务系统以基于服务、Pod和主机的有向依赖图形式运行（图2a），其中一个节点上的故障会沿两个不同通道传播：沿RPC调用链水平传播，以及跨共享基础设施垂直传播。诊断代理仅能观察到这些通道留下的遥测数据（追踪、指标和日志；图2b），并且必须逆向推理以定位源故障。

现在我们介绍PAVE流水线（图3）。给定故障注入记录和系统依赖图，它通过两个阶段重建已验证因果传播路径，这两个阶段共同对每条被认可路径强制执行三个条件。结构剪枝（第 2.2 节）枚举符合拓扑和传播规则集的候选路径，强制执行机制一致性。因果验证（第 2.3 节）随后对照注入前遥测数据检查每个候选路径，共同强制执行统计偏差和时间对齐。

对于每次注入，PAVE 消费三个输入（依赖图 $\mathcal{G}$、注入窗口上的原始遥测数据 $\mathbf{O}^{(t)}$，以及记录的干预 $do(v_{root})$），并产生一个输出：验证路径

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

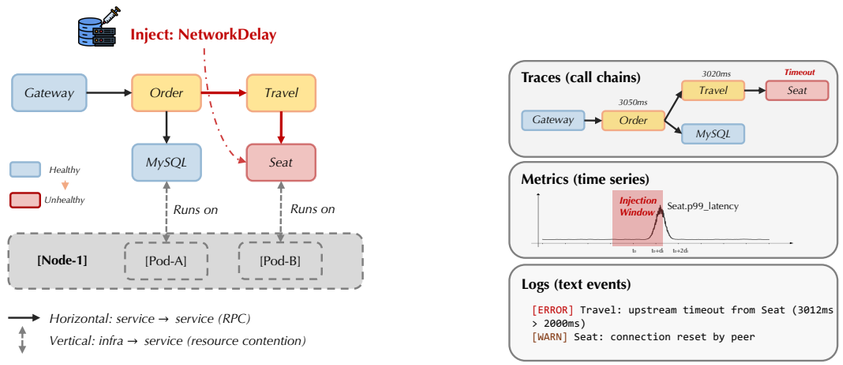


Figure 2: The RCA setting. **(a)** A microservice system forms a two-layer dependency graph. A fault injected at Seat cascades along the RPC chain (Seat → Travel → Order) and may also propagate vertically through shared infrastructure (e.g., co-located pods). **(b)** The agent observes only telemetry (traces, metrics, logs) and must reason backward to the originating fault; the intervention itself is hidden.

set Π^*. Phase 1 builds an intermediate candidate set Π_{cand} from topology and the rule set; Phase 2 yields $\Pi^* \subseteq \Pi_{cand}$ by verifying survivors on telemetry. Each injection in OpenRCA 2.0 carries Π^* as its step-wise annotation.

2.1 Setup: Forward Verification from a Known Intervention

We model a microservice system as a directed dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over services, pods, and hosts (Figure 2a), with edges encoding invocation or resource dependencies (channels and modalities in Appendix B.5). Fault propagation is a structural causal process over $\mathcal{G}$ [8]: anomalies propagate along edges with finite delay (full equation in Eq. (2), Appendix D.1). An RCA agent observes only telemetry $\mathbf{O}_{obs}$ and must jointly infer the dependency structure, the root cause, and the propagation path: an inherently *backward* abductive problem. PAVE, by contrast, has access to the intervention $do(v_{root})$ (target component, fault type, injection window $[t_0, t_0{+}\Delta t]$, and parameters), giving it a strictly richer information set:

$$\underbrace{\{\mathbf{O}_{obs}\}}_{\text{Agent}} \subset \underbrace{\{\mathbf{O}_{obs},\, do(v_{root})\}}_{\text{PAVE}} \tag{1}$$

Knowing the intervention turns an ill-posed inverse problem into a well-posed *forward* verification task: confirming that observed anomalies are consistent with the known intervention propagating through $\mathcal{G}$. The verified causal subgraph $\mathcal{G}^*$ (formed by the edges of all paths in Π^*) is the process-level ground truth (GT) against which agent reasoning is evaluated.

2.2 Phase 1: Structural Pruning

Phase 1 produces a candidate path set Π_{cand} by enforcing topological constraints from the system dependency graph. Raw telemetry $\mathbf{O}^{(t)}$ is first projected onto a discrete state alphabet Σ via a node-type-aware mapping Ψ (e.g., `HighLatency` for service nodes, `CPU_Throttled` for pods; formal definition in Appendix D.2). A propagation rule set $\mathcal{R}$ admits only transitions between (type, state) pairs $(\theta_i{:}s_i) \xrightarrow{\text{channel}} (\theta_j{:}s_j)$ backed by a known mechanism (resource contention, RPC timeout cascade, etc.). A constrained DFS over a time-expanded heterogeneous graph, whose nodes are (component, state, time) triples (v, s, t), unrolls circular dependencies into acyclic chains, yielding Π_{cand}. The state alphabet, full rule table, and time-expansion details are in Appendix D.

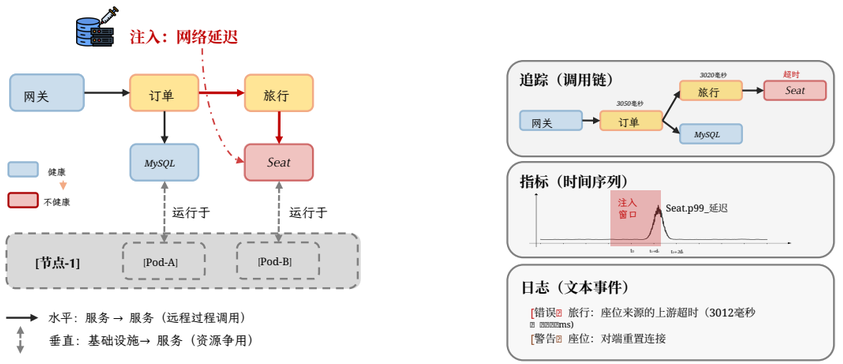


图2：根因分析设置。(a) 微服务系统构成两层依赖图。在Seat（座位）注入的故障沿RPC链（Seat → Travel（旅行） → Order（订单））产生级联故障，并可能通过共享基础设施（例如，共置Pod）垂直传播。(b) 代理仅观察遥测数据（追踪、指标、日志），必须逆向推理至源故障；干预本身是隐藏的。

集合 Π^*。第一阶段根据拓扑和规则集构建中间候选集 Π_{cand}；第二阶段通过验证遥测数据上的幸存项得到 $\Pi^* \subseteq \Pi_{cand}$。OpenRCA 2.0中的每次注入都携带 Π^*作为其逐步标注。

2.1 设置：基于已知干预的前向验证

我们将 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$建模为基于服务、Pod和主机的有向依赖图（图2a），其中边编码了调用或资源依赖关系（附录B.5中的通道和模态）。故障传播是 $\mathcal{G}$ [8]上的一种结构性因果过程：异常沿着边以有限延迟传播（完整方程见公式(2)，附录D.1）。根因分析代理仅能观察到遥测数据$\mathbf{O}_{obs}$，并且必须联合推断依赖结构、根本原因和传播路径：这是一个固有的逆向溯因问题。相比之下，PAVE可以访问干预$do(v_{root})$（目标组件、故障类型、注入窗口$[t_0, t_0{+}\Delta t]$和参数），从而拥有严格更丰富的信息集：

$$\underbrace{\{\mathbf{O}_{obs}\}}_{\text{Agent}} \subset \underbrace{\{\mathbf{O}_{obs},\, do(v_{root})\}}_{\text{PAVE}} \tag{1}$$

了解干预将病态反问题转化为适定正向验证任务：确认观察到的异常与已知干预通过 $\mathcal{G}$传播的情况一致。验证因果子图 $\mathcal{G}^*$（由 Π^*中所有路径的边组成）是进程级别的真实值（GT），用于评估代理的推理。

2.2 阶段1：结构剪枝

第一阶段通过施加系统依赖图中的拓扑约束生成候选路径集 Π_{cand}。原始遥测数据$\mathbf{O}^{(t)}$ 首先通过节点类型感知映射 Ψ 投影到离散状态字母表 Σ 上（例如，服务节点对应高延迟，Pod对应CPU_节流；正式定义见附录D.2）。传播规则集 $\mathcal{R}$ 仅允许由已知机制（如资源争用、RPC超时级联等）支持的（类型，状态）对之间的转换（$\theta_i{:}s_i$）通道 ⟶（$\theta_j{:}s_j$）。在时间扩展异构图上执行约束深度优先搜索，该图的节点为（组件，状态，时间）三元组（v，s，t），将循环依赖展开为无环链，从而得到Π_{cand}。状态字母表、完整规则表以及时间扩展细节见附录D。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

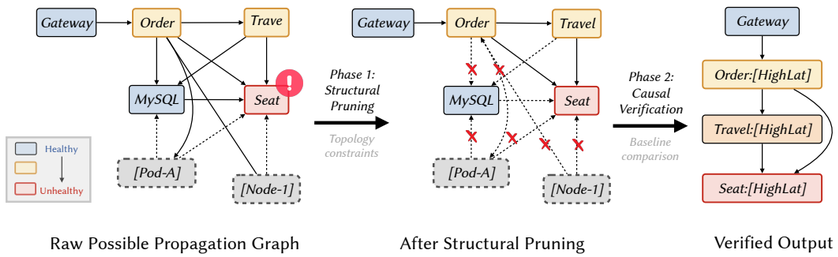


Figure 3: **Coarse-to-Fine verification refines a cluttered observation graph into one verified causal chain.** Running TrainTicket example: the true root cause is *Seat*, while *Route* is a benign service that appears anomalous due to background noise. *Input.* Every observed anomaly and every potential edge are kept, so the root cause cannot be picked out. *Phase 1 (Structural Pruning).* Edges marked $\times$ are removed because the topology admits no such dependency (e.g., *Route*→*Seat*); surviving solid edges are structurally possible, but *Route* itself is still flagged. *Phase 2 (Causal Verification).* Each candidate is scored against a pre-injection baseline; *Route*'s deviation matches baseline noise and is dropped, leaving the chain *Seat*→*Travel*→*Order*. Phase 1 asks "*could this edge happen?*" (topology); Phase 2 asks "*did this anomaly carry the signal?*" (baseline contrast); injections with no surviving chain are discarded, and the released artifact is the union of such chains.

2.3 Phase 2: Causal Verification

Phase 2 produces the verified path set Π^* by separating genuine propagation from coincidental disturbances (e.g., periodic cron jobs) that co-occur with the injection window. For each injection, we collect normal-period telemetry on a fresh, warmed-up instance as a *Reference Baseline* ($\mathcal{D}_{base}$) immediately before injection, approximating $P(\cdot \mid do(v_{root}{=}\emptyset))$. For each node v_i along a candidate path $\pi \in \Pi_{cand}$, we apply distributional tests tailored to each metric type (Z-score for Gaussian metrics, percentile-based tests for heavy-tailed metrics; Appendix D.8). A candidate path π is *verified* only if *all* of (i) every node along π exhibits a deviation beyond its baseline screen, (ii) every consecutive edge conforms to $\mathcal{R}$ (inherited from Phase 1), and (iii) each downstream anomaly onsets no earlier than its upstream cause hold simultaneously.

Phase 2 admits an injection into the released artifact only when its cascade clears the joint screen end-to-end: the fault must produce an observable downstream signature (a deviation crossing the configured SLO surface), at least one rule-admitted, temporally consistent path must connect the intervention to that signature, and the pre-injection baseline must be free of drift. Cases failing any of these conditions are dropped during pipeline-level analysis.

3 Experiments

Process-level ground truth reveals a reasoning gap in LLM-based RCA agents that outcome-only evaluation hides: across 11 frontier LLMs, correct root-cause identifications are routinely not grounded in any verified causal propagation path.

3.1 Experimental Setup

This subsection specifies the three microservice systems we run, the fault-injection campaigns that produce the raw pool, the curation that yields OpenRCA 2.0, and the agent design under evaluation.

Systems and Faults. We evaluate on 3 architecturally distinct microservice systems: *TrainTicket* [5], *Hotel Reservation* from DeathStarBench [10], and the *OpenTelemetry Demo* (OTel Demo) application [11], spanning different language stacks and synchronous/asynchronous communication patterns; deployment specifications are in Appendix B. For each system we run fault-injection workloads covering 27 chaos primary kinds across application, resource, network, and pod-level faults (per-system taxonomies in Appendix C.2); for TrainTicket we build on the released dataset of Fang et al. [7] and augment it with new campaigns on the other two systems. The resulting raw pool comprises 8137

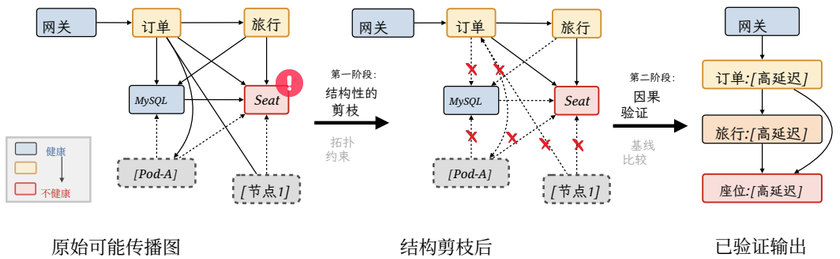


图3：粗到细验证将杂乱的观测图精炼为一条经过验证的因果链。以运行中的TrainTicket为例：真实根本原因是Seat，而Route是一个良性服务，由于背景噪声而显得异常。输入：保留所有观测到的异常和所有潜在边，因此无法识别出根本原因。第一阶段（结构剪枝）。标记为 $\times$ 的边被移除，因为拓扑结构不允许这种依赖关系（例如，Route→Seat）；存活的实线边在结构上是可能的，但Route本身仍被标记为异常。第二阶段（因果验证）。将每个候选项与注入前基线进行评分比较；Route的偏差与基线噪声相符，因此被丢弃，留下的因果链为Seat→Travel→Order。第一阶段询问"这条边是否可能发生？"（拓扑）；第二阶段询问"该异常是否携带了信号？"（基线对比）；没有存活因果链的注入被丢弃，最终发布的产物是这些因果链的并集。

2.3 阶段二：因果验证

第二阶段通过将与注入窗口同时出现的偶然干扰（例如，周期性Cron任务）与真实的传播现象分离，从而生成已验证路径集 Π^*。对于每一次注入，我们在注入前立即在新鲜的预热实例上收集正常时期的遥测数据，作为参考基线（$\mathcal{D}_{base}$），近似表示 $P(\cdot \mid do(v_{root}{=}\emptyset))$。对于候选路径 $\pi \in \Pi_{cand}$上的每个节点 v_i，我们应用针对每种指标类型定制的分布检验（高斯指标使用Z分数，重尾指标使用基于百分位的检验；见附录D.8）。当且仅当以下三个条件同时成立时，候选路径 π 才被视为验证路径：(i) π 上的每个节点都表现出超出其基线筛查范围的偏差，(ii) 每条连续的边都符合 $\mathcal{R}$（继承自第一阶段），以及 (iii) 每个下游异常的发生时间不早于其上游原因。

第二阶段仅当注入在端到端联合筛选中通过级联检验时，才将其纳入发布产物：故障必须产生可观测的下游特征（即跨越配置的服务级别目标边界的偏差），至少存在一条经规则认可且在时间上一致的路径，能够将干预与该特征关联起来，并且注入前基线必须无漂移。不满足上述任一条件的案例将在流水线级分析中被剔除。

3 实验

过程级真实值揭示了基于大语言模型的根因分析智能体中的推理差距，而仅结果评估则掩盖了这一差距：在 11 个前沿大语言模型中，正确的根因识别通常并未基于任何已验证的因果传播路径。

3.1 实验设置

本小节具体介绍了我们运行的三个微服务系统、产生原始数据池的故障注入活动、生成OpenRCA 2.0的数据整理过程以及待评估的智能体设计。

系统与应用。我们在 3 个架构各异的微服务系统上进行评估：TrainTicket [5],、来自 DeathStarBench 的酒店预定 [10], 以及 OpenTelemetry 演示（OTel 演示）应用 [11],，它们涵盖了不同的语言栈和同步/异步通信模式；部署规范见附录 B。对于每个系统，我们运行故障注入工作负载，覆盖涵盖应用、资源、网络和 Pod 级别故障的 27 种混沌主要类型（各系统分类法见附录 C.2）；对于 TrainTicket，我们基于 Fang 等人 [7] 发布的现有数据集，并在另外两个系统上新增故障注入战役。由此得到的原始数据池包含 8137

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 1: **OpenRCA 2.0 by system.** The three systems differ in size (9–44 services), language stack, and typical propagation depth, so the 500 instances cover distinct architectures rather than re-sampling one. Cluster configuration in Appendix D.4; raw pool sizes and silent-injection rates in Appendix A.

Property	TrainTicket	OTel Demo	Hotel Res.	Total
Services in system	44	15	9	
Implementation	Java/REST	polyglot/gRPC+Kafka	Go/gRPC	
Evaluable instances	320	38	142	500
Distinct chaos primary kinds	25	9	5	27
Mean longest propagation path	3.0	2.5	2.1	2.7
Avg. causal edges per instance	10.1	3.0	2.7	7.5

injected runs. After filtering silent injections, we apply the PAVE pipeline (Section 2) to the survivors and curate OpenRCA 2.0 (500 instances) by stratified selection on (system, chaos family, root service) with per-stratum caps and additional structural screens (full rules in Appendix A). Table 1 reports per-system breakdowns; the Datasheet is in Appendix A.

Ground Truth. Causal propagation paths are extracted automatically using the pipeline described in Section 2; their reliability is bounded by the two-annotator manual audit reported in Section 3.4.

Agent Design. All evaluated agents follow a tool-augmented ReAct-style [12] architecture, adopted from the Deep Research agent [13]. To isolate LLM reasoning capability from tooling effects, all models share an identical tool suite over the released telemetry and identical prompt templates; no dependency graph is provided. Each diagnosis is returned as structured output: root-cause claims and propagation edges, each paired to a re-executable SQL query as evidence (full schema, tool definitions, and prompts in Appendix F and F.3).

3.2 Benchmark Utility: Process-Level Evaluation of RCA Agents

This subsection grades 11 frontier LLMs against OpenRCA 2.0 along outcome and process layers and reads three patterns out of the pooled results: outcome scores understate the failure (Finding 1), process-level Edge F1 is universally lower than Node F1 (Finding 2), and no single backbone dominates (Finding 3).

Experimental Design. We evaluate the RCA agent architecture above with 11 frontier LLMs as backbones spanning closed-source and open-source families across Anthropic, Google, ByteDance, Xiaomi, Zhipu, Moonshot, DeepSeek, Alibaba, Tencent, and MiniMax: Claude Opus 4.7, Gemini 3.1 Pro, Claude Sonnet 4.6, MiMo 2.5 Pro, GLM 5.1, Kimi K2.6, DeepSeek V4 Pro, Qwen3.6-Max, Hy 3.0 Preview, Seed 2.0 Pro, and MiniMax M2.7. Each model receives multi-modal telemetry (traces, metrics, logs) and must diagnose the root cause while explaining the failure propagation as a causal graph.

Evaluation Metrics. We grade each diagnosis along two complementary layers (formal definitions in Appendix G). The *outcome layer* scores the agent's predicted root-cause set against ground truth: *Exact Match* (EM) requires the predicted set to match ground truth exactly; *F1*, *Precision*, and *Recall* are mean fractional scores over the (service, fault_kind) pair set; *AnySvc* is the case-level rate of naming at least one correct root-cause service ignoring fault kind, the most lenient outcome score and a service-only relaxation of Recall. The *process layer* scores the agent's reasoning: *Path Reachability* (PR) is satisfied when the agent both names at least one correct root-cause service *and* draws a valid path through its predicted causal graph from that service to a ground-truth alarm node, a component observed to behave abnormally during the injection window; *Node F1* and *Edge F1* score the predicted causal graph $\hat{\mathcal{G}}$ against $\mathcal{G}^*$ under exact matching; *SQL Exec* is the share of agent-emitted evidence SQL queries that execute cleanly. PR is the most lenient process metric: it requires only one valid path from a single correct anchor, not the full graph. We reserve the term *ungrounded diagnosis* for cases that satisfy AnySvc but not PR: a correct root-cause identification not grounded in a verified causal propagation path to the observed symptom.

Results. Three patterns govern the pooled results in Table 2. First, outcome-level scores are already low (mean F1 reaches only 34.1% and EM only 20.7% across the 11 models), yet outcome alone obscures the underlying failure: agents name a correct service in 76.0% of cases (AnySvc), and

表 1：OpenRCA 2.0 按系统划分。这三个系统在规模（9–44 个服务）、语言栈和典型传播深度上存在差异，因此 500 个实例覆盖了不同的架构，而非对单一系统进行重新采样。集群配置见附录 D.4；原始池大小和静默注入率见附录 A。

属性	TrainTicket	OTel 演示	酒店预订	总计
系统中的服务	44	15	9	
实现	Java/REST	多语言/gRPC+Kafka	Go/gRPC	
可评估实例	320	38	142	500
不同的混沌主要类型	25	9	5	27
最长平均传播路径	3.0	2.5	2.1	2.7
每个实例的平均因果边数	10.1	3.0	2.7	7.5

注入的运行。在过滤掉静默注入后，我们对剩余样本应用 PAVE流水线（第 2 节），并通过基于（系统，混沌家族，根服务）的分层抽样并设置每层上限以及额外的结构性筛选规则（完整规则见附录 A）来构建 OpenRCA 2.0（500 个实例）。表 1 报告了按系统细分的情况；数据表见附录 A。

真实值。因果传播路径采用第 2 节中描述的流水线自动提取；其可靠性上限由第 3.4 节中报告的双标注员人工审计结果确定。

智能体设计。所有被评估的智能体均采用从 Deep Research 智能体 [13] 中沿用的工具增强型 ReAct 风格 [12] 架构。为分离大语言模型的推理能力与工具效果，所有模型共享相同的针对已发布遥测数据的工具集以及相同的提示词模板；不提供依赖图。每次诊断均以结构化输出返回：根因声明和传播边，每项均配对一条可重新执行的 SQL 查询作为证据（完整模式、工具定义和提示词见附录 F 和 F.3）。

3.2 基准测试实用性：根因分析智能体的过程级评估

本节依据结果层和进程层对 11 个前沿大语言模型在 OpenRCA 2.0 上进行评估，并从汇总结果中归纳出三种模式：结果得分低估了故障（发现 1），过程级的边 F1 分数普遍低于节点 F1 分数（发现 2），且没有任何单一主干模型占据主导地位（发现 3）。

实验设计。我们使用11种前沿大语言模型作为主干模型，在跨闭源和开源系列的Anthropic、Google、ByteDance、Xiaomi、Zhipu、Moonshot、DeepSeek、Alibaba、Tencent和MiniMax之间，评估上述根因分析代理架构：Claude Opus 4.7、Gemini 3.1 Pro、Claude Sonnet 4.6、MiMo 2.5 Pro、GLM 5.1、Kimi K2.6、DeepSeek V4 Pro、Qwen3.6-Max、Hy 3.0 预览版、Seed 2.0 Pro和MiniMax M2.7。每个模型接收多模态遥测数据（追踪、指标、日志），并需在诊断根本原因的同时，以因果图的形式解释故障传播。

评估指标。我们从两个互补的层面对每个诊断进行评分（正式定义见附录 G）。结果层根据真实值对代理的预测根因集进行评分：精确匹配 (EM) 要求预测集与真实值完全一致；F1、精确率和召回率是（服务，故障_类型）对集上的平均分数比率；AnySvc 是忽略故障类型并至少命名一个正确根因服务的案例级评价比率，这是最宽松的结果评分，也是召回率仅针对服务的放宽版本。过程层对代理的推理进行评分：当代理既命名了至少一个正确的根因服务，又在其预测的因果图中绘制了一条从该服务到真实告警节点（在注入窗口期间观察到行为异常的组件）的有效路径时，满足路径可达性 (PR)；节点 F1 分数和边 F1 分数在精确匹配条件下对预测因果图 $\mathcal{G}$ 与 $\mathcal{G}^*$ 进行评分；SQL 执行是代理发出的证据 SQL查询中能够干净执行的比例。PR 是最宽松的过程指标：它仅要求从单个正确锚点出发存在一条有效路径，而不需要完整的因果图。我们将满足 AnySvc 但不满足 PR 的情况保留术语称为无依据诊断：即一个未基于通往观测症状的已验证因果传播路径的正确根因识别。

结果。表2中的汇总结果受三种模式的支配。首先，结果层面分数本身已经较低（在11个模型中，平均F1分数仅为34.1%，精确匹配率仅为20.7%），然而仅凭结果层面无法揭示潜在的故障：智能体在76.0%的情况下正确识别了服务（AnySvc），并且

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 2: Outcome- and process-level evaluation on OpenRCA 2.0 pooled across 3 systems (n=500, percentages). **Bold** marks the per-column maximum. Metric definitions in Section 3.2; per-system breakdown in Appendix Table 8.

	Outcome					*Process*			
Model	EM↑	F1↑	Precision↑	Recall↑	AnySvc↑	PR↑	Node F1↑	Edge F1↑	SQL Exec↑
Claude Opus 4.7	27.6	41.4	45.7	40.1	**82.6**	70.0	67.5	49.5	97.5
Gemini 3.1 Pro	**29.4**	**43.8**	**50.4**	**40.7**	82.0	70.4	**67.9**	36.8	**98.8**
Claude Sonnet 4.6	26.2	40.5	45.7	38.5	79.4	52.6	64.5	48.0	95.6
MiMo 2.5 Pro	22.8	35.9	40.1	34.8	77.8	59.6	64.3	44.9	92.0
GLM 5.1	21.6	34.0	38.3	32.4	79.0	69.6	66.9	**50.4**	93.4
Kimi K2.6	21.0	34.0	38.4	32.6	70.8	63.2	58.8	44.1	82.4
DeepSeek V4 Pro	20.4	32.0	36.9	29.7	65.6	53.0	53.5	40.1	74.9
Qwen3.6-Max	19.2	35.7	40.5	34.9	79.2	**71.8**	65.1	48.5	92.9
Hy 3.0 Preview	17.4	30.1	33.4	30.2	71.0	53.4	55.9	38.3	92.1
Seed 2.0 Pro	14.8	26.8	30.1	26.6	78.6	66.4	64.5	47.7	90.6
MiniMax M2.7	6.8	20.6	19.7	24.2	69.8	46.4	55.3	29.2	86.9

the gap between this lenient outcome score and the most lenient process score (61.5% PR) reveals what we call the *ungrounded diagnosis* (Finding 1). Second, within the process layer mean Edge F1 (43.4%) trails mean Node F1 (62.2%) by 18.8 pp for every model, indicating that agents identify participating services more reliably than the directed dependencies among them (Finding 2). Third, the per-column extrema are distributed across four different models with distinct recall and precision profiles, so no single backbone dominates and the appropriate ranking depends on the downstream verification cost (Finding 3). The per-system spread is wider than the cross-model spread (Hotel Reservation approaches saturation on the outcome layer for the top tier, whereas TrainTicket leaves a 30+ pp gap on the process-level metrics); we therefore keep the main table pooled to support a single comparable model ranking and report the per-system breakdown in Appendix Table 8.

(1) Outcome scores understate the failure: the lenient process check exposes ungrounded diagnoses. The outcome layer pools two distinct failure modes into one number. On the lenient end, AnySvc reaches 76.0% on average, showing that agents routinely name a correct root-cause service. The strict outcome metrics, however, collapse: pair-level F1 falls to 34.1% and EM to 20.7%, with the drop from AnySvc (76.0%) to Recall (33.2%) dominated by mislabelled fault kinds on services the agent did identify. The decisive evidence that this gap is a *reasoning* failure rather than a labelling artefact comes from the process layer: even PR, which requires only one valid path from a single correct anchor, is satisfied in only 61.5% of cases, so for 14.5 pp of cases the agent names a correct service *without* drawing any verifiable path to the observed symptom. The per-model conditional rate PR/AnySvc separates models that outcome-only grading rates as comparable: Qwen3.6-Max attains $71.8/79.2 \approx 91\%$, whereas Sonnet 4.6 attains only $52.6/79.4 \approx 66\%$ despite comparable F1, indicating that Sonnet emits correct service labels that are not integrated into a connected propagation path.

(2) Edge F1 is universally lower than Node F1, and the gap is the size of the entire process-level signal. The ordering holds for every model in the table, and the gap is substantial in absolute terms (62.2% vs. 43.4%, 18.8 pp on average). Gemini exhibits the widest split, reaching 67.9% Node F1 but only 36.8% Edge F1. Identifying the participating services approximates a classification task that pre-training prepares the model for; ordering them into a directed propagation path requires reasoning about what each abnormal signal implies for the underlying inter-service dependencies, a derivation step that outcome-only metrics do not score. This separation provides direct evidence that the agent's *output* (a service list) is closer to ground truth than its *derivation* (a directed graph), and that the discrepancy is large enough to affect model selection in practice.

3.3 Failure Mode Characterization

To probe what drives the shortfall between AnySvc and PR, we read agent trajectories across the 11 models on a stratified sample of cases tiered by joint correctness (the fraction of models clearing $F_1{>}0$ on each case). Three patterns recur across the sample, each describing how an agent commits to an incorrect or unsupported answer rather than continuing to verify, and together they account for the dominant share of ungrounded diagnoses.

表2：在OpenRCA 2.0上对3个系统进行池化后的结果层和过程级评估（n=500，百分比）。粗体标记每列的最大值。指标定义见第 3.2 节；按系统细分见附录表 8。

	结果层					进程			
模型	EM↑	F1↑	精确率↑	召回率↑	AnySvc↑	PR↑	节点F1↑	边F1↑	SQL 执行↑
Claude Opus 4.7	27.6	41.4	45.7	40.1	**82.6**	70.0	67.5	49.5	97.5
Gemini 3.1 Pro	**29.4**	**43.8**	**50.4**	**40.7**	82.0	70.4	**67.9**	36.8	**98.8**
Claude Sonnet 4.6	26.2	40.5	45.7	38.5	79.4	52.6	64.5	48.0	95.6
MiMo 2.5 Pro	22.8	35.9	40.1	34.8	77.8	59.6	64.3	44.9	92.0
GLM 5.1	21.6	34.0	38.3	32.4	79.0	69.6	66.9	**50.4**	93.4
Kimi K2.6	21.0	34.0	38.4	32.6	70.8	63.2	58.8	44.1	82.4
DeepSeek V4 Pro	20.4	32.0	36.9	29.7	65.6	53.0	53.5	40.1	74.9
Qwen3.6-Max	19.2	35.7	40.5	34.9	79.2	**71.8**	65.1	48.5	92.9
Hy 3.0 预览版	17.4	30.1	33.4	30.2	71.0	53.4	55.9	38.3	92.1
Seed 2.0 Pro	14.8	26.8	30.1	26.6	78.6	66.4	64.5	47.7	90.6
MiniMax M2.7	6.8	20.6	19.7	24.2	69.8	46.4	55.3	29.2	86.9

这一宽松的结果分数与最宽松的过程分数（61.5% 路径可达性指标）之间的差距揭示了我们所谓的无依据诊断（发现 1）。其次，在过程层中，每个模型的平均边 F1（43.4%）均落后于平均节点 F1（62.2%）18.8 个百分点，这表明智能体识别参与服务的可靠性高于识别它们之间的有向依赖（发现 2）。第三，每列极值分布在具有不同召回率和精确率个人资料的四款不同模型中，因此没有单一主干模型占据主导地位，且适当的模型排名取决于下游验证成本（发现 3）。系统间差异大于模型间差异（在结果层上，顶层模型在酒店预订任务上接近饱和，而 TrainTicket 在过程级指标上留有 30+ 个百分点的差距）；因此，我们在主表中采用池化数据以支持单一可比较的模型排名，并在附录表 8 中报告按系统细分结果。

(1) 结果得分低估了故障情况：宽松的过程检查揭示了无根据诊断。结果层将两种截然不同的失败模式合并为一个数值。在宽松标准下，AnySvc 的平均表现达到 76.0%，表明智能体通常能够识别出正确的根因服务。然而，严格结果指标出现急剧下降：配对级 F1 降至 34.1%，精确匹配率降至 20.7%，从 AnySvc（76.0%）到召回率（33.2%）的下降主要归因于在智能体已识别出的服务上标注错误的故障类型。这一差距属于推理失败而非标注伪影的决定性证据来自过程层：即使仅需从一个正确锚点到观测症状存在一条有效路径的路径可达性指标，也仅在 61.5% 的案例中得到满足，因此在 14.5 个百分点的案例中，智能体虽然正确命名了服务，却未能建立到观测症状的任何可验证路径。每模型条件率 PR/AnySvc 区分了仅结果评价认为表现相当的模型：Qwen3.6-Max 达到 $71.8/79.2 \approx 91\%$，而 Sonnet 4.6 尽管 F1 分数相当，却仅达到 $52.6/79.4 \approx 66\%$，这表明 Sonnet 输出的服务标签虽然正确，但未被整合到连贯的传播路径中。

(2) 边 F1 分数普遍低于节点 F1 分数，二者之间的差距即为整个进程级信号的量级。表中所有模型均遵循这一规律，且差距在绝对数值上显著（62.2% 对 43.4%，平均相差 18.8 个百分点）。Gemini 展现出最大的差异，其节点 F1 分数达到 67.9%，而边 F1 分数仅为 36.8%。识别参与服务近似于一个预训练已使模型准备就绪的分类任务；而将这些服务排序为有向传播路径则需要推理每个异常信号对底层服务间依赖关系的含义，这一推导步骤是仅结果指标无法评估的。这种分离直接证明代理的输出（服务列表）比其推导过程（有向图）更接近真实值，且该差异大到足以在实际应用中影响模型选择。

3.3 故障模式表征

为探究 AnySvc 与 PR 之间差距的驱动因素，我们在按联合正确性（即每个案例中清除 $F_1{>}0$ 的模型比例）分层的案例样本上，分析了跨 11 个模型的智能体轨迹。在整个样本中，反复出现三种模式，每种模式描述了智能体如何转向错误或缺乏支持的结论而非继续验证，这些模式共同解释了无根据诊断的主导比例。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

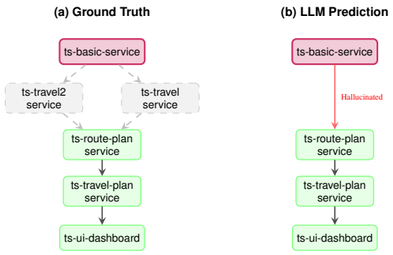


Figure 4: Spurious causal reasoning case. (a) Ground truth shows fault propagation through intermediate services (`travel`, `travel2`). (b) Claude correctly identifies root cause and symptoms but hallucinates a direct edge, missing the branching structure (Edge F1=44.4%).

(1) Premature commitment under under-exploration. After identifying one plausible root cause, the agent stops investigating and emits the answer, leaving co-occurring faults or alternative attributions unexamined. This shape is most visible on hybrid injections that pair one prominent fault with one subtle one: every evaluated model converges on the prominent half (often emitting nearly identical wording across heterogeneous backbones) and uniformly misses the subtle half, depressing Recall and EM whenever the ground truth covers more than one service.

(2) Presence bias on observable activity. Agents weight evidence by what they see and treat absence of observed activity as evidence of normal operation, rather than recognizing that the absence itself can be the fault signature. A killed-pod injection makes this failure mode crisp: during the incident window the affected service stops producing spans entirely, and many agents read the resulting silence as "this service has no errors, hence no fault here", then attribute the incident to the upstream callers that surface errors precisely because their callee disappeared. The dual of Pattern (3): attentional bias toward strong positive signals and against silent ones jointly explain the bulk of the localization shortfall.

(3) Salience capture on the loudest signal. Agents converge on the service whose error or latency aggregates carry the strongest abnormal-versus-normal delta, even when those signals are pure downstream amplification rather than the originating fault; this pattern accounts for the bulk of the localization failures responsible for AnySvc falling short of full coverage by roughly 24pp. The capture operates on aggregate metrics (a high-traffic hub on the load-generator path absorbs attention while the lower-traffic ground-truth service goes unnamed) and equally on text: a SEVERE log line claiming a timeout is taken at face value by most models even when the corresponding trace span returns in tens of milliseconds and contradicts the log claim.

The case study below illustrates Pattern (1) on the path-drawing dimension specifically: an agent commits to the correct service and stops verifying, leaving the propagation chain to skip the intermediates that the verified ground truth actually traverses.

Case Study: An Ungrounded Diagnosis. Figure 4 shows an injection where Claude correctly names the root cause (ts-basic-service) but justifies it with a graph that skips the services actually on the 4-hop ground-truth propagation path: a paradigmatic ungrounded diagnosis. The predicted graph hallucinates a direct edge from ts-basic-service to ts-route-plan-service, skipping the intermediate ts-travel2-service and ts-travel-service (Node Precision/Recall 100%/66.7%, Edge Precision/Recall 66.7%/33.3%, Edge F1 44.4%). Outcome-only evaluation scores this as a hit on AnySvc; process-level evaluation surfaces the missing edges and exposes the reasoning failure an on-call engineer would still hit in practice.

3.4 Annotation Reliability

A two-annotator manual audit on $N{=}100$ instances drawn uniformly at random from OpenRCA 2.0 reports 94% case-level agreement $(94/100)$ on the validity of $\mathcal{G}^*$, with every disagreement attributable to under-coverage of weakly supported paths rather than to a spurious included edge. Two of the paper's authors, with prior experience in distributed-system observability and the PAVE schema, independently rated each case as AGREE when every edge in $\mathcal{G}^*$ is plausibly causal from the telemetry

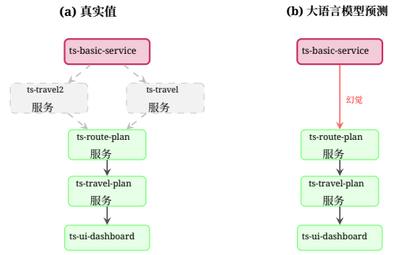


图4：虚假因果推理案例。（a）真实值显示故障通过中间服务（旅行、travel2）传播。（b）Claude 正确识别了根本原因和症状，但幻觉生成了直接边，遗漏了分支结构（边 F1=44.4%）。

(1) 过早承诺与探索不足。在识别出一个可能的根本原因后，代理停止调查并输出答案，导致并发故障或替代归因未被检查。这种模式在混合注入场景中最为明显，即将一个显著故障与一个微弱故障配对：所有评估模型都收敛于显著故障部分（在不同异构骨干网络间常输出几乎相同的措辞），并均匀地遗漏微弱故障部分，从而在真实值覆盖多个服务时压低召回率和精确匹配率。

(2) 可观察活动上的存在性偏差。智能体根据所见证据进行加权，并将观察到的活动缺失视为正常运行的证据，而非认识到这种缺失本身可能就是故障特征。Pod终止注入使这种故障模式变得清晰：在事件窗口期间，受影响的服务完全停止产生跨度，许多智能体将由此产生的沉默解读为"该服务没有错误，因此此处没有故障"，随后将事件归因于上游调用方，这些上游调用方之所以表现出错误，恰恰是因为其被调用方消失了。模式(3)的对偶形式：对强正信号的注意力偏差以及对静默信号的忽视，共同解释了大部分定位不足的原因。

（3）显著性捕获于最强信号。智能体倾向于收敛于错误率或延迟聚合指标显示出最强异常与正常差异的服务，即使这些信号纯粹是下游放大而非源故障；这一模式导致了大部分定位失败，使AnySvc的全覆盖率落后约24个百分点。这种捕获机制不仅作用于聚合指标（负载生成器路径上的高流量枢纽吸引了注意力，而低流量真实服务则未被识别），也同样作用于文本内容：即使对应的跟踪跨度在几十毫秒内返回且与日志声明相矛盾，大多数模型仍会对声称超时的严重日志行信以为真。

以下案例研究具体说明了路径绘制维度上的模式（1）：代理确定正确的服务后停止验证，导致传播链跳过真实值实际遍历的中间服务。

案例研究：一项无依据诊断。图4展示了一项注入，其中Claude正确指出了根本原因（ts-basic-service），但其依据的图跳过了4跳真实传播路径上实际涉及的服务：这是一种典型的无依据诊断。预测图幻觉出一条从ts-basic-service到ts-route-plan-service的直接边，从而跳过了中间节点ts-travel2-service和ts-travel-service（节点精确率/召回率为100%/66.7%，边精确率/召回率为66.7%/33.3%，边F1分数为44.4%）。仅结果评估将此项计为在AnySvc上的命中；过程级评估则揭示了缺失的边，并暴露了值班工程师在实践中仍会遭遇的推理失败。

3.4 注释可靠性

一项针对从 OpenRCA 2.0 中随机均匀抽取的 $N{=}100$ 实例进行的双标注者人工审计显示，在 $\mathcal{G}^*$ 有效性方面，案例级一致性为 94%（$94/100$），所有分歧均归因于对弱支持路径的覆盖不足，而非存在虚假包含边。该论文的两名作者具备分布式系统可观测性和 PAVE 模式的先前经验，他们独立地对每个案例进行评估，当 $\mathcal{G}^*$ 中的每条边根据遥测数据看似因果合理时，将其标记为同意

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

and no obvious propagation edge is missing, and DISAGREE otherwise; annotators were blind to one another's verdicts. All six disagreements were one-directional: at least one annotator considered an additional propagation path plausible that the verification gates had rejected as weakly supported, and no case was flagged for an implausible included edge. Joint adjudication attributed every under-coverage instance to the conjunctive design's deliberate strictness on borderline evidence; for a process-level reference, omitting weakly supported edges is preferable to admitting spurious ones, so we accept this precision-favoured trade-off. The full protocol, verification procedure, and per-disagreement notes are in Appendix E.1.

4 Related Work

4.1 From Classification-Based Ranking to Generative Reasoning

The formulation of Root Cause Analysis (RCA) has undergone a fundamental shift. Traditionally, RCA was cast as a *multi-class classification problem*, where each candidate component is scored and ranked by its likelihood of being the root cause, using heuristic algorithms (e.g., Random Walk [14]) or supervised graph neural networks [15]. While effective for locating faults, these "black-box" approaches lack interpretability and fail to capture the sequential propagation mechanism. With the advent of Large Language Models (LLMs), recent works have pivoted towards a *generative reasoning paradigm* [1–4], treating RCA as a causal narrative generation task. However, a critical misalignment persists: while the model architecture has evolved to handle complex reasoning, the underlying data supervision remains stagnant. Existing benchmarks [6, 7, 2] still rely on outcome-based labels that record only *which* component is the root cause, without annotating *how* the fault propagates, inherited from the classification era. This forces reasoning agents to optimize for the final answer directly, bypassing the logical derivation steps and encouraging the memorization of spurious correlations rather than learning robust causal dynamics.

4.2 Causal Structure Learning under Intervention

Our work intersects with causal discovery, particularly in dynamic systems. Classical observational methods (e.g., PC algorithm [16]) struggle to recover Directed Acyclic Graphs (DAGs) in distributed systems due to unobserved confounders and high-dimensional noise [17, 18]. Pearl's hierarchy of causation posits that interventional data ($do(x)$) is essential for distinguishing true causality from mere correlation [8]. While some RCA methods attempt to infer causal graphs relying solely on observational telemetry [19, 20], they lack the *identifiability* guarantees that controlled interventions provide. PAVE distinguishes itself by adopting an *interventional* rather than purely observational perspective during annotation. While prior benchmarks use only the identity of the injected component as the ground-truth label, PAVE leverages the full intervention specification ($do(v_{root})$) to actively verify propagation paths. This effectively transforms the intractable structure learning problem into a tractable verification task, filtering out silent injections and confounding noise that plague purely observational datasets.

4.3 Process Supervision and Verifiability

In the broader LLM landscape, focus is shifting from outcome-based supervision (Outcome Reward Models, ORMs) to *Process Supervision* (Process Reward Models, PRMs) [21–23]. Research in mathematical reasoning and code generation demonstrates that providing step-by-step verification signals significantly reduces hallucinations and improves generalization [24, 25]. However, applying this to system diagnostics is hindered by the *verification gap*: unlike math (where proofs are deterministic) or code (where unit tests exist), operational faults lack a natural "compiler" to verify reasoning steps. Current RCA benchmarks like AIOpsLab [2] or OpenRCA [1] rely on human-curated scenarios or heuristically filtered logs, which are hard to scale and lack granular process labels. PAVE addresses this verification gap for system diagnostics by synthesizing verifiable failure propagation paths rooted in system topology and causal interventions, producing the first large-scale process-level ground truth for *evaluating* diagnostic reasoning in complex systems. The resulting step-wise annotations also open a path toward training PRMs in the reliability domain, though we leave this exploration to future work.

且不存在明显缺失的传播边，则同意；否则不同意；标注员之间互不知晓彼此的裁决结果。全部六处分歧均为单向性：至少一名标注员认为验证门控机制因证据支持薄弱而拒绝的某条额外传播路径具有合理性，且没有案例被标记为包含不合理的已纳入边。联合仲裁将所有覆盖不足实例归因于合取设计对临界证据所采取的刻意严格标准；对于过程级参考而言，省略支持薄弱的边优于纳入虚假的边，因此我们接受这一偏向精确率的权衡。完整的协议、验证程序和逐分歧备注详见附录 E.1。

4 相关工作

4.1 基于分类的排名到生成式推理

根因分析（RCA）的形式化定义发生了根本性转变。传统上，RCA被构建为一个多类分类问题，使用启发式算法（例如随机游走 [14]）或监督图神经网络 [15]对每个候选组件进行评分和排名，以确定其成为根本原因的可能性。虽然这些"黑盒"方法在定位故障方面有效，但它们缺乏可解释性，且无法捕获顺序传播机制。随着大语言模型（LLMs）的出现，近期研究转向了生成式推理范式 [1–4], , 将RCA视为因果叙事生成任务。然而，一个关键的错配依然存在：尽管模型架构已演进以处理复杂推理，但底层数据监督仍然停滞不前。现有的基准测试 [6, 7, 2] 仍然依赖于基于结果的标签，这些标签仅记录哪个组件是根本原因，而未标注故障如何传播，这是从分类时代继承而来。这迫使推理代理直接优化最终答案，绕过逻辑推导步骤，并鼓励对伪相关性的记忆，而非学习稳健的因果动态。

4.2 干预下的因果结构学习

我们的工作与因果发现领域交叉，特别是在动态系统中。经典的观测方法（例如PC算法 [16]）由于未观测混杂因素和高维噪声，难以在分布式系统中恢复有向无环图（DAGs）[17, 18]。珀尔因果层级认为，干预数据（$do(x)$）对于区分真实因果关系与简单相关性至关重要 [8]。虽然一些根因分析方法尝试仅依靠观测遥测数据推断因果图 [19, 20], , 但它们缺乏受控干预所提供的可识别性保证。PAVE通过在标注过程中采用干预视角而非纯观测视角而区别于以往工作。尽管先前的基准测试仅使用注入组件的身份作为真值标签，但PAVE利用完整的干预规范（$do(v_{root})$）来主动验证传播路径。这有效地将棘手的结构学习问题转化为可处理的验证任务，从而过滤出在纯观测数据集中普遍存在的静默注入和混杂噪声。

4.3 过程监督与可验证性

在更广泛的大语言模型领域，研究重心正从基于结果的监督（Outcome-based supervision，结果奖励模型，ORMs）转向过程监督（Process Supervision，过程奖励模型，PRMs）[21–23]。数学推理和代码生成领域的研究表明，提供逐步验证信号（step-by-step verification signals）能够显著减少幻觉（hallucinations）并提升泛化能力（generalization）[24, 25]。然而，将此方法应用于系统诊断（system diagnostics）时受到验证差距（verification gap）的阻碍：与数学（其中证明（proofs）具有确定性）或代码（其中存在单元测试（unit tests））不同，操作故障（operational faults）缺乏自然的"编译器"（compiler）来验证推理步骤。当前的根因分析（RCA）基准测试（RCA benchmarks）（如 AIOpsLab [2] 或 OpenRCA [1]）依赖于人工策划场景（human-curated scenarios）或启发式过滤日志（heuristically filtered logs），这些方法难以扩展且缺乏细粒度的过程标签（process labels）。PAVE 通过合成基于系统拓扑（system topology）和因果干预（causal interventions）的可验证故障传播路径（verifiable failure propagation paths），解决了系统诊断中的这一验证差距，从而生成了首个用于评估复杂系统中诊断推理过程的大规模过程级真实值（process-level ground truth）。由此产生的逐步标注（step-wise annotations）也为在可靠性领域（reliability domain）训练过程奖励模型（process reward models）开辟了一条路径，尽管我们将这一探索留待未来工作。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 3: Differentiation of OpenRCA 2.0 from prior RCA benchmarks. "✓" = present; "–" = absent or not reported as a fixed-size benchmark; "part." = partial form (e.g., issue-type tag without a propagation graph). Instance counts are reported as in each source.

Benchmark	#Sys	#Inst.	Outcome	**Process**	SLO surface	LLM eval
OpenRCA 1.0 [1]	3	335	✓	–	implicit	✓
AIOpsLab [2]	2	48	✓	part.	implicit	✓
RCAEval [6]	3	735	✓	–	–	–
Stratus [3]	3	–	✓	–	implicit	✓
Fang et al. [7]	1	8137	✓	–	implicit	–
OpenRCA 2.0 (ours)	3	500	✓	✓	**explicit**	✓

Differentiation summary. Table 3 compares OpenRCA 2.0 to five prior RCA benchmarks on seven axes. Two columns separate our work from prior benchmarks. *Process annotations* record the step-wise causal edges of each cascade rather than only the root-cause identity; AIOpsLab provides a partial form (issue-type tags without a propagation graph), and the others provide none. The *SLO surface as an explicit method input* promotes a threshold that prior work fixes implicitly into a configurable knob.

5 Assumptions and Threats to Validity

This section delimits OpenRCA 2.0's claims along three axes: the input conditions PAVE relies on, the residual threats internal to the pipeline, and the boundary of external transfer.

Input assumptions. Four falsifiable conditions underwrite the protocol: a real controlled intervention $do(v_{root})$ rather than an inferred label, a service-layer dependency graph complete enough that genuine horizontal propagation survives structural pruning, a propagation signature in trace, metric, or log modalities, and a pre-injection baseline that approximates the no-intervention distribution closely enough for baseline-relative anomaly screening to discriminate. Where any of these fails for a target setting, the reported numbers should be read as an upper bound rather than a transferable estimate.

Internal threats. Two single-gate weaknesses remain inside the pipeline: the rule set $\mathcal{R}$ may silently prune propagation mechanisms outside its current vocabulary, and the per-node anomaly screen is calibrated toward low false negatives (calibration split in Appendix E.2) and so admits individual nodes spuriously. Neither propagates to end-to-end errors, because admission requires the conjunction of rule consistency, baseline-relative deviation, and upstream-to-downstream timing alignment (Eq. (3)), and a two-annotator audit on a uniform-random 100-case subsample reports zero false-positive edges in the audited sample (Section 3.4).

External transfer. Faithful simulation of production-scale microservice deployments is neither a goal of this benchmark nor realistically achievable inside controlled testbeds, so reported numbers apply within the testbed envelope and transfer to production must be calibrated rather than assumed. A complementary scope note concerns information conditions: PAVE's annotation step requires full telemetry, since forward verification relies on it, but the evaluation step does not. The same step-wise ground truth, established once under full data, can be reused to evaluate agents under deliberately restricted telemetry (subsampled traces, missing modalities, truncated observation windows), turning information availability into a controllable evaluation axis that we leave to future work.

6 Conclusion

We introduced PAVE, an annotation protocol that reconstructs verified causal propagation paths from fault-injection experiments by exploiting the information asymmetry between backward RCA and forward verification from a known intervention. Its application across 3 architecturally distinct microservice systems yields OpenRCA 2.0 (500 instances), the first cross-system RCA benchmark with step-wise causal annotations and process-level metrics (Path Reachability, Node F1, Edge F1) that grade the *causal path* rather than only the identified root cause. On this benchmark, process-level evaluation of 11 frontier LLMs exposes the *ungrounded diagnosis*, a failure mode invisible to outcome-only evaluation in which correct root-cause identifications are not grounded in any verified propagation path to the observed symptom. More broadly, step-wise causal ground truth raises the

表3：OpenRCA 2.0与先前根因分析基准测试的区别。"✓" =表示存在；"–" =表示缺失或未作为固定规模基准测试进行报告；"part." = 表示部分形式（例如，仅有问题类型标签而无传播图）。实例数量按各来源中的数据报告。

基准测试	#Sys	#仪器	结果	**进程**	SLO边界	大语言模型评估
OpenRCA 1.0 [1]	3	335	✓	–	隐式	✓
AIOpsLab [2]	2	48	✓	部分。	隐式	✓
RCAEval [6]	3	735	✓	–	–	–
Stratus [3]	3	–	✓	–	隐式	✓
Fang 等人 [7]	1	8137	✓	–	隐式	–
OpenRCA 2.0（本文提出的方法）	3	500	✓	✓	**显式**	✓

Differentiation Summary. Table 3 compares OpenRCA 2.0 with five prior RCA benchmarks across seven axes. Two columns distinguish our work from prior benchmarks. Process annotations record the step-wise causal edges of each cascade rather than only the root-cause identity; AIOpsLab provides a partial form (issue-type tags without a propagation graph), and the others provide none. The SLO surface as an explicit method input promotes a threshold that prior work fixes implicitly into a configurable knob.

5 假设与对有效性的威胁

本节从三个维度界定OpenRCA 2.0的声明：PAVE所依赖的输入条件、管道内部的残余威胁，以及外部迁移的边界。

输入假设。四项可证伪条件支撑该协议：一个真实的受控干预$do(v_{root})$而非推断标签；一个足够完整的服务层依赖图，使得真正的水平传播能够在结构剪枝后保留；在追踪、指标或日志模态中存在传播特征；以及一个注入前基线，其近似无干预分布足够紧密，以便基线相对异常筛查能够进行区分。若这些条件中的任何一项在目标设置下失效，则所报告的数值应被视为上界而非可转移的估计值。

内部威胁。流水线中仍存在两个单门控弱点：规则集 $\mathcal{R}$ 可能悄无声息地剪枝其当前词汇表之外的传播机制，而每节点异常筛查经过校准以尽量减少假阴性（校准划分见附录E.2），因此会虚假地准入个别节点。这些问题不会导致端到端错误，因为准入需要规则一致性、基线相对偏差以及上下游时序对齐（公式(3)）同时满足，并且在均匀分布的随机100例子样本上进行的双标注员审计报告显示，被审计样本中假阳性边为零（见第3.4节）。

外部迁移。忠实模拟生产规模的微服务部署既非本基准测试的目标，在受控测试床内也无法现实地实现，因此报告的数值仅适用于测试床环境内的范围，向生产环境的迁移必须经过校准而非直接假定。另一个相关的范围说明涉及信息条件：PAVE的标注步骤需要完整遥测数据，因为前向验证依赖于这些数据，但评估步骤则不需要。在完整数据下一次性建立的逐步真实值，可复用于在故意限制遥测数据的情况下（抽样追踪、缺失模态、截断的观测窗口）评估智能体，从而将信息可用性转化为一个可控的评估维度，我们将此留待未来工作研究。

6 结论

我们提出了PAVE，这是一种标注协议，通过利用已知干预下的回溯根因分析（backward RCA）与前向验证（forward verification）之间的信息不对称，从故障注入实验中重建已验证因果传播路径（verified causal propagation path）。该方法应用于三个架构不同的微服务系统（microservice systems），生成了OpenRCA 2.0（包含500个实例），这是首个包含逐步因果标注（step-wise causal annotations）和过程级指标（Path Reachability, Node F1, Edge F1）的跨系统根因分析基准（cross-system RCA benchmark），旨在对因果传播路径（causal propagation path）进行评分，而不仅仅是对识别出的根本原因（root cause）进行评分。在该基准测试（benchmark）上，对11个前沿大语言模型（frontier LLMs）进行的过程级评估揭示了"无依据诊断"（ungrounded diagnosis）这一失效模式，该模式在仅结果评估（Outcome-only evaluation）中不可见，即正确的根因识别并未建立在任何已验证的从观测症状（observed symptom）出发的传播路径（propagation path）之上。更广泛地说，逐步因果真值（step-wise causal ground truth）提高了

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

standard from correct localization to correct reasoning, the bar any diagnostic agent must clear to be trusted where errors carry real consequences.

References

[1] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. Openrca: Can large language models locate the root cause of software failures? In *The Thirteenth International Conference on Learning Representations*, 2025.

[2] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. Aiopslab: A holistic framework to evaluate ai agents for enabling autonomous clouds. In *MLSys*, 2025. URL `https://openreview.net/forum?id=3EXBLwGxtq`.

[3] Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. Stratus: A multi-agent system for autonomous reliability engineering of modern clouds. *arXiv preprint arXiv:2506.02009v2*, 05 2025. URL `https://arxiv.org/abs/2506.02009v2`.

[4] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Siyu Yu, Jinyang Gao, Bolin Ding, Zhonghai Wu, and Ying Li. Thinkfl: Self-refining failure localization for microservice systems via reinforcement fine-tuning. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3789262. URL `https://doi.org/10.1145/3789262`.

[5] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47(2):243–260, 2018.

[6] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. Rcaeval: A benchmark for root cause analysis of microservice systems with telemetry data. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 777–780, 2025.

[7] Aoyang Fang, Songhan Zhang, Yifan Yang, Haotong Wu, Junjielong Xu, Xuyang Wang, Rui Wang, Manyi Wang, Qisheng Lu, and Pinjia He. Rethinking the evaluation of microservice rca with a fault propagation-aware benchmark. *arXiv preprint arXiv:2510.04711v2*, 10 2025. URL `https://arxiv.org/abs/2510.04711v2`.

[8] Judea Pearl. *Causality*. Cambridge university press, 2009.

[9] Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-play only evolves when self-synthetic pipeline ensures learnable information gain. *arXiv preprint arXiv:2603.02218v1*, 02 2026. URL `https://arxiv.org/abs/2603.02218v1`.

[10] Deathstarbench. `https://github.com/delimitrou/DeathStarBench/tree/master`, 2024. Accessed: 2026-02-10.

[11] Opentelemetry demo. `https://github.com/open-telemetry/opentelemetry-demo`, 2026. Accessed: 2026-05-05.

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[13] Paichun Lin. Self-balancing agentic AI: Test-time diffusion and context engineering re-imagined for deep research. `https://github.com/thinkdepthai/Deep_Research`, 2025.

[14] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *Proceedings of the Web Conference 2021*, pages 3087–3098, 2021.

将标准从正确定位提升到正确推理，从而提高了任何诊断代理在错误具有实际后果的场景中获得信任的门槛。

参考文献

[1] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, 裴丹, 何品佳, Dongmei Zhang, and Qi Zhang. OpenRCA: 大语言模型能否定位软件故障的根本原因？In 第十三届学习表示国际会议, 2025.[2] Yinfang Chen, Manish Shetty, Gagan Somashekar, 马明华, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: 一个用于评估支持自主云的智能体AI的完整框架。In MLSys, 2025. URL https:// openreview.net/forum?id=3EXBLwGxtq.[3] Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. Stratus: 一个用于现代云自主可靠性工程的多智能体系统。arXiv preprint arXiv:2506.02009v2, 05 2025. URL https://arxiv.org/abs/2506.02009v2.[4] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Siyu Yu, Jinyang Gao, Bolin Ding, Zhonghai Wu, and Ying Li. ThinkFL: 通过强化微调实现微服务系统的自优化故障定位。ACM软件工程与方法学汇刊, 2025. doi: 10.1145/3789262. URL https://doi.org/10.1145/3789262.[5] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 微服务系统的故障分析与调试：工业调查、基准系统与实证研究。IEEE软件工程汇刊, 47(2):243–260, 2018.[6] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. RCAEval: 基于遥测数据的微服务系统根因分析基准。In Companion Proceedings of the ACM on Web Conference 2025, pages 777–780, 2025.[7] 方傲阳, 张松翰, 杨一凡, Haotong Wu, Junjielong Xu, Xuyang Wang, 王锐, Manyi Wang, 陆启胜, and 何品佳. 使用故障传播感知基准重新思考微服务根因分析的评估。arXiv preprint arXiv:2510.04711v2, 10 2025. URL https://arxiv.org/abs/2510.04711v2.[8] Judea Pearl. 因果关系. 剑桥大学出版社, 2009.[9] Wei Liu, Siya Qi, Yali Du, and Yulan He. 仅当自合成流水线确保可学习信息增益时，自博弈才会演化。arXiv preprint arXiv:2603.02218v1, 02 2026. URL https://arxiv.org/abs/2603.02218v1.[10]DeathStarBench. https://github.com/delimitrou/DeathStarBench/tree/master, 2024. Accessed: 2026-02-10.[11] OpenTelemetry演示. https://github.com/open-telemetry/opentelemetry-demo, 2026. Accessed: 2026-05-05.[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. ReAct: 在语言模型中协同推理与行动。In 第十一届学习表示国际会议, 2022.[13] Paichun Lin. 自平衡智能体AI：为深度研究重新构想的测试时扩散与上下文工程。https://github.com/thinkdepthai/Deep_Research, 2025.[14] 于广巴, 陈鹏飞, 陈宏阳, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and 李晓云. MicroRank: 在微服务环境中利用扩展谱分析进行端到端延迟问题定位。In 2021年Web会议论文集, pages 3087–3098, 2021.

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

[15] Songhan Zhang, Aoyang Fang, Yifan Yang, Ruiyi Cheng, Xiaoying Tang, and Pinjia He. Dynacausal: Dynamic causality-aware root cause analysis for distributed microservices. *arXiv preprint arXiv:2510.22613v1*, 10 2025. URL https://arxiv.org/abs/2510.22613v1.

[16] Peter Spirtes, Clark N Glymour, and Richard Scheines. *Causation, prediction, and search*. MIT press, 2000.

[17] Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Saini, Saurabh Bagchi, and Murat Kocaoglu. Root cause analysis of failures in microservices through causal discovery. *Advances in Neural Information Processing Systems*, 35:31158–31170, 2022.

[18] Xiao Han, Saima Absar, Lu Zhang, and Shuhan Yuan. Root cause analysis of anomalies in multivariate time series through granger causal discovery. In *The Thirteenth International Conference on Learning Representations*, 2025.

[19] Zeyan Li, Nengwen Zhao, Shenglin Zhang, Yongqian Sun, Pengfei Chen, Xidao Wen, Minghua Ma, and Dan Pei. Constructing large-scale real-world benchmark datasets for aiops. *arXiv preprint arXiv:2208.03938v1*, 08 2022. URL https://arxiv.org/abs/2208.03938v1.

[20] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 553–565, 2023.

[21] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

[22] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275v1*, 11 2022. URL https://arxiv.org/abs/2211.14275v1.

[23] Thomas Zeng, Shuibai Zhang, Shutong Wu, Christian Classen, Daewon Chae, Ethan Ewer, Minjae Lee, Heeju Kim, Wonjun Kang, Jackson Kunde, et al. Versaprm: Multi-domain process reward model via synthetic reasoning data. In *Forty-second International Conference on Machine Learning*, 2025.

[24] Zhangyue Yin, Qiushi Sun, Zhiyuan Zeng, Qinyuan Cheng, Xipeng Qiu, and Xuan-Jing Huang. Dynamic and generalizable process reward modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4203–4233, 2025.

[25] Yibo Yan, Jiamin Su, Jianxiang He, Fangteng Fu, Xu Zheng, Yuanhuiyi Lyu, Kun Wang, Shen Wang, Qingsong Wen, and Xuming Hu. A survey of mathematical reasoning in the era of multimodal large language model: Benchmark, method & challenges. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11798–11827, 2025.

[15] 张松翰、方傲阳、杨一凡、程瑞仪、唐晓颖和何品佳。Dynacausal：面向分布式微服务的动态因果感知根因分析。arXiv预印本 arXiv:2510.22613v1, 10 2025. URL https://arxiv.org/abs/2510.22613v1.[16] 彼得·斯皮尔特斯、克拉克·N·格莱莫尔和理查德·谢因斯。因果、预测与搜索. 麻省理工学院出版社, 2000.[17] 阿扎姆·伊克拉姆、萨塔克·查克拉伯蒂、苏布拉塔·米特拉、希夫·萨伊尼、索拉布·巴格奇和穆拉特·科恰奥卢。通过因果发现进行微服务故障根因分析. 神经信息处理系统进展, 35:31158–31170, 2022.[18] 韩晓、赛玛·阿布斯尔、张璐和袁书涵。通过格兰杰因果发现进行多元时间序列异常根因分析. 载于第十三届学习表示国际会议, 2025.[19] 李泽岩、赵能文、张圣林、孙永谦、陈鹏飞、文喜道、马明华和裴丹。构建面向AIOps的大规模真实世界基准数据集. arXiv预印本 arXiv:2208.03938v1, 08 2022. URL https://arxiv.org/abs/2208.03938v1.[20] 于广巴、陈鹏飞、李宇峰、陈宏阳、李晓云和郑子彬。Nezha：基于多模态可观测性数据的微服务可解释细粒度根因分析. 载于第31届ACM欧洲软件工程会议暨软件工程基础研讨会论文集, 页码 553–565, 2023.[21] 亨特·莱特曼、维内特·科萨拉朱、尤里·布尔达、哈里森·爱德华兹、鲍文·贝克、泰迪·李、扬·莱克、约翰·舒尔曼、伊利亚·苏茨克维和卡尔·科布。让我们逐步验证. 载于第十二届国际表示学习会议, 2023.[22] 乔纳森·乌萨托、内特·库什曼、拉马纳·库马尔、弗朗西斯·宋、诺亚·西格尔、丽莎·王、安东尼娅·克雷斯韦尔、杰弗里·欧文和伊琳娜·希金斯。利用过程和结果反馈解决数学应用题. arXiv预印本 arXiv:2211.14275v1, 11 2022. URL https://arxiv.org/abs/2211.14275v1.[23] 托马斯·曾、张水白、武书桐、克里斯蒂安·克拉森、蔡大源、伊桑·埃弗、李明宰、金喜珠、姜元俊、杰克逊·昆德等。Versaprm：通过合成推理数据实现的多领域过程奖励模型. 载于第四十二届国际机器学习会议, 2025.[24] 尹章月、孙秋实、曾致远、程沁源、邱锡鹏和黄萱菁。动态且泛化的过程奖励建模. 载于第63届计算语言学协会年度会议论文集（第1卷：长论文）, 页码 4203–4233, 2025.[25] 严一博、苏佳敏、何建祥、付方腾、郑旭、吕元辉义、王坤、王申、温庆松和胡旭明。多模态大语言模型时代的数学推理综述：基准测试、方法与挑战. 载于计算语言学协会论文集：ACL 2025, 页码 11798–11827, 2025.

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Appendix organization. This appendix is organized into five parts. *Part I* (Section A) is the dataset-side documentation: Datasheet, including a reproducibility section. *Part II* (Sections B–C) specifies the benchmark testbed. *Part III* (Section D) expands the PAVE pipeline compressed in Section 2. *Part IV* (Section E) reports the two-annotator audit on a random 100-case subsample and documents the calibration split used by the per-node anomaly screen. *Part V* (Sections F–G) documents the evaluated RCA agents and the formal metric definitions.

A Datasheet for Datasets

A.1 Motivation

OpenRCA 2.0 is designed for *process-level* evaluation of LLM-based RCA agents. The goal is to assess not only whether an agent identifies the correct root cause, but also whether it can justify that diagnosis with a faithful step-wise causal propagation path. Existing RCA benchmarks typically supervise only the final outcome label, which hides the *ungrounded diagnosis* failure mode studied in this paper. No dedicated external funding was used specifically to compile the released benchmark artifact beyond the broader research effort supporting this study.

A.2 Composition

Each instance contains three components: (i) multimodal pre- and post-injection telemetry, including distributed traces, metric time series, and structured logs in parquet format; (ii) a structured ground-truth label specifying the originating system, the injected component, and the fault type; and (iii) a step-wise causal propagation graph over services, pods, and spans (Section 2). OpenRCA 2.0 contains 500 evaluable instances drawn from 3 architecturally distinct microservice systems and covering 27 chaos primary kinds; per-system breakdowns are reported in Table 1. Curation uses two-pass stratified selection on (system, chaos family, root service) with per-stratum caps, sorted by (longest path, edge count, services), and retains only instances that pass end-to-end causal-path verification while excluding cyclic service graphs, trivial propagation depth (≤ 1), and frontend/loadgen entry points. The result is a curated, self-contained evaluation set whose sampling objective is balanced coverage rather than statistical representativeness of production incidents.

The current release does not define train/development/test splits. Retained instances are distributed as complete telemetry packs rather than field-pruned records, so field-level missingness or redaction is not a concern within the released trace, metric, or log data. Because the benchmark consists of synthetic operational telemetry generated under controlled load, it contains no PII or human-subject data; people-related subpopulation, identity, and sensitive-attribute considerations are therefore not applicable.

A.3 Collection Process

Faults are injected on three microservice systems: TrainTicket [5], the OpenTelemetry Demo application [11], and Hotel Reservation from DeathStarBench [10]. These interventions are run with Chaos Mesh on a shared 7-node Kubernetes cluster; per-system topology and deployment details are given in Section B, and the cluster configuration in Section D.4. For each intervention, a fresh system instance is deployed and warmed up, a pre-injection baseline window is recorded, the fault is applied over $[t_0, t_0{+}\Delta t]$, and post-injection telemetry is then collected. Step-wise causal labels are produced automatically by the PAVE pipeline (Sections 2.2, 2.3) and are further checked by the forward verification protocol described later in this appendix.

Reliability of this procedure was assessed by a two-annotator audit on a uniform random 100-case subsample, with 94% case-level agreement (Section 3.4; full protocol in Appendix E). The annotators were two of the paper's authors; no crowdworkers or paid external annotators were used. Because the benchmark is built from synthetic system telemetry rather than data about natural persons, consent, participant notification, and formal human-subject review considerations do not apply.

A.4 Preprocessing, Cleaning, Labeling

Telemetry is exported through OpenTelemetry into standardized parquet schemas, with one file per modality per period. The released benchmark contains only injections that pass the joint screen

附录结构。本附录分为五个部分。第一部分（第 A 节）是数据集侧文档：数据表，包括一个可复现性部分。第二部分（第 B–C 节）规定了基准测试平台。第三部分（第 D 节）展开介绍在第 2 节中压缩描述的 PAVE 流水线。第四部分（第 E 节）报告了在随机100例子样本上的双标注员审计，并记录了每节点异常筛查所使用的校准划分。第五部分（第 F–G 节）记录了被评估的 RCA 智能体和正式指标定义。

数据集数据表

A.1 动机

OpenRCA 2.0 旨在对基于大语言模型的RCA代理进行过程级评估。其目标不仅是评估代理是否识别出正确的根本原因，还要评估它是否能够通过忠实的逐步因果传播路径来证明该诊断的合理性。现有的根因分析基准通常仅监督最终结果标签，这掩盖了本文所研究的无根据诊断故障模式。除了支持本研究的更广泛的研究工作外，没有专门的外部资金用于编译发布的基准工件。

A.2 组成

每个实例包含三个组成部分：(i) 多模态注入前后遥测数据，包括Parquet格式的分布式追踪、指标时间序列和结构化日志；(ii) 结构化真实标签，指定起源系统、注入组件和故障类型；以及 (iii) 关于服务、Pod和跨度的逐步因果传播图（第 2 节）。OpenRCA 2.0 包含 500 个可评估实例，这些实例源自 3 个架构上截然不同的微服务系统，涵盖 27 种混沌主要类型；按系统细分的数据见表 1。数据整理采用针对（系统、混沌家族、根服务）的两遍分层选择并设置每层上限，按（最长路径、边数、服务）排序，仅保留通过端到端因果路径验证的实例，同时排除循环服务图、琐碎传播深度（≤ 1）以及前端/负载生成器入口点。最终得到一个经过整理的、自包含的评估集，其采样目标是实现平衡覆盖，而非对生产事故具有统计代表性。

当前版本未定义训练/开发/测试划分。保留的实例以完整的遥测数据包形式分发，而非字段修剪记录，因此发布的追踪、指标或日志数据中不存在字段级缺失或遮蔽问题。由于该基准测试由在受控负载下生成的合成运行遥测数据组成，因此不包含个人身份信息或人类受试者数据；因此，与人员相关的子群体、身份和敏感属性考量并不适用。

附录A.3 收集过程

故障注入于三个微服务系统：TrainTicket [5], OpenTelemetry演示应用程序 [11],以及来自DeathStarBench的HotelReservation [10]。这些干预通过Chaos Mesh在共享的7节点Kubernetes集群上运行；每个系统的拓扑和部署详情见B节，集群配置见D.4节。对于每次干预，都会部署并预热一个新的系统实例，记录注入前基线窗口，在 $[t_0, t_0{+}\Delta t]$期间应用故障，然后收集注入后的遥测数据。分步因果标签由PAVE流水线（第2.2、2.3节）自动生成，并进一步通过本附录稍后描述的前向验证协议进行检查。

该程序的可靠性通过在均匀随机100例子样本上进行的双标注员审计进行评估，案例级一致性达到94%（第3.4节；完整方案见附录E）。标注员为本文的两位作者；未使用众包工人或付费外部标注员。由于该基准测试的数据来源为合成系统遥测数据而非关于自然人的数据，因此不涉及同意、参与者通知以及正式的人类受试者审查考量。

A.4 预处理、清洗与标注

遥测数据通过 OpenTelemetry 导出为标准化的 Parquet架构，每个模态在每个周期对应一个文件。发布的基准测试仅包含通过联合筛选的注入。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

described in Section 2.3 (observable SLO impact, a rule-admitted propagation path, and a clean pre-injection baseline); cases failing any condition are dropped during pipeline-level analysis. The released package bundles the evaluation telemetry together with the derived labels and propagation graphs, so the released artifact is the processed benchmark itself rather than an undocumented intermediate product.

Cleaning is performed primarily at the instance level by filtering out runs whose pre-injection baseline is itself anomalous, rather than by manually editing fields inside retained telemetry records. The software responsible for preprocessing, filtering, and label construction is the released PAVE pipeline described in Section D. Pipeline reliability rests on the conjunctive verification design analyzed in Section D.8 and is corroborated by the two-annotator audit reported in Section 3.4 (Appendix E.1).

A.5 Uses

Intended uses include (i) process-level evaluation of LLM-based RCA agents and (ii) training process reward models in the reliability domain, where the step-wise edges provide natural supervision targets. At the time of submission, the benchmark is used only in this study as the primary evaluation suite for process-level RCA assessment.

The benchmark is not intended for production-readiness certification of any specific RCA agent, for cross-architecture generalization claims to service-mesh, serverless, or fully event-driven systems beyond the three included testbeds, or for claims about faults that leave no telemetry signature. It should also not be used as a training corpus for the *evaluated* RCA agents themselves, as doing so would create direct test-set leakage.

A.6 Distribution and License

An anonymized preview of the benchmark accompanies the supplementary submission, and the anonymized full benchmark package is provided in the submission materials. Upon acceptance, the full benchmark artifact and the PAVE pipeline will be released publicly with archival metadata and integrity checksums. The released benchmark is a derived artifact constructed from a curated subset of the upstream fault-injection runs together with the additional annotations and causal-process labels introduced in this work. The dataset is released under **CC-BY-SA 4.0**, and the pipeline code under **Apache 2.0**. To the best of our knowledge, no export-control or comparable regulatory restrictions apply to the released artifact.

A.7 Maintenance

To preserve anonymity in the submission version, direct maintainer contact details are omitted here and will be restored upon acceptance. Corrections, additional fault types, and cross-system extensions will be managed through versioned releases, and older public versions will remain archived for reproducibility. After public release, questions and fixes can be tracked through the GitHub issue tracker and pull requests. External contributions will be reviewed for schema compatibility, reproducibility, and annotation consistency before inclusion in an official release.

A.8 Reproducibility

The complete PAVE pipeline source is released alongside the dataset, with explicit random seeds for every stochastic component (DFS tie-breaking, baseline-window selection, fault-injection scheduling) so the numbers in this paper are regeneratable; pipeline hyperparameters and detector thresholds are tabulated in Sections D.5 and D.8, and cluster configuration is in Section D.4. A single-entry-point `reproduce.sh` accompanies the anonymized submission artifact and regenerates the Section 3.2 numbers on a downloadable instance subset; full-dataset re-evaluation requires API quota for the 11 models documented in Section F.5. No pre-trained models are released as part of the benchmark.

B Benchmark Systems and Topology

This section details the microservice systems used in our benchmark, including their architecture, scale, and the service dependency topology that defines the structural prior $\mathcal{G}$ for causal path extraction.

第 2.3 节中所述（可观测的 SLO 影响、规则准入的传播路径以及干净的注入前基线）；在流水线级分析中，未能满足任一条件的案例将被舍弃。发布的软件包将评估遥测数据与推导出的标签及传播图捆绑在一起，因此发布产物是经过处理的基准测试本身，而非未经记录的中间产物。

清洗工作主要在实例级别执行，通过过滤掉注入前基线本身即异常的运次来实现，而非手动编辑保留遥测数据记录中的字段。负责预处理、过滤和标签构建的软件是第D节中描述发布的PAVE流水线。流水线的可靠性依赖于第D.8节中分析的合取验证设计，并得到了第3.4节（附录 E.1）中报告的双标注员审计的佐证。

附录 A.5 用途

预期用途包括（i）对基于大语言模型的根因分析智能体进行过程级评估，以及（ii）在可靠性领域训练过程奖励模型，其中逐步生成的边提供了自然的监督目标。在提交时，该基准测试仅在本研究中用作过程级根因分析评估的主要评估套件。

该基准测试并非用于对任何特定 RCA 代理进行生产就绪认证，也不用于声称在除三个包含的测试平台之外的服务网格、无服务器或完全事件驱动系统中具备跨架构泛化能力，亦非用于声称那些不留下遥测特征的故障相关的能力。此外，不应将其用作被评估的 RCA 代理本身的训练语料库，因为这样做会导致直接的测试集泄露。

A.6 分发与许可证

随补充提交材料附带该基准测试的匿名预览版，完整匿名基准测试包也包含在提交材料中。一旦录用，完整的基准测试工件和 PAVE 流水线将连同归档元数据和完整性校验和一同公开发布。发布的基准测试是一个衍生工件，由经过筛选的上游故障注入运行子集以及本工作中引入的额外标注和因果过程标签构建而成。该数据集以 CC-BY-SA 4.0 许可证发布，流水线代码以 Apache 2.0 许可证发布。据我们所知，发布的产物不受出口管制或类似法规限制。

A.7 维护

为在投稿版本中保持匿名性，此处省略维护者直接联系方式，并将在录用后恢复。更正、附加故障类型及跨系统扩展将通过版本化发布进行管理，旧的公开版本将继续归档以确保可复现性。公开发布后，问题和修复可通过GitHub问题跟踪器和拉取请求进行跟踪。在纳入官方发布之前，外部贡献将针对模式兼容性、可复现性和标注一致性进行审核。

A.8 可复现性

PAVE流水线的完整源代码随数据集一同发布，并为每个随机组件（深度优先搜索平局决胜、基线窗口选择、故障注入调度）显式指定了随机种子，以确保本文中的数值可复现；流水线超参数和检测器阈值列于D.5节和D.8节，集群配置位于第D.4节。一个单入口点reproduce.sh脚本随匿名提交的产物一同提供，可在可下载的实例子集上重新生成第3.2节中的数值；对完整数据集重新进行评估需要第F.5节中记录的11个模型的API配额。作为基准测试的一部分，未发布任何预训练模型。

B 基准系统与拓扑

本节详细介绍我们基准测试中使用的微服务系统，包括其架构、规模，以及定义因果路径提取的结构先验 $\mathcal{G}$ 的服务依赖拓扑。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

B.1 Systems Overview

The benchmark spans 3 open-source microservice testbeds chosen to cover distinct propagation regimes: TrainTicket [5] for deep, layered Java/REST call chains; the OpenTelemetry Demo application [11] for polyglot gRPC services with asynchronous Kafka legs; and Hotel Reservation from DeathStarBench [10] for shallow, wide Go/gRPC services with shared in-memory caches. Together they cover three communication styles (synchronous REST, synchronous gRPC, and asynchronous queueing) and four data-store classes (relational, document, key-value cache, and message log), so the recovered ground-truth paths reflect propagation patterns that no single system would expose on its own. The subsections below summarize the topology of each system; deployment manifests, image tags, and commit hashes are released alongside the dataset.

B.2 TrainTicket Service Dependency Topology

TrainTicket's 44 microservices follow a layered architecture centered on REST-style service-to-service calls, with RabbitMQ used for asynchronous messaging and a shared MySQL instance supporting many backend services. At a coarse level, requests flow from user-facing entry services through gateway and orchestration layers into domain services and then into shared infrastructure, creating long dependency chains and frequent opportunities for multi-hop propagation.

The system exhibits several properties that make RCA challenging:

- **Deep call chains**: Booking operations traverse up to 6 service layers (Entry → Gateway → Orchestration → Domain → Foundation → Infrastructure).
- **High fan-in**: Foundation services (`Station`, `Route`, `Train`) are shared by many upstream callers, creating complex dependency patterns where a single failure can manifest in multiple symptom locations.
- **Shared infrastructure**: 23 services connect to a shared MySQL 5.7 instance, introducing vertical propagation paths from database-level faults to application-level errors.
- **Circular dependencies**: Some services exhibit mutual dependencies (e.g., `Travel` ↔ `Seat`) that require temporal unrolling for causal analysis.

B.3 OpenTelemetry Demo Service Dependency Topology

The OpenTelemetry Demo is a polyglot e-commerce reference application comprising 15 application services implemented across Go, Python, Java, .NET, Ruby, and JavaScript. The deployed stack also includes supporting components such as Kafka, Postgres, valkey/Redis, and observability infrastructure, which are not counted as application services. Inter-service communication uses gRPC for synchronous request paths (e.g., `cart` → `checkout` → `payment`) and Kafka for asynchronous fan-out (e.g., `checkout` → `accounting` / `fraud-detection`); state is held in Postgres, valkey/Redis, and the Kafka log itself.

Properties relevant for fault propagation:

- **Polyglot error semantics**: each language stack surfaces failures differently (Java stack traces, Go panics, Python tracebacks, .NET exceptions), exercising the rule set across heterogeneous log formats.
- **Synchronous + asynchronous mix**: faults injected upstream of the Kafka boundary propagate as queue back-pressure rather than RPC timeouts, producing a propagation signature absent from the REST-only TrainTicket.
- **Shared infrastructure**: Postgres and valkey are accessed by multiple services; an injected database fault produces correlated anomalies across otherwise unrelated request paths.

B.4 Hotel Reservation Service Dependency Topology

Hotel Reservation, drawn from DeathStarBench [10], consists of 9 Go services that communicate via gRPC and rely on MongoDB and Memcached.

Properties relevant for fault propagation:

- **Shallow but wide call chains**: the maximum service-to-service depth is 3 hops (`frontend` → `search` → `profile/geo/rate`), with high fan-out at the search service.

B.1 系统概览

该基准测试涵盖三个开源微服务测试平台，旨在覆盖不同的传播机制：TrainTicket [5]用于具有深层分层Java/REST调用链的系统；OpenTelemetry演示 [11] 用于包含异步Kafka链路的多语言gRPC服务；以及来自DeathStarBench [10] 的酒店预订系统，用于具有浅层宽泛Go/gRPC服务及共享内存缓存的系统。它们共同涵盖了三种通信方式（同步REST、同步gRPC和异步队列）以及四类数据存储（关系型、文档型、键值缓存和消息日志），因此恢复的真实路径反映了任何单一系统都无法独立展现的传播模式。以下小节概述了每个系统的拓扑；部署清单、镜像标签和提交哈希与数据集一同发布。

B.2 TrainTicket服务依赖拓扑

TrainTicket的44个微服务遵循以REST式服务间调用为核心的分层架构，使用RabbitMQ进行异步消息传递，并采用共享MySQL实例支持许多后端服务。在较粗粒度的级别上，请求从面向用户的入口服务经网关和编排层流向领域服务，再进入共享基础设施，从而形成长依赖链以及多跳传播的频繁机会。
该系统展现了若干使根因分析颇具挑战性的特性：

- 深层调用链：预订操作最多穿越 6 层服务（入口 → 网关 →编排 →领域 →基础 →基础设施）。
- 高扇入：基础服务（车站服务、路线服务、列车服务）被许多上游调用方共享，形成了复杂的依赖模式，使得单一故障可能在多个症状位置显现。
  共享基础设施：23 个服务连接到一个共享的 MySQL 5.7 实例，引入了从数据库级别故障到应用级别错误的垂直传播路径。
- 循环依赖：某些服务表现出相互依赖关系（例如，旅行 ↔座位），这些关系在进行因果分析时需要时间展开。

B.3 OpenTelemetry演示服务依赖拓扑

OpenTelemetry演示是一个多语言电子商务参考应用，由分布在Go、Python、Java、.NET、Ruby和JavaScript中的15个应用服务组成。部署的栈还包括Kafka、PostgreSQL、Valkey/Redis以及可观测性基础设施等支持组件，这些组件不计入应用服务。服务间通信在同步请求路径中使用gRPC（例如，cart →checkout →payment），在异步扇出中使用Kafka（例如，checkout → accounting / fraud-detection）；状态保存在PostgreSQL、Valkey/Redis以及Kafka日志本身中。

故障传播相关属性：

多语言错误语义：每种语言栈以不同方式呈现故障（Java 堆栈跟踪、Go 恐慌错误、Python 回溯信息、.NET 异常），从而在异构日志格式上检验规则集。

- 同步与异步混合：注入至 Kafka 边界上游的 + 故障以队列背压而非 RPC超时传播，从而产生了一种在仅基于REST的TrainTicket中不存在的传播特征。
- 共享基础设施：PostgreSQL和Valkey被多个服务访问；注入的数据库故障会在原本无关的请求路径之间产生相关异常。

B.4 HotelReservationService依赖拓扑

Hot来自DeathStarBench [10], 的Reservation服务由9个通过通信的Go服务组成 a gRPC并依赖MongoDB和Memcached。
与故障传播相关的属性：

- 浅而宽的调用链：服务间深度的最大值为3跳（前端 → 搜索 → 个人资料/地理位置/评价），且在搜索服务处存在高扇出。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 4: Ground truth label schema. Each fault injection instance is annotated with a structured label across multiple granularity levels. Cardinalities are system-dependent; example values are drawn from TrainTicket for concreteness.

Field	Description	Cardinality	Example Value
`service`	Target microservice	system-dependent	`ts-order-service`
`pod`	Target Kubernetes Pod	system-dependent	`ts-order-service-7b8f4-xk9z2`
`container`	Target container within Pod	system-dependent	`ts-order-service`
`function`	Target function or handler (code-level faults)	system-dependent	`com.order.service.impl.create`
`span`	Target trace span (HTTP faults)	system-dependent	`POST /api/v1/orderservice/order`
`metric`	Anomalous golden signal	system-dependent	`http.server.request.duration`

- **In-memory cache layer**: Memcached sits between every read-heavy service and its MongoDB backing store; cache faults degrade many downstream services in parallel.
- **Terse error surface**: Go services return gRPC status codes directly rather than wrapping errors in stack traces, so log signals are sparser and trace-side evidence carries a comparatively larger share of the verification load.

B.5 Propagation Channels and Telemetry Modalities

The dependency graph $\mathcal{G}$ supports two propagation channels that the verification pipeline must distinguish. *Horizontal propagation* follows RPC call chains: a slow `Order` service causes its upstream callers to time out, producing cascading latency spikes along the call graph. *Vertical propagation* crosses abstraction layers: CPU throttling on a Kubernetes node degrades all co-located services simultaneously, even those with no mutual RPC dependency. A single injection can trigger both channels, creating multi-hop paths that mix horizontal and vertical edges.

These channels leave distinct signatures in the three telemetry modalities, which Phase 2 verification (Section 2.3) exploits. *Traces* record per-request call trees with span-level latencies and status codes; horizontal propagation manifests as anomalous parent→child span timing that reveals the cascade direction. *Metrics* are per-service and per-host time series (CPU usage, p99 latency, error rate); vertical propagation appears as correlated anomalies among co-located services sharing a throttled resource. *Logs* are semi-structured text events (e.g., `[ERROR] upstream timeout`) that supply causal context absent from numerical signals alone.

C Root Cause Taxonomy

This section formalizes the ground truth label schema and provides the complete fault type catalog.

C.1 Ground Truth Schema

Each fault injection instance is annotated with a structured ground truth label that specifies the root cause at multiple granularity levels. Table 4 shows the schema fields; cardinalities vary by system, and TrainTicket values are shown as concrete examples.

The multi-level annotation enables evaluation at different abstraction levels. Service-level evaluation ($|\mathcal{V}| = 44$) is the most commonly reported granularity in prior work. Finer-grained levels (function, span) provide more precise localization but increase label space complexity substantially.

C.2 Complete Fault Type Catalog

Table 5 presents the fault types implemented in our chaos engineering framework, organized by Chaos Mesh CRD category. Each fault type is classified by:

- **Target layer**: Whether the fault targets infrastructure resources (Pod, network, CPU/memory) or application logic (HTTP endpoints, JVM methods, database queries).
- **Applicable ground truth levels**: Which fields of the ground truth schema are relevant. Infrastructure faults are labeled at the `service`/`pod` level, while application faults include `function` or `span` annotations.

表 4：真实标签模式。每个故障注入实例都标注有涵盖多个粒度级别的结构化标签。基数是系统依赖的；示例值取自 TrainTicket 以增强具体性。

字段	描述	基数	示例值
服务	目标微服务	系统依赖	ts-order-service
pod	目标 Kubernetes Pod	系统依赖	ts-order-service-7b8f4-xk9z2
容器	Pod 内的目标容器	系统依赖	ts-order-service
函数	目标函数或处理程序（代码级别的故障）	系统依赖	com.order.service.impl.create
span	目标跟踪跨度（HTTP 故障）	系统依赖	POST /api/v1/orderservice/order
指标	异常黄金信号	系统依赖	http.server.request.duration

- 内存缓存层：Memcached 位于每个读密集型服务与其 MongoDB 后端存储之间；缓存故障会并行导致许多下游服务性能下降。
- 简洁错误接口：Go服务直接返回gRPC状态码，而不是将错误封装在堆栈跟踪中，因此日志信号更为稀疏，链路侧证据在验证负载中占据相对较大的比重。

B.5 传播通道与遥测模态

依赖图 $\mathcal{G}$ 支持验证流水线必须区分的两种传播通道。水平传播沿 RPC 调用链进行：缓慢的订单服务会导致其上游调用方超时，从而在调用图上产生级联延迟尖峰。垂直传播跨越抽象层：Kubernetes 节点上的 CPU 限流会同时降低所有共存服务的性能，即使这些服务之间不存在相互 RPC 依赖。单次注入可同时触发这两种通道，形成混合水平边和垂直边的多跳路径。

这些通道在三种遥测模态中留下不同的特征，第二阶段验证（第2.3节）利用这些特征。追踪记录每个请求的调用树，包含跨度级延迟和状态码；水平传播表现为父级→子级跨度时序异常，从而揭示级联方向。指标是每个服务和每个主机的时间序列（CPU使用率、P99延迟、错误率）；垂直传播表现为共享受限资源的共存服务之间的相关异常。日志是半结构化文本事件（例如 [ERROR] upstream timeout），提供了仅靠数值信号无法获得的因果上下文。

C 根因分类法

This 本节对真实标签模式进行了规范化，并提供了完整故障类型目录。 .

C.1 真实数据模式

每个故障注入实例都标注有一个结构化基准真值标签，该标签以多个粒度级别指定根本原因。表 4 展示了模式字段；基数因系统而异，并以 TrainTicket 值作为具体示例。

多层级标注支持在不同抽象级别进行评估。服务级评估（$|\mathcal{V}| = 44$）是先前工作中最常报告的粒度。更细粒度的级别（函数、跨度）能够提供更精确的定位，但会显著增加标签空间复杂度。

C.2 完整故障类型目录

表 5 展示了我们混沌工程框架中实现的故障类型，按 Chaos Mesh CRD 类别组织。每种故障类型按以下方式分类：

- 目标层：故障是作用于基础设施资源（Pod、网络、CPU/内存）还是应用逻辑（HTTP 端点、JVM 方法、数据库查询）。
- 适用的基准真值级别：真实值模式中的哪些字段是相关的。基础设施故障在服务/Pod 级别进行标注，而应用故障则包含函数或跨度标注。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 5: Fault type taxonomy across 7 Chaos Mesh CRD categories. "Applicable Levels" lists the ground-truth fields relevant to each fault type.

Category	Fault Type	Parameters	Target Layer	Applicable Levels	Propagation Channel
PodChaos	PodKill	Kill signal (SIGKILL)	Infrastructure	service, pod	Vertical
	PodFailure	Unavailability duration	Infrastructure	service, pod	Vertical
	ContainerKill	Target container	Infrastructure	service, pod, container	Vertical
StressChaos	CPUStress	Core count, load %	Infrastructure	service, pod	Vertical
	MemoryStress	Memory size (MB)	Infrastructure	service, pod	Vertical
HTTPChaos	HTTPRequestAbort	Abort code, target path	Application	service, span	Horizontal
	HTTPResponseAbort	Abort code, target path	Application	service, span	Horizontal
	HTTPRequestDelay	Delay (ms), target path	Application	service, span	Horizontal
	HTTPResponseDelay	Delay (ms), target path	Application	service, span	Horizontal
	HTTPResponseReplaceBody	Body type {empty, random}	Application	service, span	Horizontal
	HTTPResponsePatchBody	Patch content	Application	service, span	Horizontal
	HTTPRequestReplacePath	New path	Application	service, span	Horizontal
	HTTPRequestReplaceMethod	Method {GET, POST, ...}	Application	service, span	Horizontal
	HTTPResponseReplaceCode	Status code	Application	service, span	Horizontal
DNSChaos	DNSError	Target hostname	Infrastructure	service, pod	Vertical
	DNSRandom	Target hostname	Infrastructure	service, pod	Vertical
NetworkChaos	NetworkDelay	Delay (ms), jitter	Infrastructure	service, pod	Vertical + Horizontal
	NetworkLoss	Loss rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkDuplicate	Duplicate rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkCorrupt	Corrupt rate (%)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkBandwidth	Rate (bps)	Infrastructure	service, pod	Vertical + Horizontal
	NetworkPartition	Partition direction	Infrastructure	service, pod	Vertical + Horizontal
TimeChaos	TimeSkew	Offset (ms)	Infrastructure	service, pod	Vertical
JVMChaos	JVMLatency	Delay (ms), target class.method	Application	service, function	Horizontal
	JVMReturn	Return value, type {String, Int}	Application	service, function	Horizontal
	JVMException	Exception class, target method	Application	service, function	Horizontal
	JVMGarbageCollector	—	Application	service, pod	Vertical
	JVMCPUStress	CPU count	Application	service, pod	Vertical
	JVMMemoryStress	Type {heap, stack}, size	Application	service, pod	Vertical
	JVMMySQLLatency	Delay (ms), target method	Application	service, function	Horizontal
	JVMMySQLException	Exception, target method	Application	service, function	Horizontal

Table 6: Mapping from raw `chaos_type` (Table 5) to the observable fault-kind labels used at evaluation time. Rows listing multiple source identifiers are many-to-one collapses where the listed mechanisms produce indistinguishable telemetry signatures; the remaining rows map one-to-one.

Fault kind	Source chaos_type(s)	Primary observation channel
Pod / container lifecycle		
`pod_failure`	`PodKill`, `ContainerKill`	trace emission gap, brief, recovers in window
`pod_unavailable`	`PodFailure`	trace emission gap, sustained to window end
L3/L4 network (direction recorded in GT)		
`network_delay`	`NetworkDelay`	`container.network.*`, trace latency tail
`network_loss`	`NetworkLoss`	`container.network.*`, trace retry pattern
`network_partition`	`NetworkPartition`	`container.network.*`, timeout share approaches one
`network_corrupt`	`NetworkCorrupt`	`container.network.*`, TCP retransmit signal
`network_duplicate`	`NetworkDuplicate`	`container.network.*`, duplicate-ack pattern
`network_bandwidth_limit`	`NetworkBandwidth`	`container.network.*`, throughput drop
HTTP application layer		
`http_aborted`	`HTTPRequestAbort`, `HTTPResponseAbort`	HTTP span status, request-failure rate
`http_slow`	`HTTPRequestDelay`, `HTTPResponseDelay`	HTTP span duration distribution
`http_payload_modified`	`HTTPResponseReplaceBody`, `HTTPResponsePatchBody`, `HTTPRequestReplacePath`, `HTTPRequestReplaceMethod`	response body / path / method differs from baseline
`http_response_status_modified`	`HTTPResponseReplaceCode`	HTTP span status-code distribution
Resource exhaustion		
`cpu_stress`	`CPUStress`	`container.cpu.*` saturated
`mem_stress`	`MemoryStress`	`container.memory.*` saturated
`jvm_thread_cpu_stress`	`JVMCPUStress`	`jvm.cpu.*` elevated, container CPU normal
`jvm_heap_stress`	`JVMMemoryStress`	`jvm.memory.used` climbs to limit
`jvm_gc_pressure`	`JVMGarbageCollector`	`jvm.gc.duration` histogram spikes
Code-level JVM		
`jvm_method_exception`	`JVMException`	log exception class on a specific method
`jvm_jdbc_exception`	`JVMMySQLException`	exception on a JDBC client call
`jvm_method_latency`	`JVMLatency`	elevated span duration on a method
`jvm_jdbc_latency`	`JVMMySQLLatency`	elevated span duration on a JDBC call
`jvm_method_mutated`	`JVMReturn`, `JVMRuntimeMutator`	method returns a value differing from baseline
DNS / clock		
`dns_resolution_failed`	`DNSError`, `DNSChaos`	DNS error log entries on the affected service
`dns_resolution_wrong`	`DNSRandom`	resolution succeeds but returns wrong target
`clock_skew`	`TimeSkew`, `TimeChaos`	cross-service timestamp drift in correlated logs

- **Propagation channel**: Whether the fault propagates horizontally (between logical entities via RPC) or vertically (from infrastructure to logical layers via resource contention), or both.

C.3 Observable Fault-Kind Vocabulary for Evaluation

The chaos-mesh API names in Table 5 describe *how* a fault is injected, not what an agent observes from telemetry; in many cases distinct injection mechanisms produce indistinguishable signatures in the parquet data (for example `HTTPRequestAbort` and `HTTPResponseAbort` both surface as request failures from the client side, and `PodKill` and `ContainerKill` both manifest as a brief gap in the affected service's trace emission). An agent therefore cannot recover the raw chaos-mesh API name from the released telemetry alone. We define an observable fault-kind vocabulary by relabeling

表 5：跨越 7 个 ChaosMesh CRD 类别的故障类型分类法。"适用层级"列出了与每个故障类型相关的真实标签字段。

类别	故障类型	参数	目标层	适用层级	传播通道
PodChaos	Pod杀死	终止信号（SIGKILL）	基础设施	服务, Pod	垂直
	Pod故障	不可用持续时间	基础设施	服务, Pod	垂直
	容器杀死	目标容器	基础设施	服务、Pod、容器	垂直
压力混沌	CPU 压力	核心数，负载百分比	基础设施	服务, Pod	垂直
	内存压力	内存大小 (MB)	基础设施	服务, Pod	垂直
HTTP 混沌	HTTP请求中止	中断代码，目标路径	应用	服务，跨度	水平
	HTTP响应中止	中断代码，目标路径	应用	服务，跨度	水平
	HTTP 请求延迟	延迟 (ms), 目标路径	应用	服务，跨度	水平
	HTTP 响应延迟	延迟 (ms), 目标路径	应用	服务，跨度	水平
	HTTP 响应替换主体	正文类型 {空, 随机}	应用	服务，跨度	水平
	HTTP 响应修补主体	补丁内容	应用	服务，跨度	水平
	HTTP 请求替换路径	新路径	应用	服务，跨度	水平
	HTTP 请求替换方法	方法 {GET, POST, ...}	应用	服务，跨度	水平
	HTTPResponseReplaceCode	状态码	应用	服务，跨度	水平
域名系统混沌	域名系统错误	目标主机名	基础设施	服务, Pod	垂直
	DNS随机性	目标主机名	基础设施	服务, Pod	垂直
网络混沌	网络延迟	延迟 (毫秒)，抖动	基础设施	服务, Pod	垂直 + 水平
	网络丢包	丢包率 (%)	基础设施	服务, Pod	垂直 + 水平
	网络包重复	重复率 (%)	基础设施	服务, Pod	垂直 + 水平
	网络数据损坏	损坏率 (%)	基础设施	服务, Pod	垂直 + 水平
	网络带宽	速率 (bps)	基础设施	服务, Pod	垂直 + 水平
	网络分区	分区方向	基础设施	服务, Pod	垂直 + 水平
时间混沌	时间偏差	偏移量（毫秒）	基础设施	服务, Pod	垂直
JVM混沌	JVM 延迟	延迟 (毫秒), 目标类.方法	应用	服务，函数	水平
	JVM 返回	返回值，类型 {字符串, 整数}	应用	服务，函数	水平
	JVM异常	异常类，目标方法	应用	服务，函数	水平
	JVM垃圾收集器	—	应用	服务, Pod	垂直
	JVM CPU 压力	CPU数量	应用	服务, Pod	垂直
	JVM内存压力	类型 {堆, 栈}，大小	应用	服务, Pod	垂直
	JVM MySQL 延迟	延迟 (毫秒), 目标方法	应用	服务，函数	水平
	JVM MySQL异常	异常，目标方法	应用	服务，函数	水平

表 6：从原始混沌_类型（表 5）到评估时间使用的可观测故障种类标签的映射。列出了多个来源标识符的行表示多对一归并，其中列出的机制产生不可区分的遥测签名；其余行则为一对一映射。

故障类型	源混沌类型	主要观测通道
Pod/容器生命周期		
Pod故障	Pod杀死，容器杀死	追踪发射间隙，短暂，窗口内恢复
Pod不可用	Pod故障	追踪发射间隙，持续至窗口结束
L3/L4网络（方向记录在真实数据中）		
网络延迟	网络延迟	容器网络.*，追踪延迟尾部
网络丢包	网络丢包	容器网络.*，追踪重试模式
网络分区	网络分区	容器网络.*，超时占比接近→
网络数据损坏	网络数据损坏	容器网络.*，TCP重传信号
网络数据包重复	网络包重复	container.network.*，duplicate-ack pattern
网络带宽限制	网络带宽	容器网络.*，吞吐量下降
HTTP 应用层		
HTTP中断	HTTP请求中止, HTTP响应中止	HTTP 跨度状态, 请求失败率
HTTP缓慢	HTTP 请求延迟, HTTP 响应延迟	HTTP 跨度持续时间分布
HTTP载荷修改	HTTP 响应替换主体、HTTP 响应修补主体、HTTP 请求替换路径、HTTP 请求替换方法	响应体 / 路径 / 方法与基线不同
HTTP响应状态修改	HTTPResponseReplaceCode	HTTP 跨度状态码分布
资源耗尽		
CPU压力	CPU 压力	容器.cpu.* 饱和
内存压力	内存压力	容器.memory.* 饱和
JVM线程CPU压力	JVM CPU 压力	jvm.cpu.* 升高，容器CPU 正常
JVM堆压力	JVM内存压力	jvm.memory.used 攀升至限制
JVM GC压力	JVM垃圾收集器	jvm.gc.duration 直方图尖峰
代码级JVM		
JVM方法异常	JVM异常	在特定方法上记录异常类
JVM JDBC异常	JVM MySQL异常	JDBC 客户端调用异常
JVM方法延迟	JVM 延迟	方法跨度持续时间升高
JVM JDBC延迟	JVM MySQL 延迟	JDBC 调用跨度持续时间升高
JVM方法变异	JVM 返回，JVM 运行时变异器	方法返回与基准不同的值
域名系统 / 时钟		
DNS解析失败	域名系统错误，域名系统混沌	受影响服务上的域名系统错误日志条目
DNS解析错误	DNS随机性	解析成功但返回错误目标
时钟偏差	时间偏差，时间混沌	关联日志中的跨服务时间戳漂移

- 传播通道：故障是水平传播（逻辑实体之间通过远程过程调用）还是垂直传播（从基础设施到逻辑层通过资源争用），抑或两者兼有。

C.3 评估用可观测故障类型词汇表

表 5 中的 chaos-mesh API 名称描述的是故障的注入方式，而非代理从遥测数据中观察到的内容；在许多情况下，不同的注入机制会在 Parquet 数据中产生无法区分的特征（例如，HTTP 请求中止和 HTTP响应中止从客户端侧看都表现为请求失败，而 Pod杀死和容器杀死都表现为受影响服务追踪数据发送中的短暂中断）。因此，代理无法仅从发布的遥测数据中恢复原始的 chaos-mesh API 名称。我们通过重新标注来定义可观测故障类型词汇表

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

each chaos mechanism according to the feature signature it produces in metrics, traces, and logs, and judge an agent correct when its predicted label matches this observable label rather than the underlying API name. Table 6 lists the mapping from raw `chaos_type` to observable label together with the primary signal carrying each label; the same projection is applied at ground-truth extraction and at agent-prediction matching so both sides of the comparison share one vocabulary.

D PAVE Pipeline and Experimental Details

This appendix details (i) the pipeline internals compressed in Section 2 for space (Sections D.1–D.3, plus the rule set, hyperparameters, and detector thresholds in Sections D.6, D.5, D.8), and (ii) the operational and calibration details supporting reproduction (Sections D.4–D.8).

D.1 Structural Causal Model

Fault propagation over $\mathcal{G}$ [8] is realized at each node as

$$S_i^{(t)} = f_i\big(\mathbf{S}_{Pa(v_i)}^{(<t)},\, U_i^{(t)}\big), \tag{2}$$

where $\mathbf{S}_{Pa(v_i)}^{(<t)}$ are the recent states of v_i's causal parents and $U_i^{(t)}$ is an exogenous shock. The agent's observation $\mathbf{O}_{obs} \subset \mathbf{O}_{true}$ is a strict subset of system state (instrumentation gaps, sampling), which is the source of the information asymmetry in inequality (1).

D.2 Phase 1: Discrete State Abstraction Ψ

We project high-dimensional raw telemetry $\mathbf{O}^{(t)} \in \mathbb{R}^d$ onto a discrete state alphabet Σ via a node-type-aware mapping $\Psi(O_i, \theta(v_i)) \to s_i \in \Sigma$, where $\theta(v_i)$ is the type of node v_i (matching the type tag θ_i used in the Phase 1 rule transitions of Section 2.2). For *logical nodes* (services, spans), states capture performance violations such as `HighLatency` and `HighErrorRate`; for *infrastructure nodes* (pods, containers, hosts), states reflect resource exhaustion such as `CPU_Throttled` and `MemoryPressure`. This abstraction filters high-frequency noise (U_i), retaining only semantically meaningful events as inputs to the rule-filtering and DFS stages.

D.3 Phase 1: Time-Expanded Heterogeneous Graph and Constrained DFS

To enumerate candidate paths Π_{cand}, we construct a Time-Expanded Heterogeneous Graph whose nodes are tuples (v, s, t): a system node $v \in \mathcal{V}$ with state $s \in \Sigma$ activated at time t. Edges connect tuples whose state transitions are admitted by the propagation rule set $\mathcal{R}$ (Section D.6) and whose times are causally ordered. A constrained Depth-First Search starting from the injection point unrolls circular dependencies (common in microservice meshes) into acyclic chains, with the configured maximum path length and per-node revisit budget (Section D.5). The resulting paths are topologically valid sequences of state transitions; statistical and temporal verification (Section 2.3) then admits or rejects them.

D.4 Cluster Configuration

Our testbed runs on Kubernetes v1.29 with the following specifications:

- **Cluster**: 1 control-plane node (`master1`, 48 vCPU, 64GB RAM) and 6 worker nodes running Debian 12 with containerd 1.7.5; `worker1`–`worker3` each provide 32 vCPU and 64GB RAM, while `worker4`–`worker6` each provide 128 vCPU and 128GB RAM
- **Systems hosted**: TrainTicket (44 Java/Spring services), OpenTelemetry Demo (15 polyglot services), and Hotel Reservation (9 Go services), each deployed in separate Kubernetes namespaces on the shared cluster
- **Service-count convention**: Reported per-system service counts are application `Deployments` obtained via `kubectl get deploy` on the running benchmark, excluding load generators, the OpenTelemetry collector, datastores, message brokers, and the TrainTicket UI dashboard
- **Deployment versions**: Exact system commit hashes and image tags for the three deployments are pinned in the archived manifests released with the benchmark

根据每种混沌机制在指标、追踪和日志中产生的特征签名对其进行标注，并在代理的预测标签与该可观测标签匹配时判定代理正确，而非基于其底层 API 名称。表 6 列出了从原始混沌_类型到可观测标签的映射，以及承载每个标签的主要信号；在真值提取和代理预测匹配中均应用相同的投影，从而使比较的双方共享同一词汇表。

D PAVE流水线与实验细节

本附录详细介绍了（i）为节省篇幅而在第 2 节中压缩的流水线内部机制（D.1–D.3 节，以及 D.6、D.5、D.8 节中的规则集、超参数和检测器阈值），以及（ii）支持复现的操作和校准细节（D.4–D.8 节）。

D.1 结构因果模型

故障传播通过 $\mathcal{G}$ [8] 在每个节点上实现为

$$S_i^{(t)} = f_i\big(\mathbf{S}_{Pa(v_i)}^{(<t)},\, U_i^{(t)}\big), \tag{2}$$

其中，$\mathbf{S}_{Pa}^{(<t)}(v_i)$ 是 v_i 的因果父节点的近期状态，$U(t)_i$ 是外生冲击。代理的观测 $\mathbf{O}_{obs} \subset \mathbf{O}_{true}$ 是系统状态（仪器化缺口、采样）的真子集，这是不等式 (1) 中信息不对称的来源。

D.2 第一阶段：离散状态抽象 Ψ

我们将高维原始遥测数据$\mathbf{O}^{(t)} \in \mathbb{R}^d$ 通过节点类型感知的映射 $\Psi(O_i, \theta(v_i)) \to s_i \in \Sigma$投影到离散状态字母表 Σ 上，其中 $\theta(v_i)$是节点 v_i 的类型（对应于第 2.2 节中第一阶段规则转换所使用的类型标签 θ_i）。对于逻辑节点（服务、跨度），状态捕获性能违规，例如高延迟和高错误率；对于基础设施节点（Pod、容器、主机），状态反映资源耗尽，例如CPU_Throttled和内存压力。这种抽象过滤了高频噪声（U_i），仅保留语义有效事件作为规则过滤和DFS 阶段的输入。

D.3 阶段一：时间扩展异构图与约束深度优先搜索

为了枚举候选路径 Π_{cand}，我们构建了一个时间扩展异构图，其节点为元组 (v, s, t)：即一个状态 $s \in \Sigma$ 在时间 t 被激活的系统节点 $v \in \mathcal{V}$。边连接那些状态转换被传播规则集 $\mathcal{R}$（第 D.6 节）所允许且时间在因果上有序的元组。从注入点开始的受限深度优先搜索将循环依赖（在微服务网格中常见）展开为无环链，并考虑了配置的最大路径长度和每节点重访预算（第 D.5 节）。生成的路径是状态转换的拓扑有效序列；随后，统计与时序验证（第 2.3 节）决定接受或拒绝这些路径。

D.4 集群配置

我们的测试平台运行于 Kubernetes v1.29，具有以下规格：

- 集群：1个控制平面节点（master1，48虚拟 CPU，64GB RAM）和6个工作节点，运行Debian 12和containerd 1.7.5；worker1–worker3每个节点提供32虚拟 CPU和64GB RAM，而worker4–worker6每个节点提供128虚拟 CPU和128GB RAM
- 托管系统：TrainTicket（44个Java/Spring服务）、OpenTelemetry演示（15个多语言服务）和酒店预订（9个Go服务），各部署于共享集群上独立的Kubernetes命名空间中
- 服务计数约定：报告的各系统服务计数为通过kubectl get deploy在运行中的基准测试上获取的应用部署，不包括负载生成器、OpenTelemetry 收集器、数据存储、消息代理和 TrainTicket 用户界面仪表盘
- 部署版本：与该基准测试一同发布的归档清单中，明确了这三个部署的精确系统提交哈希和镜像标签

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 7: Complete propagation rule set $\mathcal{R}$. *Src*/*Dst* denote source and destination entity types; *Path* shows intermediate hops for multi-hop rules.

ID	Category	Src Kind : States	Path	Dst Kind : States	Mechanism
Vertical: Container → Span					
C-01	vertical	container : {cpu, mem}	pod → svc →	span : {lat, p99, timeout, missing, err}	Resource contention
C-02	vertical	container : {restart}	pod → svc →	span : {missing, err, timeout}	Process termination
Vertical: Pod → Span					
P-01	vertical	pod : {cpu, mem}	svc →	span : {lat, p99, timeout}	Resource contention
First-hop: Service → Span					
S-01	first-hop	service : {any}	—	span : {lat, p99, timeout}	Latency inheritance
S-02	first-hop	service : {err, unavail}	—	span : {err, timeout, reset, missing, log_err}	Error inheritance
Horizontal: Span → Span (RPC cascade)					
H-01	horizontal	span : {lat, p99}	—	span : {lat, p99, timeout}	Callee slowdown
H-02	horizontal	span : {err, timeout}	—	span : {err, timeout}	Error propagation
H-03	horizontal	span : {missing}	—	span : {err, timeout, missing}	Missing callee
H-04	horizontal	span : {err, missing}	healthy span →	span : {err, missing}	Controller bypass
H-05	horizontal	span : {err, timeout}	—	span : {timeout}	Retry-induced wait
H-06	horizontal	span : {inj_affected}	—	span : {err, lat, p99, timeout, missing}	Infra fault relay
H-07	horizontal	span : {healthy}	healthy span →	span : {err, lat, p99, missing}	JVM injection bypass
Cross-channel: Span → Pod → Span					
X-01	cross	span : {lat, p99}	svc → pod → svc →	span : {lat, p99, timeout}	JVM stress via shared pod

- **Traffic load**: Synthetic user sessions are generated by per-system load generators after a warm-up period, with sustained request streams used to establish pre-injection baselines and expose post-injection propagation
- **Monitoring stack**: OpenTelemetry-based collection with per-system telemetry export to standardized parquet artifacts, together with cluster-level collectors and Prometheus-based metric scraping
- **Fault injection**: Chaos Mesh with programmatic CRD generation for application, resource, network, and pod-level interventions
- **Stateful backends**: Across the three systems, the benchmark exercises MySQL, PostgreSQL, MongoDB, Memcached/valkey, RabbitMQ, and Kafka-backed propagation paths

D.5 PAVE Hyperparameters

Key parameters for the PAVE annotation pipeline:

- **State Detection Window** Δ: 3 s for span/service-level metrics; 5 s for pod, container, and machine-level metrics
- **State Activation**: Dual detection combines fixed utilization thresholds (>80% CPU/memory) with Z-score baseline comparison ($Z > 3\sigma$ for critical anomalies), plus adaptive multiplier thresholds for latency metrics based on coefficient of variation (CV)
- **Maximum Path Length**: 5 hops (default), with diamond-shaped revisits allowed (max 2 visits per node)
- **Rule Confidence**: Default 0.8 per propagation rule, with per-rule overrides

D.6 Propagation Rule Set

Table 7 lists the complete rule set $\mathcal{R}$ used in Phase 1 (Section 2.2). Rules are constructed by enumerating all (source entity type × source state × edge type × destination entity type) combinations in the system dependency graph, then retaining only those for which a known physical or logical mechanism exists. We group rules into four categories by injection granularity.

Vertical rules (container→span, pod→span) encode resource contention: a throttled container or pod degrades all spans of its co-located service through the Kubernetes scheduling hierarchy (Container ⊂ Pod ⊂ Node). **First-hop rules** (service→span) bridge the aggregation layer, since services are logical groupings without directly observable states. **Horizontal rules** (span→span) capture RPC call-chain cascades, the dominant propagation channel, accounting for over 80% of observed edges. **Cross-channel rules** (span→pod→span) model JVM stress scenarios where resource contention on a shared pod affects sibling services.

The rule set is intentionally over-inclusive: rules with zero empirical usage (e.g., service error→span) are retained for completeness, and Phase 2's statistical verification (Section 2.3) filters false positives.

表 7：完整的传播规则集 $\mathcal{R}$。Src/Dst 表示源和目标类型；Path 显示多跳规则的中间跳数。

ID	类别	源类型：状态	Path	目标类型：状态	机制
垂直：容器 → 跨度					
C-01	垂直	容器: {中央处理器, 内存}	Pod → 服务 →	跨度：{lat, p99, 超时, 缺失, 错误}	资源争用
C-02	垂直	容器：{restart}	Pod →服务 →	跨度：{缺失, 错误, 超时}	进程终止
垂直：*Pod* →跨度					
P-01	垂直	Pod：{中央处理器, 内存}	服务 →	跨度：{lat, p99, 超时}	资源争用
第一跳：服务 →跨度					
S-01	第一跳	服务 : {any}	—	跨度：{lat, p99, 超时}	延迟继承
S-02	第一跳	服务 : {错误, 不可用}	—	跨度：{错误, 超时, 重置, 缺失, 日志_错误}	错误继承
水平：跨度 →跨度（*RPC*级联）					
H-01	水平	跨度：{延迟, P99}	—	跨度：{lat, p99, 超时}	被调用方变慢
H-02	水平	跨度 : {错误, 超时}	—	跨度 ：{错误, 超时}	错误传播
H-03	水平	跨度：{missing}	—	span : {错误, 超时, 缺失}	缺失被调用方
H-04	水平	span ： {错误， 缺失}	健康跨度 →	跨度：{错误， 缺失}	控制器旁路
H-05	水平	跨度 : {错误, 超时}	—	跨度：{timeout}	重试引起的等待
H-06	水平	跨度：{inj_affected}	—	跨度：{err, lat, p99, 超时, 缺失}	基础设施故障中继
H-07	水平	跨度：{healthy}	健康跨度 →	跨度：{错误, 延迟, P99, 缺失}	JVM注入绕过
跨通道：跨度 →*Pod* →跨度					
X-01	跨	跨度：{延迟, P99}	服务 →Pod →服务 →	span：{lat, p99, timeout}	通过共享Pod进行JVM压力测试

• 流量负载：在每个系统的预热期结束后，由各系统负载生成器生成合成用户会话，并使用持续请求流来建立注入前基线并揭示注入后传播

• 监控栈：基于OpenTelemetry的采集，按系统级别进行遥测数据导出，并转换为标准化的Parquet 工件，同时结合集群级采集器和基于Prometheus的指标抓取

• 故障注入：Chaos Mesh，支持应用、资源、网络和 Pod 级别干预的程序化 CRD 生成

• 有状态后端：在这三个系统中，基准测试涉及由MySQL、PostgreSQL、MongoDB、Memcached/Valkey、RabbitMQ和Kafka支持的传播路径

D.5 PAVE 超参数

PAVE 标注流水线的关键参数：

• 状态检测窗口 Δ：跨度/服务级指标为 3 秒；Pod、容器和机器级指标为 5 秒

• 状态激活：双重检测结合了固定利用率阈值（>80% CPU/内存）与 Z 分数基线比较（$Z > 3\sigma$ 用于严重异常），以及基于变异系数（CV）的延迟指标的自适应乘数阈值

• 最大路径长度：5 跳（默认），允许菱形重访（每个节点最多访问 2 次）

规则置信度：默认每条传播规则为 0.8，并支持针对单条规则进行覆盖

D.6 传播规则集

表 7 列出了第一阶段（第 2.2 节）中使用的完整规则集 $\mathcal{R}$ 。规则的构造方法是枚举系统依赖图中的所有（源实体类型 × 源状态 × 边类型 × 目标实体类型）组合，然后仅保留那些存在已知物理或逻辑机制的组合。我们根据注入粒度将规则分为四类。

垂直规则（container→span，pod→span）编码资源争用：受限的 container 或 pod 通过 Kubernetes 调度层级（Container⊂Pod ⊂ Node）降低其同驻 service 的全部 span 的性能。第一跳规则（service→span）桥接聚合层，因为服务是无直接可观测状态的状态的逻辑分组。水平规则（span→span）捕获 RPC 调用链级联，这是主导传播通道，占观察到的边的 80% 以上。跨通道规则（span→pod→span）建模 JVM 压力场景，其中共享 Pod 上的资源争用会影响兄弟服务。

该规则集有意过度包含：零实证使用的规则（例如服务错误→跨度）被保留以确保完整性，而第二阶段（第 2.3 节）的统计验证会过滤假阳性。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Rules lacking any empirical support after 500+ injection experiments were removed in version 2.0 (e.g., circuit-breaker bypass, pod-level disk/network faults), as noted in the version history.

D.7 Scalability and Computational Cost

All experiments were conducted on a virtual machine with 32 CPU cores and 64GB RAM. The path extraction process for a single fault injection scenario, covering a 15-minute telemetry window (∼1GB of metrics and traces), completes in an average of 3.5 minutes. Computational complexity scales with the number of active anomalous entities rather than total system components, demonstrating practical scalability.

D.8 Anomaly Detection Thresholds

The per-node anomaly screen in Phase 2 (Section 2.3) uses three detector families, each matched to the distributional characteristics of the underlying metric.

Z-score detector (for approximately Gaussian metrics such as CPU and memory utilization). A metric value x is flagged if its Z-score $z = (x - \mu_{base})/\max(\sigma_{base}, \epsilon)$ exceeds a threshold, where μ_{base} and σ_{base} are the baseline mean and standard deviation and $\epsilon = 10^{-6}$ prevents division by zero. We use three severity levels: *critical* ($z > 3.0$), *warning* ($z > 2.0$), and *moderate* ($z > 1.5$).

Percentile detector (for heavy-tailed metrics such as error counts). Instead of parametric assumptions, anomalies are flagged relative to empirical baseline percentiles: *critical* if the value exceeds the baseline P99.9 or 2×P99; *warning* if it exceeds P99 or 1.5×P99; *moderate* if it exceeds P90 or 1.2×P99.

Adaptive latency detector (for request latency, which spans orders of magnitude across services). The threshold multiplier is $\tau = (\tau_{cv\text{-}base} + \tau_{cv\text{-}range} \cdot (1 - e^{-cv/\alpha})) \cdot (l_{ref}/l_{base})^{\beta}$, where cv is the baseline coefficient of variation, l_{base} the baseline mean latency, and l_{ref} a reference latency. This formulation encodes the principle that low-baseline services (e.g., 1 ms mean) tolerate large relative spikes with minimal user impact, while high-baseline services (e.g., 5 s mean) require tighter thresholds.

Design rationale. All three detectors are intentionally permissive at the per-node level: they aim to avoid false negatives (missing a genuinely affected node) at the cost of admitting some false positives (flagging coincidental deviations). False positives are subsequently eliminated by the conjunctive verification criterion: a candidate path is accepted only when every node passes the anomaly screen, every edge conforms to the propagation rule set $\mathcal{R}$, and downstream anomalies follow upstream causes in time. This layered design decouples the sensitivity of individual detectors from the overall precision of the ground truth.

Why conjunction filtering is effective. A candidate path π of length k is verified only if three independent conditions hold simultaneously at every hop: (i) statistical anomaly, (ii) structural rule conformance, and (iii) temporal ordering. Let p_s, p_r, and p_t denote the per-hop false-positive rates of each condition in isolation. Under an independence assumption, the probability that a spurious (non-causal) path of length k accidentally satisfies all three conditions at every hop is bounded by:

$$P(\text{false path accepted}) \leq (p_s \cdot p_r \cdot p_t)^k \tag{3}$$

Even with individually permissive thresholds (e.g., p_s=0.2, p_r=0.3, p_t=0.5), the per-hop joint false-positive rate is $0.2 \times 0.3 \times 0.5 = 0.03$, and for a typical path of length k=3 this yields $0.03^3 \approx 2.7 \times 10^{-5}$. This exponential decay in k means that longer propagation paths, which are harder to verify, are also the most resistant to spurious acceptance. The independence assumption is approximate: in practice the three conditions share some information (e.g., structural constraints partially determine temporal ordering). Nevertheless, the exponential scaling ensures robust filtering even when the conditions are moderately correlated.

E Audit Protocol and Calibration

This appendix details the two-annotator audit conducted on a random 100-case subsample of Open-RCA 2.0 (Section 3.4) and the calibration split used by the per-node anomaly screen. Soundness of the verified causal subgraph $\mathcal{G}^*$ rests on the conjunctive verification design analyzed in Section D.8:

在 500+ 注入实验中缺乏任何实证支持的规则已从inversion2.0中移除（例如熔断器绕过、Pod级别磁盘/网络故障），如版本历史记录中所述。

D.7 可扩展性与计算成本

全部实验均在配备32个CPU核心和64GB RAM的虚拟机上进行。针对单个故障注入场景，覆盖15分钟遥测窗口（∼1GB的指标和追踪数据）的路径提取过程平均耗时3.5分钟。计算复杂度随活跃异常实体的数量而非系统组件总数缩放，证明了实际的可扩展性。

D.8 异常检测阈值

第 2 节中的 Phase2 每节点异常筛查使用三种检测器家族，每种检测器家族都与底层指标的分布特征相匹配。

Z分数检测器（适用于近似高斯的指标，例如中央处理器和内存利用率）。当指标值 x 的 Z分数 $z = (x - \mu_{base})/\max(\sigma_{base}, \epsilon)$ 超过阈值时，该值会被标记，其中 μ_{base} 和 σ_{base} 分别是基线均值和标准差，$\epsilon = 10^{-6}$ 用于防止除以零。我们使用三个严重性级别：严重 ($z > 3.0$)、警告 ($z > 2.0$)和中等 ($z > 1.5$)。

百分位检测器（用于错误计数等重尾指标）。不同于参数化假设，异常标记是相对于经验基线百分位确定的：如果值超过基线 P99.9 或 2×P99，则为严重；如果超过 P99 或 1.5×P99，则为警告；如果超过 P90 或 1.2×P99，则为中等。

自适应延迟检测器（用于请求延迟，不同服务间的延迟跨度可达数个数量级）。阈值乘数为 $\tau = (\tau_{cv\text{-}base} + \tau_{cv\text{-}range} \cdot (1 - e^{-cv/\alpha})) \cdot (l_{ref}/l_{base})^{\beta}$，其中 cv 是基线变异系数，l_{base} 是基线平均延迟，l_{ref} 是参考延迟。该公式体现了一个原理：低基线服务（例如平均延迟 1 ms）可以容忍较大的相对尖峰而对用户影响极小，而高基线服务（例如平均延迟 5s）则需要更严格的阈值。

设计原理。所有三个检测器在单节点级别上都刻意保持宽容：其目标是通过容忍一定的假阳性（标记偶然的偏差）来避免假阴性（遗漏真正受影响的节点）。假阳性随后由合取验证准则消除：只有当每个节点都通过异常筛选、每条边都符合传播规则集 $\mathcal{R}$，且下游异常在时间上跟随上游原因时，候选路径才会被接受。这种分层设计将单个检测器的灵敏度与真实值的整体精确率解耦。

合取过滤为何有效。长度为 k 的候选路径 π 仅在三个独立条件在每一跳同时成立时才会被验证：(i) 统计异常，(ii) 结构规则符合性，以及 (iii) 时间顺序。设 p_s、p_r 和 p_t 分别表示每个条件单独存在时的每跳假阳性率。在独立性假设下，长度为 k 的虚假的（非因果）路径在每一跳偶然满足所有三个条件的概率受限于：

$$P(\text{false path accepted}) \leq (p_s \cdot p_r \cdot p_t)^k \tag{3}$$

即使采用宽松阈值（例如，p_s=0.2、p_r=0.3、p_t=0.5），每跳联合假阳性率仍为 $0.2 \times 0.3 \times 0.5 = 0.03$，而对于长度为 k=3 的典型路径，其值为 $0.03^3 \approx 2.7 \times 10^{-5}$。这种 k 的指数衰减意味着，更长的传播路径虽然更难验证，但也最不容易发生虚假接受。独立性假设只是一种近似：在实践中，三个条件共享某些信息（例如，结构约束部分决定了时间顺序）。然而，即使条件之间存在中等程度的相关性，指数缩放仍能确保鲁棒过滤。

E 审计协议与校准

本附录详细说明了在Open-RCA 2.0的随机100例子样本（第3.4节）上进行的双标注员审计，以及每节点异常筛查所使用的校准划分。验证因果子图 $\mathcal{G}^*$ 的可靠性依赖于第D.8节中分析的合取验证设计：

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

$do(v_{root})$ pins the root direction, and an admitted path must clear three independently sourced gates simultaneously. The audit reported here is the empirical complement to that design argument.

E.1 Annotator Audit on a Random Subsample

Goal. Bound the reliability of $\mathcal{G}^*$ by measuring agreement between the verified causal subgraphs produced by PAVE and the propagation paths that domain-experienced annotators consider plausible from the same telemetry.

Sampling. A simple random sample of $N{=}100$ instances was drawn without replacement from the 500 evaluable instances of OpenRCA 2.0. Sampling was uniform rather than stratum-balanced, so the audit's verdict reflects the dataset's natural mix of (system, fault family) pairs; post-hoc, the sample covered all 3 systems and every chaos primary kind retained in the benchmark.

Annotators. Two of the paper's authors served as annotators. Both have research experience in distributed-system observability and operational fault analysis, and are familiar with the PAVE pipeline output schema. The annotators worked independently from disjoint copies of each case's telemetry and were blind to one another's verdicts until the adjudication step.

Materials per case. For each instance, an annotator received: (i) the multimodal telemetry over both pre- and post-injection windows (distributed traces, metric series, structured logs, all in their original parquet schema); (ii) the injected-fault metadata, comprising v_{root}, fault type, and the injection window $[t_0, t_0{+}\Delta t]$; and (iii) the pipeline-produced cascade $\mathcal{G}^*$ rendered as a labelled directed graph over services, pods, and spans. The pipeline output was shown so the annotator could assess it directly; the underlying telemetry was provided so the assessment was grounded in the raw evidence rather than the pipeline's own intermediate features.

Annotation rubric. Each annotator recorded a single binary verdict per case: AGREE when every edge in $\mathcal{G}^*$ is plausibly causal given the telemetry *and* no obvious propagation edge is missing; DISAGREE otherwise. When an annotator marked DISAGREE, they were instructed to flag the specific edge or path that was either implausible (an included edge with no telemetry support) or missing (a propagation step visible in the telemetry but absent from $\mathcal{G}^*$).

Verification procedure. The binary verdict is grounded in a fixed three-stage inspection of each case, applied uniformly across the sample. The annotator first reviews the injected-fault metadata to anchor expectation: which component received the intervention $do(v_{root})$, which fault primitive was applied, and the injection window $[t_0, t_0{+}\Delta t]$. The annotator then proceeds through three stages.

Stage 1: anomaly verification (per node). For each node in $\mathcal{G}^*$ that carries a non-healthy state annotation, the annotator opens the underlying telemetry slice for that node over the abnormal window (span series for span nodes, metric and log streams for service and pod nodes) and verifies that the annotated abnormality is visible in the raw evidence. A node whose label cannot be located in the raw signal is flagged as a detector false positive.

Stage 2: hop-level causality verification (per edge). For each edge $u \to v$ in $\mathcal{G}^*$, the annotator assesses three properties: (i) *temporal precedence*, where u's onset of abnormality precedes v's within plausible propagation latency; (ii) *structural plausibility*, where the architectural relationship between u and v (caller–callee, container-of-pod, span-within-service, or other declared edge kind) admits the proposed propagation mechanism for the injected fault primitive; and (iii) *causal versus coincidental*, where the change in v is consistent with a downstream consequence of the abnormality at u (correlated latency, error-rate, or saturation pattern) rather than an unrelated fluctuation that happens to fall within the same time window. An edge that fails any of (i)–(iii) is flagged as implausible.

Stage 3: coverage check. The annotator scans the telemetry for nodes that exhibit clear abnormality during the injection window but are absent from $\mathcal{G}^*$. For each such omission the annotator asks whether the missing node lies on a plausible propagation hop from v_{root} to a labelled alarm in $\mathcal{G}^*$; if it does, the missing path is flagged.

A case receives AGREE only when stages 1–3 produce no flags; any flag triggers DISAGREE, and the flagged node, edge, or missing path is recorded for adjudication.

Adjudication. After both annotators completed the sample, every case on which the two verdicts diverged was reviewed jointly. The two annotators discussed the flagged edges, returned to the underlying telemetry where needed, and reached consensus on the source of each disagreement. This

$do(v_{root})$ 确定根方向，且被认可路径必须同时通过三个独立来源门控。此处报告的审计是该设计论证的实证补充。

E.1 对随机子样本的标注员审计

目标。通过衡量 PAVE 生成的已验证因果子图与资深领域标注员基于相同遥测数据认为合理的传播路径之间的一致性，来界定 $\mathcal{G}^*$ 的可靠性。

采样。从OpenRCA 2.0的500个可评估实例中无放回地进行了 $N{=}100$ 实例的简单随机抽样。该采样采用均匀分布而非分层平衡方式，因此审计结果反映了数据集中（系统，故障家族）对的天然混合分布；事后分析表明，该样本覆盖了全部3个系统以及基准测试中保留的所有混沌主要类型。

标注员。本文两位作者担任标注员。两人均具有分布式系统可观测性和运维故障分析的研究经验，并熟悉PAVE管线输出模式。标注员基于每个案例遥测数据的不重叠副本独立开展工作，且在裁定步骤之前对彼此的裁决结果保持盲态。

每个案例的材料。对于每个实例，标注员收到：(i) 覆盖注入前后窗口的多模态遥测数据（分布式追踪、指标序列、结构化日志，均保留其原始Parquet模式）；(ii) 注入故障元数据，包括 v_{root}、故障类型以及注入窗口 $[t_0, t_0{+}\Delta t]$；以及 (iii) 由流水线生成的级联效应 $\mathcal{G}^*$，以基于服务、Pod和跨度的有标签有向图形式呈现。展示流水线输出是为了让标注员能够直接对其进行评估；提供底层遥测数据则是为了确保评估基于原始证据而非流水线自身的中间特征。

标注准则。每位标注员对每个案例记录一个二元判定：当 $\mathcal{G}^*$ 中的每一条边在给定遥测数据的情况下均看似因果合理，且没有明显的缺失传播边时，判定为同意；否则判定为不同意。当标注员判定为不同意时，他们需明确标出具体不合理或缺失的边或路径——不合理指包含某条边但缺乏遥测支持，缺失指遥测数据中可见传播步骤但 $\mathcal{G}^*$中未包含该步骤。

验证程序。二元判定基于对每个案例固定执行的三阶段检查，该检查在所有样本中均匀应用。标注员首先审阅注入故障的元数据以确立期望：哪个组件接受了干预 $do(v_{root})$，应用了哪种故障原语，以及注入窗口 $[t_0, t_0{+}\Delta t]$。随后，标注员按三个阶段进行检查。

阶段1：异常验证（按节点）。对于 $\mathcal{G}^*$ 中携带非健康状态标注的每个节点，标注员打开该节点在异常窗口内的底层遥测切片（对于Span节点为Span序列，对于服务和Pod节点为指标和日志流），并验证标注的异常在原始证据中可见。如果节点的标签无法在原始信号中定位，则将其标记为检测器假阳性。

阶段2：跳级因果关系验证（逐边进行）。对于每条边 $u \to v$ 到 $\mathcal{G}^*$，标注员评估三个属性：(i) 时间先后关系，即 u 的异常起始在合理的传播延迟内先于 v；(ii) 结构合理性，即 u 与 v 之间的架构关系（调用者-被调用者、容器-Pod关系、服务内Span或其他已声明的边类型）支持所提出的注入故障原语的传播机制；以及 (iii) 因果与巧合，即 v 的变化与 u 处异常的下游后果（相关延迟、错误率或饱和模式）一致，而非恰好落在同一时间窗口内的无关波动。未通过 (i)–(iii) 中任一项验证的边将被标记为不合理。

阶段三：覆盖检查。标注员扫描遥测数据，查找在注入窗口期间表现出明显异常但缺失于 $\mathcal{G}^*$中的节点。对于每一个此类遗漏，标注员需判断该缺失节点是否位于从 v_{root} 到 $\mathcal{G}^*$中某个已标记告警的合理传播跳上；如果是，则标记该缺失路径。

只有当阶段1–3未产生任何标记时，案例才会获得“同意”；任何标记都会触发“不同意”，并且被标记的节点、边或缺失路径会被记录下来以供仲裁。

仲裁。在两位标注员完成样本标注后，对两位裁决结果存在分歧的所有案例进行联合审查。两位标注员就标记边进行了讨论，必要时返回查看底层遥测数据，并就各分歧根源达成一致意见。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

adjudication produced a per-disagreement note used in the analysis below; it did not modify the released $\mathcal{G}^*$ for those cases, since the audit is a measurement of the pipeline's output, not a relabelling pass.

Result. Inter-annotator agreement on the case-level verdict was $94/100 = 94\%$. The six disagreements were entirely one-directional: in each case, at least one annotator considered an additional propagation path plausible that $\mathcal{G}^*$ did not include. No case was flagged in the opposite direction, that is, no annotator considered any edge already in $\mathcal{G}^*$ implausible, so within the audited sample the released cascades contain no detected false-positive edges.

Disagreement analysis. Joint review attributed every under-coverage instance to the conjunctive verification design's deliberate strictness on borderline evidence: the gates had rejected paths whose statistical-deviation, rule-conformance, or temporal-alignment signal sat near the admission threshold. The flagged paths were not spurious (annotators considered them plausible) but they were weakly supported, and in each instance the path corresponded to a service receiving low traffic during the abnormal window, where the per-node anomaly screen could not cleanly separate the propagated effect from baseline jitter. This direction of error is consistent with the protocol's stated property: $\mathcal{G}^*$ is constructed as a high-precision over-approximation of the underlying causal structure, prioritizing precision (no spurious edges admitted) over recall (some weakly supported edges may be missed). For OpenRCA 2.0's intended use as a process-level reference for evaluating LLM agents' propagation-path reasoning, admitting a false edge into the reference would directly mis-credit or mis-penalize an agent's diagnosis; omitting a weakly supported edge does not. The audit therefore confirms the design's labelling discipline: in the audited sample the released cascades are sound (no implausible included edges) at the cost of conservative coverage, which we accept as the appropriate trade-off for the benchmark's intended use.

E.2 Baseline Cal-Eval Split

For each instance, the pre-injection baseline window $\mathcal{D}_{base}$ is split into two halves: the first calibrates per-node thresholds (Z-score parameters, percentile cutoffs, adaptive-latency multipliers; see Section D.8), the second evaluates them. Calibrating and evaluating on the same window inflates the per-node admission rate because the screen would fit the noise it then tests against; the split prevents this overfit. Default calibration set size is half the baseline window length (~ 7.5 minutes).

F LLM-Based RCA Agent Details

This section details the evaluation framework, agent architecture, and prompt design for the LLM-based RCA agents evaluated in our benchmark.

F.1 Agent Evaluation Framework

Each LLM agent receives a datapack containing multimodal telemetry (traces, metrics, logs in parquet format) and produces a structured CausalGraph: a directed graph with annotated root causes, affected nodes, and causal edges. Evaluation operates at two levels: outcome-level (root cause accuracy) and process-level (causal edge and node fidelity against ground truth). This dual evaluation design enables the stratified Edge F1 analysis presented in Section 3.2.

F.2 Agent Architecture

All agents follow a tool-augmented LLM architecture adapted from the Deep Research agent [13], operating in a ReAct-style [12] investigation loop with three nodes:

1. `llm_call` (Decision Node): The LLM analyzes the current investigation state, consisting of the incident description, prior tool observations, and its own reflections, and either issues additional tool calls or terminates the loop.
2. `tool_node` (Execution Node): Dispatches all tool calls from the LLM response, executes them asynchronously, and returns observations as `ToolMessage` objects. Errors are caught and returned as observations (the loop does not crash on tool failures).
3. `compress_rca_findings` (Synthesis Node): Invoked when the LLM stops issuing tool calls. A separate LLM call synthesizes all accumulated evidence into a structured CausalGraph JSON.

仲裁过程生成了用于下文分析的逐分歧备注；但它并未修改这些案例中已发布的 $\mathcal{G}^*$，因为该审计是对流水线输出的测量，而非重新标记过程。

结果。案例级裁决的标注者间一致性为 $94/100 = 94\%$。六处分歧完全具有单向性：在每种情况下，至少有一名标注员认为一条额外的传播路径是合理的，而 $\mathcal{G}^*$ 未将其纳入。没有任何案例出现相反方向的标记，即没有标注员认为 $\mathcal{G}^*$ 中已有的任何边是不合理的，因此在审计样本中，发布的级联故障未发现误报边。
分歧分析。联合评审将所有覆盖不足实例归因于合取验证设计对临界证据的刻意严格性：门控机制拒绝了那些统计偏差、规则符合度或时间对齐信号接近准入阈值的路径。被标记的路径并非虚假的（标注员认为它们是合理的），但支持证据较弱，并且在每个实例中，该路径对应于在异常窗口期间低流量的服务，此时每节点异常筛查无法干净地将传播效应与基线抖动区分开来。这种错误方向与协议所述属性一致： $\mathcal{G}^*$被构建为底层因果结构的高精度过近似，优先考虑精确率（不允许多余的错误边），而非召回率（可能会遗漏一些支持证据较弱的边）。对于OpenRCA 2.0作为评估大语言模型智能体路径传播推理的过程级参考这一预期用途而言，向参考中引入错误边会直接错误地奖励或惩罚智能体的诊断结果；而遗漏一条支持证据较弱的边则不会。因此，审计证实了该设计的标记纪律：在审计样本中，发布的级联故障是可靠的（没有不可信的包含边），但代价是保守覆盖，我们认为这是该基准测试预期用途所接受的适当权衡。

E.2 基线校准-评估划分

对于每个实例，注入前基线窗口 $\mathcal{D}_{base}$ 被划分为两半：前半部分用于校准逐节点阈值（Z-score参数、百分位截止值、自适应延迟倍数；见第 D.8 节），后半部分用于对其进行评估。在同一窗口上进行校准和评估会虚高逐节点准入率，因为筛选器会对它随后进行测试的噪声进行拟合；这种划分可防止此类过拟合。默认校准集大小为基线窗口长度的一半（~ 7.5 分钟）。

F 基于大语言模型的根因分析智能体详情

本节详细介绍了我们在基准测试中评估的大语言模型根因分析代理的评估框架、智能体架构和提示词设计。

F.1 代理评估框架

每个大语言模型代理接收一个包含多模态遥测数据（Parquet格式的追踪、指标、日志）的数据包，并生成一个结构化的因果图：一个带有根本原因、受影响节点和因果边注释的有向图。评估在两个级别上进行：结果级（根因准确率）和过程级（因果边和节点相对于真实值的保真度）。这种双重评估设计使得第 3.2 节中提出的分层边F1分析成为可能。

F.2 智能体架构

All 智能体遵循源自 Deep Research 智能体 [13 的工具增强型大语言模型架构], oper以ReAct风格的 [12]调查循环，包含三个节点：

1. llm_call（决策节点）：大语言模型分析当前调查状态，包括事件描述、先前工具观测结果及其自身的反思，然后发出额外的工具调用或终止循环。2. tool_node（执行节点）：分发大语言模型响应中的全部工具调用，异步执行这些调用，并将观测结果作为工具消息对象返回。错误会被捕获并作为观测结果返回（循环不会因工具故障而崩溃）。3. compress_rca_findings（综合节点）：当大语言模型停止发出工具调用时触发。通过单独的大语言模型调用，将所有累积证据合成为结构化的因果图JSON。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

The control flow follows:

$$\texttt{START} \rightarrow \texttt{llm_call} \xrightarrow{\text{tool_calls?}} \texttt{tool_node} \rightarrow \texttt{llm_call} \rightarrow \cdots \rightarrow \texttt{compress} \rightarrow \texttt{END}$$

Tool suite. The agent has access to four tools:

- `list_tables_in_directory`: Discovers available parquet files and their metadata (row/column counts).
- `get_schema`: Inspects the column names, types, and row count of a parquet file before querying.
- `query_parquet_files`: Executes SQL queries over parquet files (supports SELECT, WHERE, JOIN, aggregations).
- `think_tool`: A reflection tool that records the agent's internal reasoning. The prompt instructs the agent to call this tool after each query to summarize findings and plan the next investigation step.

The prompt recommends a tool-call budget of ∼50 calls per case and enforces a hard cap of 100 at the runtime layer; the LangGraph recursion limit is set well above this so the cap is the binding constraint. All models share the same tool definitions and identical API retry logic (exponential backoff with max 5 retries).

F.3 Output Schemas: Ground Truth and Agent Answer

Evaluation uses two distinct schemas: the released ground-truth graph in each case directory, and the structured JSON the agent emits. The ground-truth schema preserves per-node states and onset timestamps for process-level metrics; the agent schema is a smaller, verifiable contract built from `(service, fault_kind)` root-cause claims, propagation edges, and SQL evidence.

Ground-truth representation (released artifact). Each case directory ships a `causal_graph.json` file in the `CausalGraph` schema below. This is the per-case process-level label produced by the PAVE pipeline (Section 2). Every node carries a controlled-vocabulary state and an onset timestamp, so the process-level metrics in Section G can be computed directly from this representation.

SCHEMA Ground-truth CausalGraph JSON

```
{
  "nodes": [
    {"component": "ts-order-service",
     "state": ["HIGH_ERROR_RATE"],
     "timestamp": 1718234567000000000},
    {"component": "ts-travel-service",
     "state": ["TIMEOUT"],
     "timestamp": 1718234569000000000},
    {"component": "ts-station-service",
     "state": ["UNAVAILABLE"],
     "timestamp": 1718234565000000000}
  ],
  "edges": [
    {"source": "ts-station-service",
     "target": "ts-travel-service"},
    {"source": "ts-travel-service",
     "target": "ts-order-service"}
  ],
  "root_causes": [
    {"component": "ts-station-service",
     "state": ["UNAVAILABLE"],
     "timestamp": 1718234565000000000}
  ],
  "alarm_nodes": [
    {"component": "span|loadgenerator::POST /api/order"}
  ]
}
```

The four top-level fields are:

控制流如下：

$$\texttt{START} \rightarrow \texttt{llm_call} \xrightarrow{\text{tool_calls?}} \texttt{tool_node} \rightarrow \texttt{llm_call} \rightarrow \cdots \rightarrow \texttt{compress} \rightarrow \texttt{END}$$

工具集。代理可以访问以下四个工具：

- list_tables_in_directory：发现可用的parquet文件及其元数据（行数/列数）。
- get_schema：在查询之前检查parquet文件的列名、类型和行数。
- query_parquet_files：在parquet文件上执行SQL查询（支持SELECT、WHERE、JOIN、聚合）。
- think_tool：一种反思工具，用于记录代理的内部推理过程。提示词指示代理在每次查询后调用此工具，以总结发现并规划下一步调查步骤。

提示词建议每个案例的工具调用预算为 ∼50 次调用，并在运行时层强制实施100次的硬性上限；LangGraph的递归限制设置得远高于此值，因此该上限是约束瓶颈。所有模型共享相同的工具定义和相同的API重试逻辑（采用指数退避，最大重试次数为5）。

F.3 输出模式：真实值与代理答案

评估使用两种不同的模式：每个案例目录中发布的真值图，以及代理发出的结构化JSON。真值模式保留了进程级指标每个节点的状态和起始时间戳；代理模式是一个较小的、可验证的契约，由（服务，故障_类型）根因声明、传播边和SQL证据构成。

真值表示（发布产物）。每个案例目录均附带一个因果图模式下的causal_graph.json文件。这是由PAVE流水线（第 2 节）生成的单案例过程级标签。每个节点均带有受控词汇状态和起始时间戳，因此第G节中的过程级指标可直接根据此表示形式计算得出。

真值 因果图JSON

```
{
  "节点": [
    {"component": "ts-order-service",
     "state": ["高_错误_率"],
     "timestamp": 1718234567000000000},
    {"component": "ts-travel-service",
     "state": ["超时"],
     "timestamp": 1718234569000000000},
    {"component": "ts-station-service",
     "状态": ["不可用"],
     "timestamp": 1718234565000000000}
  ],
  "边": [
    ts-station-service,
     "target": "ts-travel-service"},
    ts-travel-service,
     "target": "ts-order-service"}
  ],
  "root_causes": [
    {"component": "ts-station-service",
     "状态": ["不可用"],
     "timestamp": 1718234565000000000}
  ],
  "alarm_节点": [
    {"component": "跨度|负载生成器::POST /api/order"}
  ]
}
```

四个顶层字段是：

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

- `nodes`: all involved components, each tagged with abnormal state(s) and an onset timestamp.
- `edges`: directed causal arcs; an edge (`source`, `target`) means the failure of `source` caused the failure of `target`.
- `root_causes`: the subset of `nodes` that initiated the cascade.
- `alarm_nodes`: the downstream nodes observed to behave abnormally during the injection window, used as targets for path-reachability scoring.

In the example, `ts-station-service` became `UNAVAILABLE` at t_0, propagating to `ts-travel-service` (`TIMEOUT`) and then `ts-order-service` (`HIGH_ERROR_RATE`), yielding a three-hop path.

Agent answer schema. The agent emits a strict-JSON `AgentRCAOutput` with two flat lists. `root_causes` contains entries of the form (`service`, `fault_kind`, `evidence[]`), where `fault_kind` is drawn from a controlled enum and `evidence` is non-empty. `propagation` contains directed (`from`, `to`) edges, each also with non-empty `evidence`. The schema is uniform across single-fault and hybrid cases; the agent fills `root_causes` with as many entries as the evidence supports.

SCHEMA Agent answer (`AgentRCAOutput`)

```
{
  "root_causes": [
    {"service": "ts-station-service",
     "fault_kind": "pod_unavailable",
     "evidence": [
       {"kind": "metric",
        "sql": "SELECT time, value FROM abnormal_metrics WHERE service_name = '
    ts-station-service' AND metric LIKE 'pod_phase' ORDER BY time",
        "claim": "ts-station-service pod is in NotReady throughout the window"}
     ]}
  ],
  "propagation": [
    {"from": "ts-station-service", "to": "ts-travel-service",
     "evidence": [
       {"kind": "trace",
        "sql": "SELECT span_name, attr.status_code, COUNT(*) FROM
    abnormal_traces WHERE service_name = 'ts-travel-service' AND span_name LIKE
    '%station%' GROUP BY 1,2",
        "claim": "ts-travel-service calls into ts-station-service return TIMEOUT
    "}
     ]}
  ]
}
```

Each `Evidence` item declares (i) its modality (`kind` $\in$ {`metric`,`trace`,`log`}), (ii) the actual DuckDB `sql` it claims is supported by the case parquets, and (iii) a $\leq$20-word `claim` stating what the rows demonstrate. The contract requires every `root_cause` and every `propagation` edge to carry at least one evidence item, and the SQL must be re-executable by the judge against the case directory. Service names are normalized at match time (lowercase, hyphens and underscores stripped) to absorb naming-style drift.

Both schemas feed the evaluation pipeline: the (`service, fault_kind`) multiset and the propagation edge set are extracted from the agent answer, while their counterparts plus the alarm-node set are read off the ground-truth graph. Section G defines the per-axis comparisons.

F.4 Prompt Design

The agent runs a two-phase prompt. The *investigation phase* (`RCA_ANALYSIS_SP/UP`) lets the model freely query the case parquets via tool calls and fixes the goal and playbook without prescribing the output JSON. The *synthesis phase* (`COMPRESS_FINDINGS_SP/UP`) converts the accumulated tool messages into one strict-JSON `AgentRCAOutput` (Section F.3) using a canonical contract block spliced in from the SDK at format time.

- 节点：全部涉及的组件，每个组件均标记有异常状态和起始时间戳。
- 边：有向因果弧；边 (source, target) 表示 source 的故障导致了 target 的故障。

- root_causes：引发级联效应的节点子集。
- 告警_节点：在注入窗口期间观察到行为异常的下游节点，用作路径可达性评分的目标。

在示例中，ts-station-service 在 t_0 变为 不可用，错误传播至 ts-travel-service（超时），随后传播至 ts-order-service（HIGH_ERROR_RATE），从而形成一条三跳路径。

智能体答案模式。智能体输出一个严格 JSON 格式的 AgentRCAOutput，其中包含两个扁平列表。root_causes 包含形式为 (service, fault_kind, evidence[]) 的条目，其中 fault_kind 取自受控枚举，且 evidence 非空。propagation 包含有向的 (from, to) 边，每条边也包含非空的 evidence。该模式在单故障和混合故障情形下保持统一；智能体根据证据支持的数量填充 root_causes 中的条目。

架构 代理 回答（AgentRCAOutput）

```
{ "root_causes": [{"服务": "ts-station-service", "故障_类型": "pod_不可用", "证据": [{"类型":
" 指标 ", "SQL": "SELECT 时间 , 值 FROM 异常 _ 指标 WHERE 服务 _ 名称 = '
ts-station-service' AND 指标 LIKE '%pod_阶段' ORDER BY 时间", "声明": "在整个窗口期间,
ts-station-service Pod 处于未就绪状态"}]} ,"传播": [{"来源": "ts-station-service", "目标": "
ts-travel-service", "证据": [{"类型": "追踪", "SQL": "SELECT 跨度_名称, attr.状态_码,
COUNT(*) FROM 异常_追踪 WHERE 服务_名称 = 'ts-travel-service' AND 跨度_名称 LIKE '
%车站%' GROUP BY 1,2", "声明": "ts-travel-service 对 ts-station-service 的调用返回超时 "}]}
]}
```

每个证据项声明：(i) 其模态（类型 $\in$ {metric,trace,log}），(ii) 其声称由案例 Parquet 文件支持的 DuckDB SQL，以及 (iii) 一个 $\leq$20 字的声明，说明这些行所证明的内容。合约要求每个根本原因和每个传播边至少携带一个证据项，并且该 SQL 必须可由评判者针对案例目录重新执行。服务名称在匹配时间进行标准化（小写化，去除连字符和下划线）以吸收命名风格漂移。

两种模式均输入评估流水线：从代理回答中提取（服务, 故障_类型）多重集和传播边集，而它们的对应项加上告警节点集则从真值图中读取。第G节定义了每轴比较。

F.4 提示词设计

代理运行两阶段提示词。调查阶段（RCA_ANALYSIS_SP/UP）允许模型通过工具调用自由查询案例 Parquet 文件，并固定目标和操作手册，而不规定输出 JSON。综合阶段（COMPRESS_FINDINGS_SP/UP）使用在格式化时从 SDK 拼接进规范合同块，将累积的工具消息转换为一个严格 JSON 的 AgentRCAOutput（第 F.3 节）。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

F.4.1 Investigation Phase

SYSTEM: RCA_ANALYSIS_SP

You are a Root Cause Analysis (RCA) expert investigating a microservices incident.

For context, today's date is {date}.

Goal

Identify the root cause(s) of the SLO violation from the telemetry. There may be more than one. Let the data tell you how many faults there are and where they sit.

Your job in this phase is INVESTIGATION ONLY. A separate synthesis step will turn your findings into the final structured output. Do not try to emit JSON or final answers here.

Available data

Parquets in this case directory:

- abnormal_metrics.parquet, abnormal_traces.parquet, abnormal_logs.parquet
- abnormal_metrics_histogram.parquet, abnormal_metrics_sum.parquet
- normal_metrics.parquet, normal_traces.parquet, normal_logs.parquet

Common columns:

- metrics: time, metric, value, service_name, attr.k8s.pod.name, attr.k8s.namespace.name
- metrics_sum: same shape; carries jvm.cpu.*, jvm.memory.*, jvm.thread.count, jvm.gc.*, container.cpu.*, container.memory.*
- metrics_histogram: time, metric, service_name, count, sum, min, max, attr.jvm.gc.action, attr.jvm.gc.name
- traces: time, trace_id, span_id, parent_span_id, span_name, service_name, duration, attr.status_code, attr.http.request.method, attr.http.response.status_code
- logs: time, trace_id, span_id, level, service_name, message

Tools

1. query_parquet_files: DuckDB SQL on parquets in this case dir.
2. list_tables_in_directory: list parquets.
3. get_schema: column types of a parquet.
4. think_tool: REQUIRED after each query; summarize, plan next step.

Hard limits

- Tool-call budget: aim for ~50 calls; extend if the evidence genuinely warrants it. Hard cap is 100, at which point the runtime forces a stop.
- Spend the budget efficiently: list_tables_in_directory once, get_schema on the files you actually plan to query, then spend the rest on query_parquet_files.

Investigation playbook

1. list_tables_in_directory to confirm the parquet files.
2. get_schema on the relevant ones (start with abnormal_traces).
3. Diff abnormal vs normal: error rates, latency, status codes, log levels.
4. Trace the call chain (parent_span_id → span_id) to find the earliest service whose own work, not its dependency's, went wrong.
5. Decide every root cause and every propagation edge. More than one root cause is possible; note each separately when evidence supports it.

USER: RCA_ANALYSIS_UP

{incident_description}

F.4.1 调查阶段

系统：根本原因分析 SP_ _

你是一名根因分析（RCA）专家，正在调查一起微服务事故。

目标
目标
从遥测数据中识别SLO违规的根本原因。可能存在多个。让数据告诉你有多少个故障以及它们位于何处。

你在此阶段的任务仅限于调查。一个独立的综合步骤会将你的发现转化为最终的结构化输出。不要尝试在此处输出 JSON 或最终答案。

可用数据
案例目录中的Parquet文件：
- 异常指标.Parquet, 异常_追踪.Parquet, 异常_日志.Parquet
- 异常_指标_直方图.parquet, 异常_指标_总和.parquet
- 正常_指标.Parquet，正常_追踪.Parquet，正常_日志.Parquet

公共列：
- 指标：时间、指标、值、服务名称、属性.k8s.pod名称、属性.k8s.命名空间名称_
- 指标_总和：形状相同；包含jvm.cpu.*、jvm.memory.*、jvm.线程计数、jvm.gc.*、容器.cpu.*、容器.memory.*
- 指标_直方图：时间，指标，服务_名称，计数，总和, 最小值, 最大值, 属性.jvm.gc.操作, 属性.jvm.gc.名称
- 追踪: 时间, 追踪_标识, 跨度_标识,父跨度__标识, 跨度_名称, 服务_名称, 持续时间, attr.status_code, 属性.http.请求方法, attr.http.response.status_code
- 日志: 时间, trace_标识, span_标识,级别, 服务_名称, 消息

工具
1. query_Parquet文件_：在本例目录中对Parquet文件执行DuckDB SQL。2. list_tables_in_directory：列出Parquet文件。3. get_schema：Parquet文件的列类型。4. think_tool：每次查询后必须使用；总结并规划下一步

.

硬性限制
- 工具调用预算：目标为 ~50 次调用；若证据确实需要，可予以扩展。硬性上限为100，达到该上限时运行时将强制停止。
- 高效利用预算：_在_目录中列出_表一次，针对实际计划查询的文件获取_架构，然后将剩余预算用于查询_Parquet_文件。

调查手册
1. list_tables_in_directory以确认parquet文件。2. get_schema on the relevant ones（从异常_traces开始）。3. 对比异常与正常：错误率、延迟、状态码、日志级别。4. Trace the call chain（parent_span_id → span_id）以找出其自身工作而非其依赖项出错的第一个服务。5. 确定每个根本原因和每条传播边。可能存在多个根本原因；当证据支持时，请分别列出每一个。

用户：根本原因分析 UP_ _

{事件_描述}

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

F.4.2 Synthesis Phase

SYSTEM: COMPRESS_FINDINGS_SP

```
You are an RCA synthesizer.  Today's date is {date}.
Your job:  convert the investigation messages above into a single STRICT JSON
object matching the RCA schema.  Output JSON only, with no markdown, commentary,
or prose.
{agent_contract}   (spliced canonical block; see "Agent Contract" below.)
## Filtering investigation messages
• Include:  query results showing anomalies, errors, or elevated latency.
• Exclude:  think_tool outputs (the agent's own reasoning notes).
```

USER: COMPRESS_FINDINGS_UP

```
## Incident
{incident_description}
Synthesize the investigation messages above into ONE JSON object as specified
in the system prompt.  No prefix, no suffix, no markdown fences, no explanation.
Begin output now.
```

F.4.3 Agent Contract (spliced into synthesis)

The synthesis prompt does not hand-maintain the output schema or the fault-kind enumeration. Both are pulled at agent-startup from the SDK, so the contract version follows the SDK version automatically. The block below is the canonical text spliced in via `{agent_contract}`; it states the JSON shape of `AgentRCAOutput` (Section F.3), enumerates the `fault_kind` vocabulary, and lists the field, SQL, and hard rules.

SPLICED: agent_contract

```
## Output schema (STRICT JSON, no markdown / commentary)
JSON shape of AgentRCAOutput as in Section F.3:  root_causes[] of {service,
fault_kind, evidence[]}, and propagation[] of {from, to, evidence[]}, where
every evidence is {kind, sql, claim}.
## fault_kind enum (pick exactly one per root_cause)
pod_failure, pod_unavailable, network_delay, network_loss, network_partition,
network_corrupt, network_duplicate, network_bandwidth_limit, http_aborted,
http_slow, http_payload_modified, http_response_status_modified, cpu_stress,
jvm_thread_cpu_stress, mem_stress, jvm_heap_stress, jvm_gc_pressure,
jvm_method_exception, jvm_jdbc_exception, jvm_method_latency, jvm_jdbc_latency,
jvm_method_mutated, dns_resolution_failed, dns_resolution_wrong, clock_skew
## Field rules
• Service names must match strings present in the data; do not invent.
• One entry per distinct root cause; do NOT collapse multiple distinct faults
into a single root_cause.
## SQL rules
• DuckDB on this case dir's parquets.
• SELECT only; one statement per sql field.  No DDL/DML/ATTACH/etc.
• Reference parquets via read_parquet('<file>.parquet') with bare filenames, OR
directly FROM abnormal_traces (each *.parquet is mounted as a same-named view).
• Each evidence.sql MUST be the actual SQL, not a description.
• Each evidence.claim is the ≤20-word natural-language assertion the SQL rows
back.
## Hard rules
• EVERY root_cause carries ≥1 evidence (real runnable SQL).
```


F.4.2 综合阶段

系统：压缩发现 SP_ _

```
你是一个根因分析合成器。今天的日期是 {date}。
将上述调查消息转换为一个符合 RCA 架构的严格JSON对象。仅输出 JSON，不含
Markdown、注释或正文。
{agent_contract}（拼接规范块；参见下方"智能体契约"。）
过滤调查消息
• 包含：显示异常、错误或延迟升高的查询结果 .
• 排除：think_工具输出（代理自身的推理笔记）。
```

压缩发现 UP_ _

```
事件
{事件_描述}
将上述调查消息合成为一个按系统提示词指定的JSON对象。无前缀，无后缀，无Markdown代
码块，无解释。现在开始输出。
```

F.4.3 智能体契约（拼接入合成）

综合提示词不会手动维护输出模式或故障类型枚举。两者均在智能体启动时从SDK获取，因此契约版本会自动跟随SDK版本。以下代码块是通过 {agent_contract} 拼接的规范文本；它声明了 AgentRCAOutput 的 JSON格式（第F.3节），列出了故障_kind 词汇表，并包含了字段、SQL和硬性规则。

SPLICED: 代理_contract

```
输出模式（严格JSON，无Markdown/注释）
AgentRCAOutput 的 JSON格式 如第 F.3 节所示：root_causes[] ，包含 {service, fault_kind,
evidence[]}，以及 propagation[]，包含 {来源, 目标, evidence[]}，其中每个 evidence 均为 {kind,
sql, claim}。
故障_类型 enum（每个根_因只选一个）
pod_failure, pod_unavailable, network_delay,network_loss, network_partition, network_
corrupt,network_duplicate, network_bandwidth_limit, http_aborted, http_slow, http_payload_
modified, http_response_status_modified, 中央处理器_stress, jvm_thread_中央处理器_stress, 内存_
stress, jvm_堆_stress, jvm_gc_pressure, jvm_方法_异常, jvm_jdbc_异常, jvm_方法_延迟, jvm_jdbc_延
迟, jvm_方法_mutated,域名系统_resolution_failed, 域名系统_resolution_wrong, 时钟_skew
字段规则
• 服务名称必须与数据中存在的字符串匹配；不得编造。
• 每个不同根本原因对应一个条目；切勿将多个不同故障合并为一个根本_原因。
SQL规则
• 在此案例目录的Parquet文件上使用DuckDB。
• 仅 SELECT；每个 SQL 字段一条语句。   No 数据定义语言/数据操纵语言/附件/etc.
通过 read_parquet('<file>.parquet') 引用 Parquet文件，仅使用文件名，或者
直接来自异常_追踪（每个*.Parquet文件均挂载为同名视图）。
• 每个证据SQL 都必须是实际的 SQL，而不是描述。
• 每个证据主张均为 ≤20词的自然语言声明，对应SQL行
返回
## 硬性规则
• 每个根_因都附有 ≥1证据（真实可执行SQL）。
```


翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

- EVERY propagation edge carries $\geq$1 evidence. Do not emit edges you cannot back with a concrete trace or metric query.

F.4.4 Key Design Choices

- **Investigation/synthesis separation.** The investigation prompt withholds the JSON schema so the model spends tokens on collecting evidence rather than packing partial answers. The synthesis prompt then receives the canonical contract block alongside the full tool history, eliminating schema drift between agent answer and judge.
- **Controlled fault-kind vocabulary.** The `fault_kind` enum is observable-effect-based: two chaos primitives whose telemetry is indistinguishable share a kind, and two whose signals are distinguishable get separate kinds. This lets the matcher score classification (*fault_kind_accuracy*) deterministically without an LLM-as-judge.
- **Mandatory SQL evidence.** Every claim, including both root causes and propagation edges, must cite at least one DuckDB-runnable SQL. The judge re-executes each SQL on the case directory; un-runnable evidence (*sql_executable_rate*) and rows that do not back the claim (*evidence_support_rate*, Section G) each surface as a scoreable axis.
- **No topology leakage.** The agent is *not* given the service dependency graph; service relations must be inferred from trace data, testing investigative ability rather than pattern matching over a known graph.
- **Paired normal/abnormal telemetry.** Both pre- and post-injection parquets ship in the case directory, so the agent can run differential queries (error rates, latency tails, log volumes) on identical schemas.

F.5 Model Configurations

We evaluate Claude Opus 4.7, Claude Sonnet 4.6, Gemini 3.1 Pro, MiMo 2.5 Pro, GLM 5.1, Kimi K2.6, DeepSeek V4 Pro, Qwen3.6-Max, Hy 3.0 Preview, Seed 2.0 Pro, and MiniMax M2.7 in the main results, plus GPT-5.4 and GPT-5.5 reported separately in Appendix H.3.
All models use identical system prompts, tool definitions, and temperature (0.7) for reproducibility. GPT-5.4 and GPT-5.5 have limited benchmark coverage and are reported separately in Appendix H.3.

G Evaluation Protocol and Metric Definitions

We organize our evaluation metrics into two layers: *outcome* (set classification of the predicted root-cause set against ground truth) and *process* (causal reasoning quality on the predicted graph plus evidence executability), complemented by *dataset difficulty* signals that characterize each instance.

Claim-to-Metric Mapping. Each headline claim in Section 3.2 is anchored to a specific metric defined below:

- *Outcome scores understate the failure; the lenient process check exposes ungrounded diagnoses* (Finding 1): measured by the share of cases satisfying AnySvc but not PR, and by the conditional rate PR/AnySvc in Table 2.
- *Edge F1 lags Node F1 universally* (Finding 2): measured by the per-row Node F1 vs. Edge F1 columns.
- *No single best model* (Finding 3): measured by the distribution of per-column extrema (bolded in Table 2) across distinct models.
- *Per-system difficulty is uneven* (Appendix Section H.1): measured by the spread of EM and PR across the TrainTicket / OTel Demo / Hotel Reservation rows of Appendix Table 8.

G.1 Outcome-Level Metrics

The outcome-level metrics grade only the agent's predicted root-cause set against the ground-truth root-cause set, ignoring the predicted causal graph. Let $\hat{R}_q$ denote the set of (service, fault_kind) pairs the agent emits as root causes for case q, R_q^* the corresponding ground-truth set, and $\mathcal{Q}$ the set of all evaluation queries. Service names are normalized as in Section G.2 before comparison; for network-class faults, both endpoints of the netem rule are accepted because the rule is observable on either side.

- 每一条传播边都承载 $\geq$1 证据。请勿输出无法用具体的追踪记录或指标查询加以支撑的边。

F.4.4 关键设计选择

调查/综合分离。调查提示词隐藏了JSON模式，从而使模型将令牌用于收集证据而非打包部分答案。综合提示词随后接收规范合同块以及完整的工具历史，消除了Agent 回答与评判者之间的模式漂移。

• 受控故障类型词汇表。故障_类型枚举基于可观测效应：两个遥测数据不可区分的混沌原语共享一个类型，而两个信号可区分的则分配不同的类型。这使得匹配器能够以确定性方式对分类（故障_类型_准确率）进行评价，而无需大语言模型评判。

• 强制SQL证据。每个声明，包括根本原因和传播边，都必须引用至少一个可执行SQL。评判者在案例目录上重新执行每条SQL；不可执行的证据（sql_executable_rate）和不支持该声明的行（evidence_support_rate，第G节）各自成为一个可计分的维度。

• 无拓扑泄露。代理不被提供服务依赖图；服务关系必须从追踪数据中推断，以测试调查能力，而非在已知图上的模式匹配。

• 配对正常/异常遥测。注入前后Parquet文件均包含在案例目录中，因此代理可以在相同的模式上运行差异查询（错误率、延迟尾部、日志量）。

F.5 模型配置

我们在主要结果中评估了Claude Opus 4.7、Claude Sonnet 4.6、Gemini 3.1 Pro、MiMo 2.5 Pro、GLM 5.1、Kimi K2.6、DeepSeek V4 Pro、Qwen3.6-Max、Hy 3.0 预览版、Seed 2.0 Pro和MiniMax M2.7，此外，GPT-5.4和GPT-5.5在附录H.3中单独报告。
全部模型使用相同的系统提示词、工具定义和温度（0.7），以确保可复现性。GPT-5.4和GPT-5.5的基准测试覆盖范围有限，分别在附录H.3中单独报告。

G 评估协议和指标定义

我们将评估指标划分为两个层面：结果层（将预测的根本原因集与真实值进行集合分类）和进程层（基于预测图的因果推理质量及证据可执行性），并辅以刻画每个实例的数据集难度信号。

主张到指标映射。第 3.2 节中的每个主要主张均锚定于下面定义的特定指标：

• 结果得分低估了故障的影响；宽松的检查暴露了无根据诊断（发现 1）：通过满足 AnySvc 但不满足 PR 的案例比例，以及表 2 中 PR 在 AnySvc 下的条件评价来衡量。

• 边 F1 分数普遍低于节点 F1 分数（发现 2）：通过每行的节点 F1 分数与边 F1 分数列进行衡量。

• 没有单一的最佳模型（发现 3）：通过测量跨不同模型的每列极值的分布（表2中加粗显示）来确定。

• 各系统难度不均（附录第H.1节）：通过附录表 8 中 TrainTicket / OTel 演示 / 酒店预订 行的精确匹配率和路径可达性指标的分布来衡量。

G.1 结果级指标

结果级指标仅根据代理预测的根本原因集与真值根本原因集进行评分，而忽略预测的因果图。令 R_q 表示代理为案例 q 发出的根本原因（服务、故障_类型）对的集合，R_q^* 表示对应的真值集，$\mathcal{Q}$ 表示全部评估查询的集合。在比较之前，服务名称按第G.2节所述进行标准化；对于网络类故障，由于 netem规则 在两侧均可观测，因此接受该规则的两个端点。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

AnySvc (any service hit, kind-agnostic).

$$\text{AnySvc} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\exists (\hat{s}, \hat{k}) \in \hat{R}_q,\ (s^*, k^*) \in R_q^* : \hat{s} = s^* \Big] \tag{4}$$

AnySvc is satisfied when at least one predicted root-cause service appears in the ground truth, irrespective of the predicted fault kind.

Precision, Recall, F1 on (service, fault_kind) pairs. For each case q, the per-case precision and recall on the pair set are

$$\mathrm{P}_q = \frac{|\hat{R}_q \cap R_q^*|}{|\hat{R}_q|}, \quad \mathrm{R}_q = \frac{|\hat{R}_q \cap R_q^*|}{|R_q^*|}, \quad \mathrm{F1}_q = \frac{2\,\mathrm{P}_q\,\mathrm{R}_q}{\mathrm{P}_q + \mathrm{R}_q} \tag{5}$$

where intersection uses HIT-and-equivalence semantics (a predicted pair counts as matched if there is a ground-truth pair with the same normalized service and an equivalent fault_kind in the equivalence vocabulary of Section C.3). We report the mean of P_q, R_q, and $\mathrm{F1}_q$ over $\mathcal{Q}$. Recall captures fractional coverage of the ground-truth pairs and is therefore a strictly finer measurement than AnySvc, which only checks whether at least one service is named.

Exact Match (EM).

$$\text{EM} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\hat{R}_q = R_q^* \Big] \tag{6}$$

EM requires the predicted root-cause set to coincide with the ground truth as sets, penalizing both missing and spurious entries.

G.2 Process-Level Metrics

These metrics evaluate the quality of the agent's causal reasoning by comparing the predicted CausalGraph against the verified causal subgraph $\mathcal{G}^*$. We decompose the evaluation into *primary* metrics (service-level, deterministic matching) and *secondary* metrics (component-level, with LLM-assisted semantic matching for name normalization).

Loadgen filtering. Edges incident on the synthetic load-generator endpoint are removed from both $\hat{\mathcal{E}}_q$ and $\mathcal{E}_q^*$ before any graph-shape metric (Edge F1, Node F1, Path Reachability) is computed. Loadgen is an evaluation-time client and not part of the system under test, so neither admitting nor omitting loadgen-touching edges should be scored.

G.2.1 Primary Metrics (Service-Level)

All primary metrics operate on service-level representations. Service names are normalized (lowercased, `ts-` prefix stripped, hyphens removed) before comparison.

Edge F1. Let $\hat{\mathcal{E}}$ and $\mathcal{E}^*$ be the service-level edge sets of the predicted and ground-truth causal graphs, respectively:

$$\text{Edge Precision} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\hat{\mathcal{E}}|}, \quad \text{Edge Recall} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\mathcal{E}^*|} \tag{7}$$

Edge F1 is the harmonic mean.

Node F1. Let $\hat{\mathcal{V}}$ and $\mathcal{V}^*$ be the service-level node sets:

$$\text{Node Precision} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\hat{\mathcal{V}}|}, \quad \text{Node Recall} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\mathcal{V}^*|} \tag{8}$$

Node F1 measures whether the agent correctly identified the set of affected services.

Boundary case. When both the predicted and ground-truth sets are empty for a metric (e.g., both graphs have zero edges), we assign a perfect score of 1.0, since the agent correctly predicted the absence of that element.

AnySvc（任意服务命中，类型无关）。

$$\text{AnySvc} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\exists (\hat{s}, \hat{k}) \in \hat{R}_q,\ (s^*, k^*) \in R_q^* : \hat{s} = s^* \Big] \tag{4}$$

只要至少一个预测根因服务出现在真实值中，无论预测的故障类型如何，AnySvc 即被视为满足。

（服务，故障_类型）配对上的精确率、召回率和 F1 分数。对于每个案例 q，在该配对集上的单例精确率和召回率为

$$\mathrm{P}_q = \frac{|\hat{R}_q \cap R_q^*|}{|\hat{R}_q|}, \quad \mathrm{R}_q = \frac{|\hat{R}_q \cap R_q^*|}{|R_q^*|}, \quad \mathrm{F1}_q = \frac{2\,\mathrm{P}_q\,\mathrm{R}_q}{\mathrm{P}_q + \mathrm{R}_q} \tag{5}$$

其中，交集采用命中及等价语义（若存在一个真实值对，其归一化服务相同且故障_类型在C.3节的等价词汇表中互为等价，则预测对计为匹配）。我们报告$_q$、$_q$、1_q 在 $\mathcal{Q}$上的平均值。召回率衡量真实值对的分数覆盖，因此比仅检查是否至少提及一个服务的AnySvc是一种更精细的度量。

精确匹配 (EM)

$$\text{EM} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\hat{R}_q = R_q^* \Big] \tag{6}$$

精确匹配率要求预测根本原因集与真实值在集合上完全一致，并对缺失条目和虚假条目进行惩罚。

G.2 过程级指标

这些指标通过将预测因果图与验证因果子图 $\mathcal{G}^*$ 进行比较，来评估代理因果推理的质量。我们将评估分解为主要指标（服务级、确定性匹配）和次要指标（组件级，并借助大语言模型进行语义匹配以实现名称归一化）。

负载生成器过滤。在计算任何图结构指标（边 F1 分数、节点 F1 分数、路径可达性）之前，从 $\mathcal{E}_q$ 和 $\mathcal{E}_q^*$ 中移除连接合成负载生成器端点的边。负载生成器是评估时客户端，而非被测系统的一部分，因此无论是否包含连接负载生成器的边，均不应进行评分。

G.2.1 主要指标（服务级）

所有主要指标均在服务级表示上运行。在比较之前，服务名称会被标准化（转换为小写、去除 ts-前缀、移除连字符）。

边 F1 分数。令 $\mathcal{E}$ 和 $\mathcal{E}^*$ 分别为预测因果图和真实值因果图的服务级边集：

$$\text{Edge Precision} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\hat{\mathcal{E}}|}, \quad \text{Edge Recall} = \frac{|\hat{\mathcal{E}} \cap \mathcal{E}^*|}{|\mathcal{E}^*|} \tag{7}$$

边 F1 分数是调和平均数。

节点 F1 分数。设 $\mathcal{V}$ 和 $\mathcal{V}^*$ 为服务级节点集：

$$\text{Node Precision} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\hat{\mathcal{V}}|}, \quad \text{Node Recall} = \frac{|\hat{\mathcal{V}} \cap \mathcal{V}^*|}{|\mathcal{V}^*|} \tag{8}$$

节点 F1 分数用于衡量代理是否正确识别了受影响的服务集合。

边界情况。当某个指标的预测集和真实值集合均为空时（例如，两个图的边数均为零），我们赋予满分1.0，因为代理正确预测了该元素的缺失。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

G.2.2 SQL Exec

The agent answer schema (Section F.3) ties every claim to at least one piece of SQL evidence. Every `evidence.sql` in the agent answer is re-executed on a fresh in-memory DuckDB connection with each `*.parquet` of the case directory mounted as a same-named view. The result is one of three statuses: `OK` (executed, ≥ 1 row), `EMPTY` (executed, 0 rows), or `SQL_ERROR` (parse or runtime failure). Per case,

$$\text{SQL Exec} = \frac{|\{e \in \mathcal{E}_{\text{ev},q} : \text{status}(e) = \texttt{OK}\}|}{|\mathcal{E}_{\text{ev},q}|} \tag{9}$$

where $\mathcal{E}_{\text{ev},q}$ is the union of all evidence items in q's root_causes and propagation edges. This axis is mechanical; whether the rows actually back the claim is not scored here.

G.2.3 Diagnostic Reasoning Axes

Path Reachability (PR). Path reachability measures the fraction of all instances where the agent both names a correct root-cause service *and* constructs at least one valid directed path from that service to a ground-truth alarm node in its predicted graph:

$$\text{PR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\text{AnySvc}_q = 1 \ \wedge \ \exists \text{path } \hat{r}_q \rightsquigarrow \hat{s}_q \text{ in } \hat{\mathcal{G}}_q \mid \hat{r}_q \in R_q^*,\, \hat{s}_q \in A_q^*\Big] \tag{10}$$

where R_q^* and A_q^* are the ground-truth root cause and alarm-node sets for instance q, and $\text{AnySvc}_q = 1$ when at least one of the agent's predicted root-cause services lies in R_q^* (kind-agnostic). Anchoring on AnySvc rather than on a kind-correct hit lets PR vary independently from the fault-kind-tagged Recall: PR isolates whether the agent constructed a faithful chain, while Recall captures whether it also labelled the fault kind correctly. Cases where the agent named no correct service receive PR $= 0$ by definition, since no service-correct anchor exists for the path. PR is therefore strictly upper-bounded by AnySvc, and the share of AnySvc cases failing PR quantifies how often a correctly localized service lacks even a single actionable propagation path.

HIT-anchored variant. For backward comparison we also report a HIT-anchored variant of PR (Appendix Table 9), which restricts the anchor to root-cause claims that match both service *and* fault kind. The HIT-anchored variant is strictly upper-bounded by Recall and is what the SDK stores by default; the AnySvc-anchored variant in the main table is computed locally from the same trajectories.

H Auxiliary Evaluation Tables

H.1 Per-system breakdown

Table 8 expands the pooled main-paper table (Table 2) into per-system rows for TrainTicket, OTel Demo, and Hotel Reservation. The pooled **All** row of each model matches the corresponding row in the main table.

The per-system spread is wide enough to dominate the cross-model spread on the easier metrics. On Hotel Reservation (142 cases, 9 services, mean longest path 2.1 hops) Opus reaches 99.3% AnySvc and 54.9% EM, while on TrainTicket (320 cases, 44 services, mean longest path 3.0 hops) the same model drops to 75.3% AnySvc and 15.6% EM; OpenTelemetry Demo (38 cases, 15 services, 2.5 hops) sits between, at 81.6% AnySvc and 26.3% EM. Both service count and chain depth track the difficulty ordering in the same direction. Finding (2) in Section 3.2 suggests path depth is the load-bearing factor: Edge F1 trails Node F1 by $\sim$19 pp pooled, and the deficit compounds along longer paths. Pooled scores across systems with disparate chain depth are therefore sensitive to system-mix manipulation; the per-system rows below are the direct read for any narrow-regime comparison.

H.2 HIT-anchored Path Reachability

Table 9 reports PR under the stricter HIT anchor (the SDK default): the agent must match both service *and* fault kind on the anchoring root-cause claim before the path is checked. The HIT-anchored variant is what the per-case `result.json` stores; the main-table PR (Table 2) relaxes the anchor to AnySvc so PR varies independently from fault-kind labelling.

G.2.2 SQL 执行

智能体答案模式（第 F.3 节）将每一条声明与至少一条 SQL证据关联。在每次执行时，智能体答案中的每个 证据SQL 都在一个全新的 内存中的DuckDB连接 上重新执行，并将案例目录下的每个 *.parquet 文件挂载为 同名视图。执行结果为三种状态之一：成功（已执行，≥ 1 行）、空结果（已执行，0 行）或 SQL_错误（解析或运行时 故障）。对于每个案例，

$$\text{SQL Exec} = \frac{|\{e \in \mathcal{E}_{\text{ev},q} : \text{status}(e) = \texttt{OK}\}|}{|\mathcal{E}_{\text{ev},q}|} \tag{9}$$

其中 $\mathcal{E}_{\text{ev}}$ 和 $_q$ 是 q 的根本原因和传播边中全部证据项的并集。该维度是机械性的；行数据是否实际支持该声明并不在此处评分。

G.2.3 诊断推理维度

路径可达性 (PR)。路径可达性衡量在所有实例中，代理既命名了正确的根因服务，又在其预测图中构建了至少一条从该服务到真实告警节点的有效有向路径的比例：

$$\text{PR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{1}\Big[\text{AnySvc}_q = 1 \ \wedge \ \exists \text{path } \hat{r}_q \rightsquigarrow \hat{s}_q \text{ in } \hat{\mathcal{G}}_q \mid \hat{r}_q \in R_q^*,\, \hat{s}_q \in A_q^*\Big] \tag{10}$$

其中 R_q^* 和 A_q^* 分别是实例 q 的真实根因集合和告警节点集合，当代理的预测根因服务中至少有一个位于 R_q^* 时，AnySvc 为真（类型无关）。以 AnySvc 而非类型正确命中作为锚点，使得 PR 能够独立于故障类型标记召回率进行变化：PR 单独衡量代理是否构建了忠实链条，而召回率则捕捉其是否也正确标记了故障类型。当代理未命名任何正确服务时，根据定义其 PR 为 $= 0$，因为不存在用于该路径的服务正确锚点。因此，PR 严格受 AnySvc 的上限约束，而 AnySvc 案例中未通过 PR 的比例量化了正确定位的服务甚至缺乏单条可操作传播路径的频率。

命中锚定变体。为便于与以往结果进行对比，我们还报告了 PR 的命中锚定变体（见附录表9），该变体将锚点限制为同时匹配服务和故障类型的根因声明。命中锚定变体严格以上限为召回率，并且是 SDK 默认存储的变体；主表中的任意服务命中锚定变体是直接从相同轨迹本地计算得到的。

H 辅助评估表

H.1 按系统细分

表8将汇总主论文表格（表2）扩展为TrainTicket、OTel 演示和酒店预订的按系统行。每个模型的汇总全部行与主表中的对应行一致。

系统间差异的幅度足够大，以至于在较容易的指标上主导了模型间差异。在酒店预订（142 个案例，9 个服务，平均最长路径 2.1 跳）上，Opus模型达到 99.3% AnySvc 和 54.9% 精确匹配率，而在 TrainTicket（320 个案例，44 个服务，平均最长路径 3.0 跳）上，同一模型下降至 75.3% AnySvc 和 15.6% 精确匹配率；OpenTelemetry演示（38 个案例，15 个服务，2.5 跳）介于两者之间，为 81.6% AnySvc 和 26.3% 精确匹配率。服务数量和链条深度都以相同方向反映了难度排序。第 3.2 节中的发现 (2) 表明路径深度是关键因素：池化的边 F1 分数比节点 F1 分数低 $\sim$19 个百分点，且这一差距在更长的路径上进一步加剧。因此，在链条深度差异巨大的系统之间进行池化分数汇总，容易受到系统混合操纵的影响；下面的分系统行数据是进行任何窄域比较的直接依据。

H.2 HIT锚定路径可达性

表9报告了更严格的HIT锚点（SDK默认设置）下的路径可达性指标：代理必须在锚定根本原因断言中同时匹配服务和故障类型，然后才能检查路径。命中锚定变体存储在逐案例的result.json中；而主表路径可达率（表2）将锚点放宽至AnySvc，从而使路径可达性指标独立于故障类型标注而变化。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 8: Per-system breakdown of Table 2 (TrainTicket, OTel Demo, Hotel Reservation; pooled aggregate in the **All** row; percentages).

		Outcome					*Process*			
Model	System	EM↑	F1↑	Precision↑	Recall↑	AnySvc↑	PR↑	Node F1↑	Edge F1↑	SQL Exec↑
Claude Opus 4.7	TrainTicket	15.6	22.3	23.4	22.7	75.3	55.9	58.9	36.3	98.7
	OTel Demo	26.3	50.5	51.8	52.6	81.6	81.6	77.4	66.5	98.7
	Hotel Res.	54.9	81.9	94.4	76.1	99.3	98.6	84.2	74.7	94.4
	All	**27.6**	**41.4**	**45.7**	**40.1**	**82.6**	**70.0**	**67.5**	**49.5**	**97.5**
Gemini 3.1 Pro	TrainTicket	18.1	25.2	28.0	24.1	75.0	57.8	59.8	27.1	98.7
	OTel Demo	36.8	57.5	67.1	52.6	81.6	76.3	77.3	31.9	99.0
	Hotel Res.	52.8	82.2	96.5	75.0	97.9	97.2	83.8	60.0	98.9
	All	**29.4**	**43.8**	**50.4**	**40.7**	**82.0**	**70.4**	**67.9**	**36.8**	**98.8**
Claude Sonnet 4.6	TrainTicket	15.0	20.8	21.9	20.9	70.3	30.0	55.1	34.1	96.8
	OTel Demo	34.2	57.6	61.4	57.9	89.5	81.6	72.5	62.1	96.6
	Hotel Res.	49.3	80.2	95.1	72.9	97.2	95.8	83.7	75.4	92.8
	All	**26.2**	**40.5**	**45.7**	**38.5**	**79.4**	**52.6**	**64.5**	**48.0**	**95.6**
MiMo 2.5 Pro	TrainTicket	11.2	16.8	17.0	18.0	68.8	42.2	56.0	32.1	93.5
	OTel Demo	31.6	47.5	50.0	47.4	86.8	76.3	71.7	56.0	94.4
	Hotel Res.	46.5	75.8	89.4	69.4	95.8	94.4	81.0	70.5	88.1
	All	**22.8**	**35.9**	**40.1**	**34.8**	**77.8**	**59.6**	**64.3**	**44.9**	**92.0**
GLM 5.1	TrainTicket	9.4	14.7	15.5	15.0	70.9	56.9	59.8	38.6	93.9
	OTel Demo	21.1	39.6	42.1	39.5	84.2	78.9	77.1	68.7	94.3
	Hotel Res.	49.3	75.9	88.7	69.7	95.8	95.8	80.3	71.9	92.2
	All	**21.6**	**34.0**	**38.3**	**32.4**	**79.0**	**69.6**	**66.9**	**50.4**	**93.4**
Kimi K2.6	TrainTicket	10.6	15.3	15.7	15.9	61.6	50.0	49.3	30.8	78.8
	OTel Demo	18.4	41.9	43.9	44.7	68.4	65.8	61.0	51.5	72.9
	Hotel Res.	45.1	73.9	88.0	66.9	92.3	92.3	79.6	72.1	93.0
	All	**21.0**	**34.0**	**38.4**	**32.6**	**70.8**	**63.2**	**58.8**	**44.1**	**82.4**
DeepSeek V4 Pro	TrainTicket	10.9	14.1	15.0	13.9	54.1	35.6	42.1	25.8	70.1
	OTel Demo	23.7	39.4	42.5	38.2	65.8	57.9	58.0	46.5	68.7
	Hotel Res.	40.8	70.3	84.9	63.0	91.5	90.8	78.1	70.5	87.4
	All	**20.4**	**32.0**	**36.9**	**29.7**	**65.6**	**53.0**	**53.5**	**40.1**	**74.9**
Qwen3.6-Max	TrainTicket	9.4	17.2	17.3	19.1	70.6	60.3	56.1	36.7	93.8
	OTel Demo	21.1	47.4	55.3	44.7	86.8	76.3	77.1	67.5	89.4
	Hotel Res.	40.8	74.4	88.8	68.0	96.5	96.5	82.0	69.8	92.0
	All	**19.2**	**35.7**	**40.5**	**34.9**	**79.2**	**71.8**	**65.1**	**48.5**	**92.9**
Hy 3.0 Preview	TrainTicket	6.2	11.7	11.0	14.4	60.6	34.7	43.2	22.9	89.5
	OTel Demo	21.1	37.7	42.1	36.8	63.2	57.9	69.9	50.3	98.0
	Hotel Res.	41.5	69.4	81.5	64.1	96.5	94.4	80.6	69.8	96.6
	All	**17.4**	**30.1**	**33.4**	**30.2**	**71.0**	**53.4**	**55.9**	**38.3**	**92.1**
Seed 2.0 Pro	TrainTicket	8.8	14.3	14.1	16.1	72.5	53.8	57.3	35.7	89.8
	OTel Demo	10.5	24.2	26.6	25.0	73.7	71.1	72.9	63.3	87.2
	Hotel Res.	29.6	55.9	67.1	50.7	93.7	93.7	78.7	70.5	93.3
	All	**14.8**	**26.8**	**30.1**	**26.6**	**78.6**	**66.4**	**64.5**	**47.7**	**90.6**
MiniMax M2.7	TrainTicket	1.9	8.3	7.1	11.4	64.7	33.1	46.9	21.2	86.6
	OTel Demo	2.6	15.4	14.5	18.4	52.6	39.5	62.2	43.6	88.3
	Hotel Res.	19.0	49.7	49.5	54.6	85.9	78.2	72.5	43.4	87.3
	All	**6.8**	**20.6**	**19.7**	**24.2**	**69.8**	**46.4**	**55.3**	**29.2**	**86.9**

H.3 Limited-coverage models

GPT-5.4 and GPT-5.5 hit API budget limits before completing the full benchmark, so their sample sizes are smaller than the main-table models. We include their partial results below for reference only; they should not be compared directly to Table 2.

表8：表2的按系统细分（TrainTicket、OTel 演示、酒店预订；All行的汇总聚合；百分比）。

		结果层					进程			
模型	系统	EM↑	F1↑	精确率↑	召回率↑	AnySvc↑	PR↑	节点 F1↑	边 F1↑	SQL 执行↑
Claude Opus 4.7	TrainTicket	15.6	22.3	23.4	22.7	75.3	55.9	58.9	36.3	98.7
	OTel 演示	26.3	50.5	51.8	52.6	81.6	81.6	77.4	66.5	98.7
	酒店预订	54.9	81.9	94.4	76.1	99.3	98.6	84.2	74.7	94.4
	All	**27.6**	**41.4**	**45.7**	**40.1**	**82.6**	**70.0**	**67.5**	**49.5**	**97.5**
Gemini 3.1 Pro	TrainTicket	18.1	25.2	28.0	24.1	75.0	57.8	59.8	27.1	98.7
	OTel 演示	36.8	57.5	67.1	52.6	81.6	76.3	77.3	31.9	99.0
	酒店预订	52.8	82.2	96.5	75.0	97.9	97.2	83.8	60.0	98.9
	All	**29.4**	**43.8**	**50.4**	**40.7**	**82.0**	**70.4**	**67.9**	**36.8**	**98.8**
Claude Sonnet 4.6	TrainTicket	15.0	20.8	21.9	20.9	70.3	30.0	55.1	34.1	96.8
	OTel 演示	34.2	57.6	61.4	57.9	89.5	81.6	72.5	62.1	96.6
	酒店预订	49.3	80.2	95.1	72.9	97.2	95.8	83.7	75.4	92.8
	All	**26.2**	**40.5**	**45.7**	**38.5**	**79.4**	**52.6**	**64.5**	**48.0**	**95.6**
MiMo 2.5 Pro	TrainTicket	11.2	16.8	17.0	18.0	68.8	42.2	56.0	32.1	93.5
	OTel 演示	31.6	47.5	50.0	47.4	86.8	76.3	71.7	56.0	94.4
	酒店预订	46.5	75.8	89.4	69.4	95.8	94.4	81.0	70.5	88.1
	All	**22.8**	**35.9**	**40.1**	**34.8**	**77.8**	**59.6**	**64.3**	**44.9**	**92.0**
GLM 5.1	TrainTicket	9.4	14.7	15.5	15.0	70.9	56.9	59.8	38.6	93.9
	OTel 演示	21.1	39.6	42.1	39.5	84.2	78.9	77.1	68.7	94.3
	酒店预订	49.3	75.9	88.7	69.7	95.8	95.8	80.3	71.9	92.2
	All	**21.6**	**34.0**	**38.3**	**32.4**	**79.0**	**69.6**	**66.9**	**50.4**	**93.4**
Kimi K2.6	TrainTicket	10.6	15.3	15.7	15.9	61.6	50.0	49.3	30.8	78.8
	OTel 演示	18.4	41.9	43.9	44.7	68.4	65.8	61.0	51.5	72.9
	酒店预订	45.1	73.9	88.0	66.9	92.3	92.3	79.6	72.1	93.0
	All	**21.0**	**34.0**	**38.4**	**32.6**	**70.8**	**63.2**	**58.8**	**44.1**	**82.4**
DeepSeek V4 Pro	TrainTicket	10.9	14.1	15.0	13.9	54.1	35.6	42.1	25.8	70.1
	OTel 演示	23.7	39.4	42.5	38.2	65.8	57.9	58.0	46.5	68.7
	酒店预订	40.8	70.3	84.9	63.0	91.5	90.8	78.1	70.5	87.4
	All	**20.4**	**32.0**	**36.9**	**29.7**	**65.6**	**53.0**	**53.5**	**40.1**	**74.9**
Qwen3.6-Max	TrainTicket	9.4	17.2	17.3	19.1	70.6	60.3	56.1	36.7	93.8
	OTel 演示	21.1	47.4	55.3	44.7	86.8	76.3	77.1	67.5	89.4
	酒店预订	40.8	74.4	88.8	68.0	96.5	96.5	82.0	69.8	92.0
	All	**19.2**	**35.7**	**40.5**	**34.9**	**79.2**	**71.8**	**65.1**	**48.5**	**92.9**
Hy 3.0 预览版	TrainTicket	6.2	11.7	11.0	14.4	60.6	34.7	43.2	22.9	89.5
	OTel 演示	21.1	37.7	42.1	36.8	63.2	57.9	69.9	50.3	98.0
	酒店预订	41.5	69.4	81.5	64.1	96.5	94.4	80.6	69.8	96.6
	All	**17.4**	**30.1**	**33.4**	**30.2**	**71.0**	**53.4**	**55.9**	**38.3**	**92.1**
Seed 2.0 Pro	TrainTicket	8.8	14.3	14.1	16.1	72.5	53.8	57.3	35.7	89.8
	OTel 演示	10.5	24.2	26.6	25.0	73.7	71.1	72.9	63.3	87.2
	酒店预订	29.6	55.9	67.1	50.7	93.7	93.7	78.7	70.5	93.3
	All	**14.8**	**26.8**	**30.1**	**26.6**	**78.6**	**66.4**	**64.5**	**47.7**	**90.6**
MiniMax M2.7	TrainTicket	1.9	8.3	7.1	11.4	64.7	33.1	46.9	21.2	86.6
	OTel 演示	2.6	15.4	14.5	18.4	52.6	39.5	62.2	43.6	88.3
	酒店预订	19.0	49.7	49.5	54.6	85.9	78.2	72.5	43.4	87.3
	All	**6.8**	**20.6**	**19.7**	**24.2**	**69.8**	**46.4**	**55.3**	**29.2**	**86.9**

H.3 有限覆盖模型

GPT-5.4 和 GPT-5.5 在完成完整基准测试之前触发了 API 预算限制，因此它们的样本量小于主表模型。我们在下方列出其部分结果仅供参考；不应将这些结果与表2中的结果直接比较。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 9: HIT-anchored Path Reachability (PR^{HIT}) per (model, system). Strictly upper-bounded by Recall. Cf. Table 2 for the AnySvc-anchored PR used in the main table.

Model	TrainTicket	OTel Demo	Hotel Res.	**All**
Claude Opus 4.7	22.2	60.5	94.4	**45.6**
Gemini 3.1 Pro	24.4	60.5	97.2	**47.8**
Claude Sonnet 4.6	16.9	57.9	94.4	**42.0**
MiMo 2.5 Pro	16.6	44.7	90.8	**39.8**
GLM 5.1	16.2	44.7	89.4	**39.2**
Kimi K2.6	15.3	39.5	88.7	**38.0**
DeepSeek V4 Pro	13.4	39.5	85.2	**35.8**
Qwen3.6-Max	20.6	60.5	91.5	**43.8**
Hy 3.0 Preview	12.2	39.5	83.8	**34.6**
Seed 2.0 Pro	16.2	34.2	70.4	**33.0**
MiniMax M2.7	7.8	15.8	70.4	**26.2**

Table 10: GPT models with partial coverage due to budget limits (pooled across systems; for reference only).

Model	n	EM↑	F1↑	AnySvc↑	PR↑	Node F1↑	Edge F1↑
GPT-5.4	114	18.4	22.5	85.1	58.8	68.1	45.8
GPT-5.5	32	15.6	19.8	90.6	87.5	70.9	53.3

表9：按（模型，系统）分组的基于HIT锚点的路径可达性（PR^{HIT}）。其值严格受召回率上限约束。主表中使用的基于AnySvc锚点的可达性参见表2。

模型	TrainTicket	OTel 演示	酒店预订	**All**
Claude Opus 4.7	22.2	60.5	94.4	**45.6**
Gemini 3.1 Pro	24.4	60.5	97.2	**47.8**
Claude Sonnet 4.6	16.9	57.9	94.4	**42.0**
MiMo 2.5 Pro	16.6	44.7	90.8	**39.8**
GLM 5.1	16.2	44.7	89.4	**39.2**
Kimi K2.6	15.3	39.5	88.7	**38.0**
DeepSeek V4 Pro	13.4	39.5	85.2	**35.8**
Qwen3.6-Max	20.6	60.5	91.5	**43.8**
Hy 3.0 预览版	12.2	39.5	83.8	**34.6**
Seed 2.0 Pro	16.2	34.2	70.4	**33.0**
MiniMax M2.7	7.8	15.8	70.4	**26.2**

表 10：因预算限制而仅实现部分覆盖的 GPT 模型（跨系统池化；仅供参考）。

模型	n	EM↑	F1↑	AnySvc↑	PR↑	节点 F1↑	边 F1↑
GPT-5.4	114	18.4	22.5	85.1	58.8	68.1	45.8
GPT-5.5	32	15.6	19.8	90.6	87.5	70.9	53.3

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next